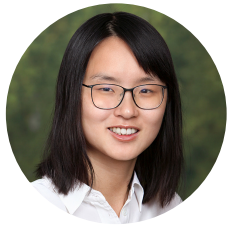
Generating Natural Language Proofs with Verifier-Guided Search

Kaiyu Yang, Jia Deng, and Danqi Chen



PRINCETON UNIVERSITY

PRINCETON NLP

Deductive Reasoning in Natural Language

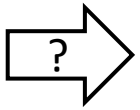
homes are buildings

energy is used for heating buildings

solar is a kind of energy

solar is renewable

Assumptions



solar is a kind of renewable energy for heating homes

Conclusion

Deductive Reasoning in Natural Language

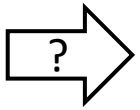
homes are buildings

energy is used for heating buildings

solar is a kind of energy

solar is renewable

Assumptions



solar is a kind of renewable energy for heating homes

Conclusion

- Studied extensively in formal theorem proving. **But why natural language?**

Deductive Reasoning in Natural Language

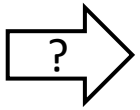
homes are buildings

energy is used for heating buildings

solar is a kind of energy

solar is renewable

Assumptions



solar is a kind of renewable energy for heating homes

Conclusion

- Studied extensively in formal theorem proving. But why natural language?
- **Natural language is a suitable vehicle for reasoning**
  - Expressive and flexible enough for representing all human knowledge
  - Scalable, no manual formalization needed
  - Interface with humans

Reasoning Challenges Posed by Natural Language

- Fuzzy, imprecise, requiring implicit knowledge
- No finite, well-defined inference rules

Plausible Solution: Semantic Parsing

- First convert natural language to formal logic; then apply a theorem prover

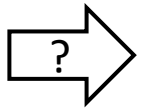
[Mineshima et al. EMNLP 2015]

homes are buildings

energy is used for heating buildings

solar is a kind of energy

solar is renewable



solar is a kind of renewable energy for heating homes

Plausible Solution: Semantic Parsing

- First convert natural language to formal logic; then apply a theorem prover

[Mineshima et al. EMNLP 2015]

$building(Home)$

$\forall x\ \forall y, energy(x) \wedge building(y) \rightarrow heats(x, y)$

$energy(Solar)$

$renewable(Solar)$

Theorem prover

?

$renewable(Solar) \wedge energy(Solar)$
$\wedge\ heats(Solar, Home)$

Plausible Solution: Semantic Parsing

- First convert natural language to formal logic; then apply a theorem prover

[Mineshima et al. EMNLP 2015]

$building(Home)$

$\forall x\ \forall y, energy(x) \wedge building(y) \rightarrow heats(x, y)$

$energy(Solar)$

$renewable(Solar)$

Theorem prover

?

$renewable(Solar) \wedge energy(Solar)$
$\wedge\ heats(Solar, Home)$

- **Need an ontology to capture the entire natural language: infeasible**

Reasoning with Large Language Models

- "Soft" reasoning over natural languages
- No formal representations or symbol manipulation

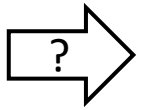
[Clark et al. IJCAI 2020]

homes are buildings

energy is used for heating buildings

solar is a kind of energy

solar is renewable



solar is a kind of renewable energy for heating homes

Reasoning with Large Language Models

- "Soft" reasoning over natural languages
- No formal representations or symbol manipulation

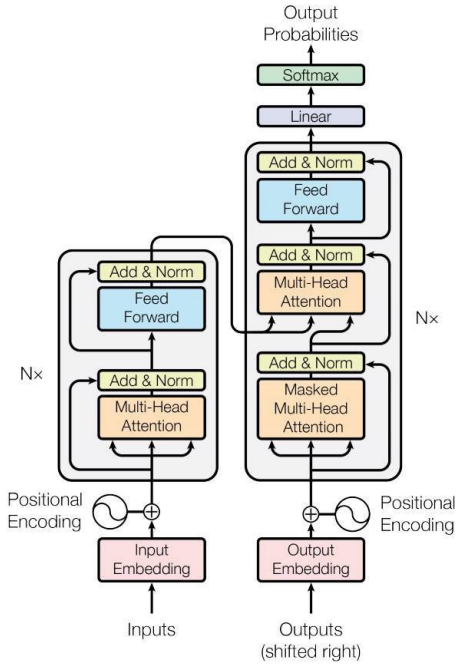
[Clark et al. IJCAI 2020]

homes are buildings

energy is used for heating buildings

solar is a kind of energy

solar is renewable



[Vaswani et al. NeurIPS 2017]

solar is a kind of renewable energy for heating homes

Task Formulation of Proof Generation

[Dalvi et al. EMNLP 2021]

Hypothesis (h):

h: solar is a kind of renewable energy for heating homes

Supporting facts (C):

sent1: homes are buildings
sent2: solar is renewable
sent3: wind is a kind of energy
sent4: solar is a kind of energy
sent5: energy is used for heating buildings
sent6: coal is nonrenewable
…
…

Input

Task Formulation of Proof Generation

[Dalvi et al. EMNLP 2021]

Hypothesis (h):

h: solar is a kind of renewable energy for heating homes

Supporting facts (C):

sent1: homes are buildings
sent2: solar is renewable
sent3: wind is a kind of energy
sent4: solar is a kind of energy
sent5: energy is used for heating buildings
sent6: coal is nonrenewable
…
…

Input

Task Formulation of Proof Generation

[Dalvi et al. EMNLP 2021]

Hypothesis (h):

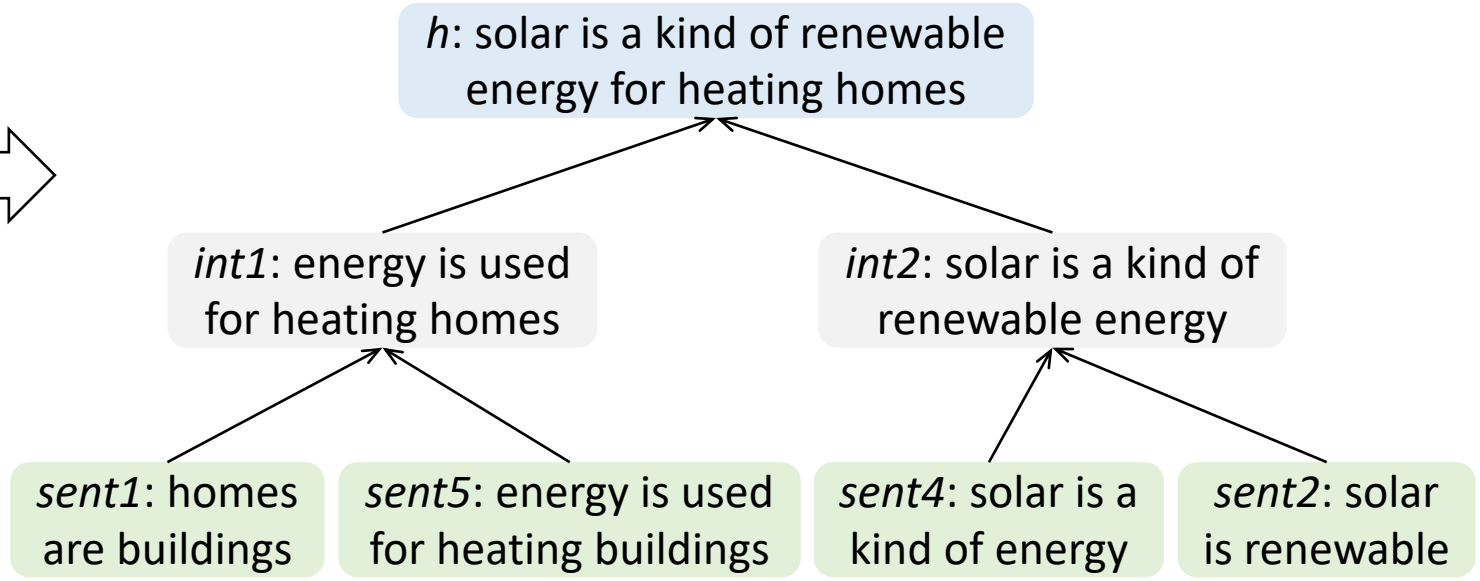
h: solar is a kind of renewable energy for heating homes

Supporting facts (C):

sent1: homes are buildings
sent2: solar is renewable
sent3: wind is a kind of energy
sent4: solar is a kind of energy
sent5: energy is used for heating buildings
sent6: coal is nonrenewable
…
…

Input

Proof tree (T):



Output

Task Formulation of Proof Generation

[Dalvi et al. EMNLP 2021]

Hypothesis (h):

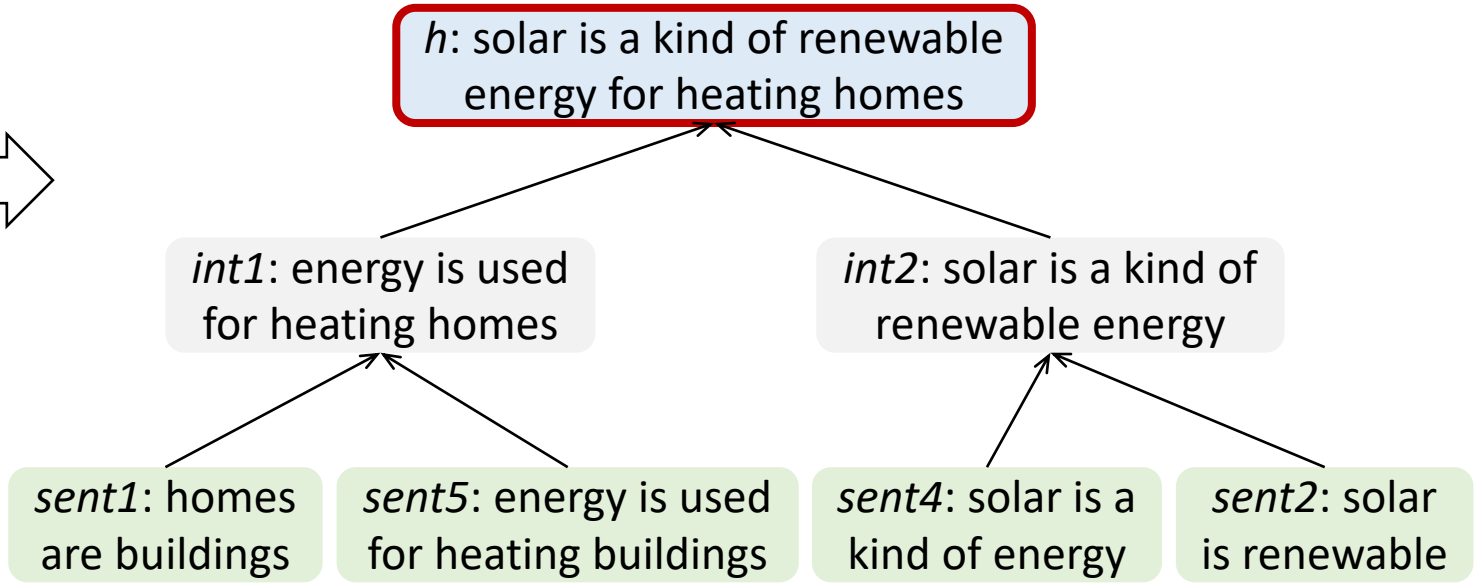
h: solar is a kind of renewable energy for heating homes

Supporting facts (C):

sent1: homes are buildings
sent2: solar is renewable
sent3: wind is a kind of energy
sent4: solar is a kind of energy
sent5: energy is used for heating buildings
sent6: coal is nonrenewable
…
…

Input

Proof tree (T):



Output

Task Formulation of Proof Generation

[Dalvi et al. EMNLP 2021]

Hypothesis (h):

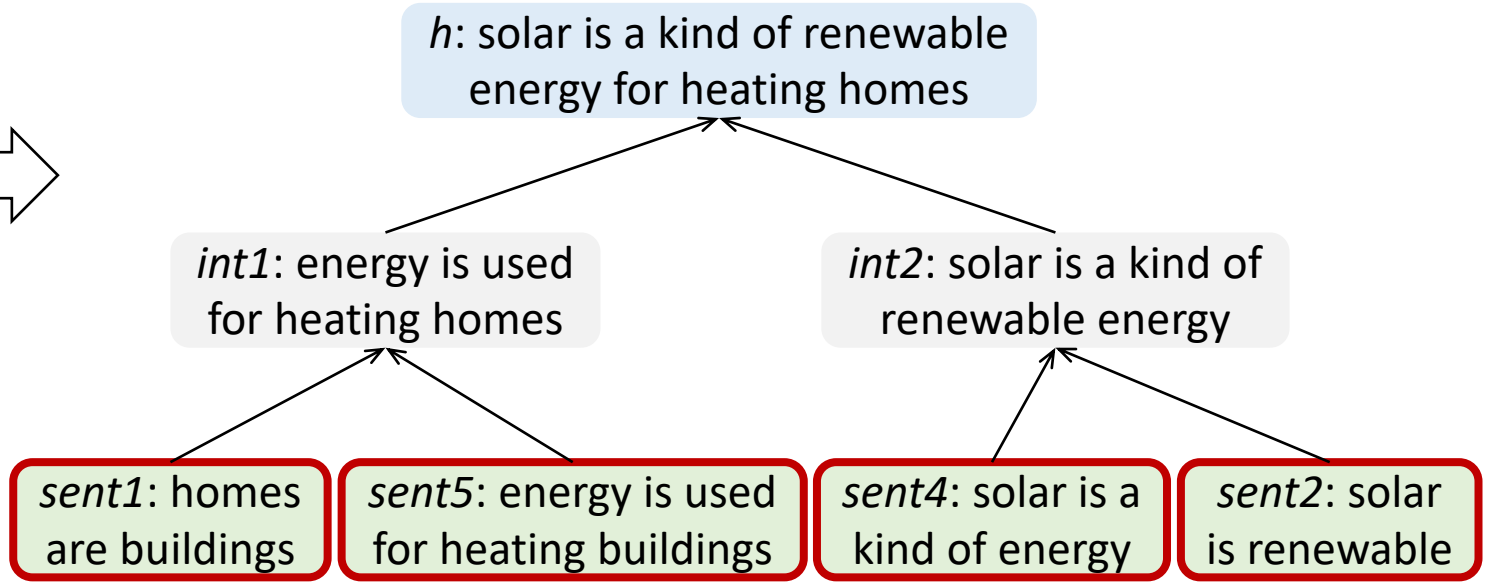
h: solar is a kind of renewable energy for heating homes

Supporting facts (C):

sent1: homes are buildings
sent2: solar is renewable
sent3: wind is a kind of energy
sent4: solar is a kind of energy
sent5: energy is used for heating buildings
sent6: coal is nonrenewable
…
…

Input

Proof tree (T):



Output

Task Formulation of Proof Generation

[Dalvi et al. EMNLP 2021]

Hypothesis (h):

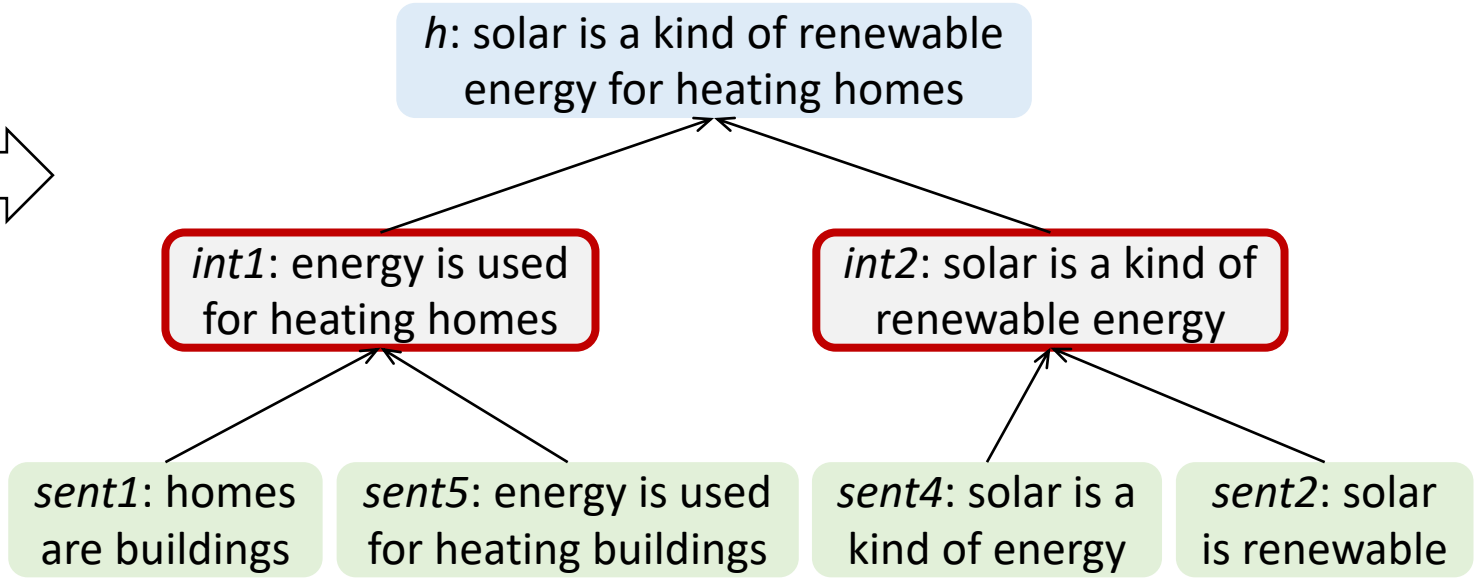
h: solar is a kind of renewable energy for heating homes

Supporting facts (C):

sent1: homes are buildings
sent2: solar is renewable
sent3: wind is a kind of energy
sent4: solar is a kind of energy
sent5: energy is used for heating buildings
sent6: coal is nonrenewable
…
…

Input

Proof tree (T):



Output

Single-Shot Methods

[Saha et al. EMNLP 2020]
[Dalvi et al. EMNLP 2021]

Generate the entire proof altogether

Hypothesis (h):

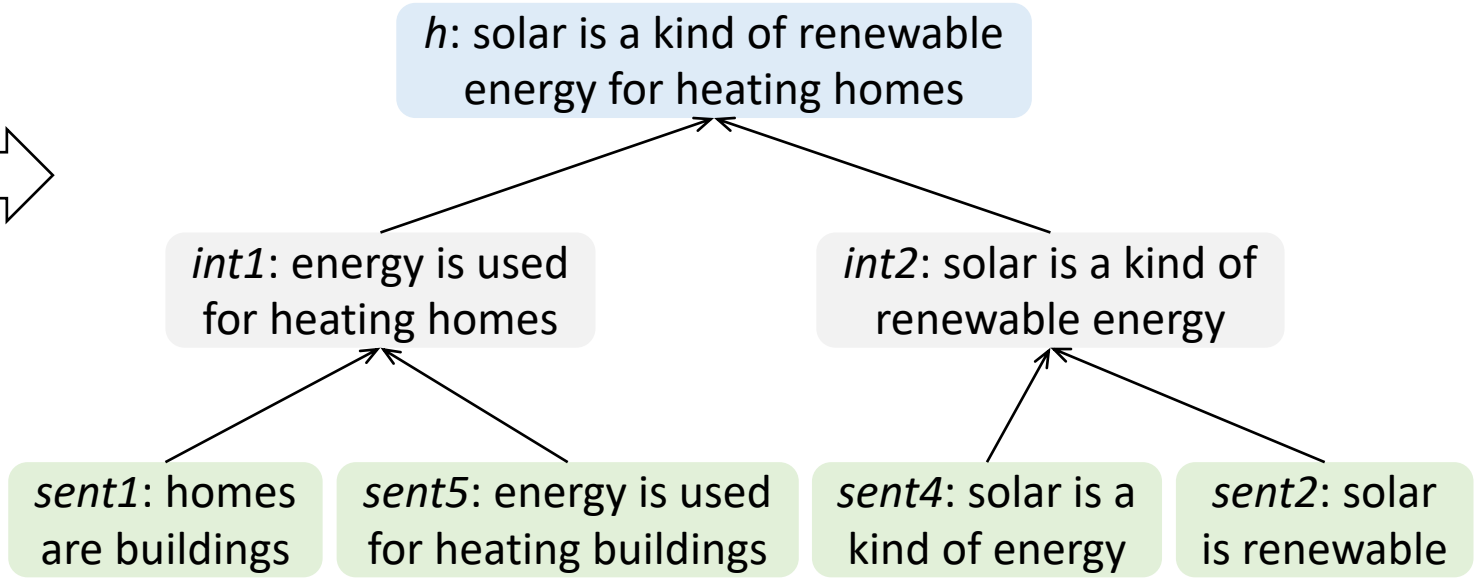
h: solar is a kind of renewable energy for heating homes

Supporting facts (C):

sent1: homes are buildings
sent2: solar is renewable
sent3: wind is a kind of energy
sent4: solar is a kind of energy
sent5: energy is used for heating buildings
sent6: coal is nonrenewable
…
…

Input

Proof tree (T):



Output

Stepwise Methods

[Tafjord et al. Findings of ACL 2021]
[Sanyal et al. ACL 2022]
[Bostrom et al. arXiv 2022]

Generate the proof step by step

Hypothesis (h):

h: solar is a kind of renewable energy for heating homes

Supporting facts (C):

sent1: homes are buildings
sent2: solar is renewable
sent3: wind is a kind of energy
sent4: solar is a kind of energy
sent5: energy is used for heating buildings
sent6: coal is nonrenewable
…
…

Input

Proof tree (T):

Step 1

int1: energy is used for heating homes

sent1: homes are buildings

sent5: energy is used for heating buildings

Output

Stepwise Methods

[Tafjord et al. Findings of ACL 2021]
[Sanyal et al. ACL 2022]
[Bostrom et al. arXiv 2022]

Generate the proof step by step

Hypothesis (h):

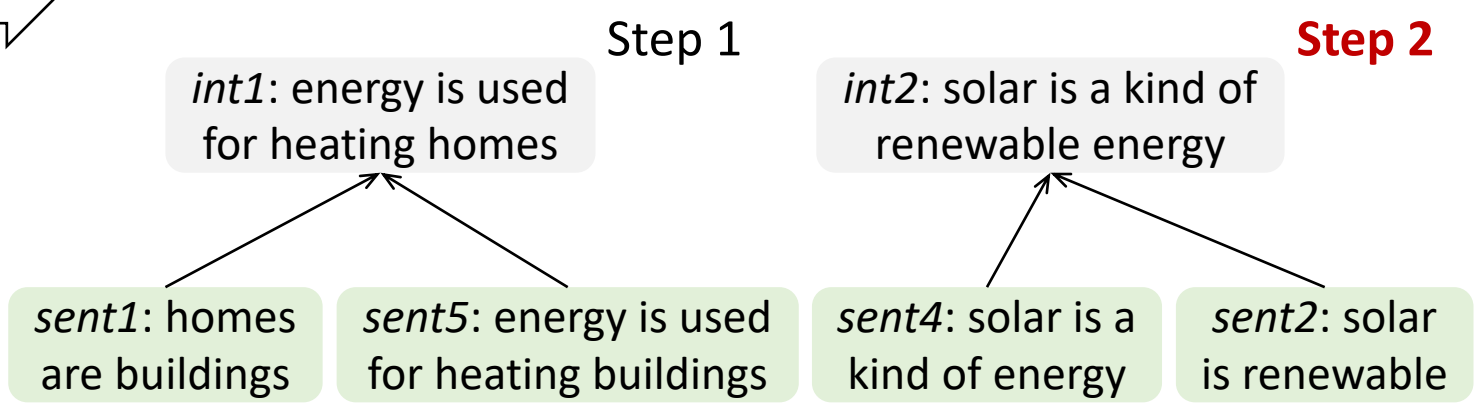
h: solar is a kind of renewable energy for heating homes

Supporting facts (C):

sent1: homes are buildings
sent2: solar is renewable
sent3: wind is a kind of energy
sent4: solar is a kind of energy
sent5: energy is used for heating buildings
sent6: coal is nonrenewable
…
…

Input

Proof tree (T):



Output

Stepwise Methods

[Tafjord et al. Findings of ACL 2021]
[Sanyal et al. ACL 2022]
[Bostrom et al. arXiv 2022]

Generate the proof step by step

Hypothesis (h):

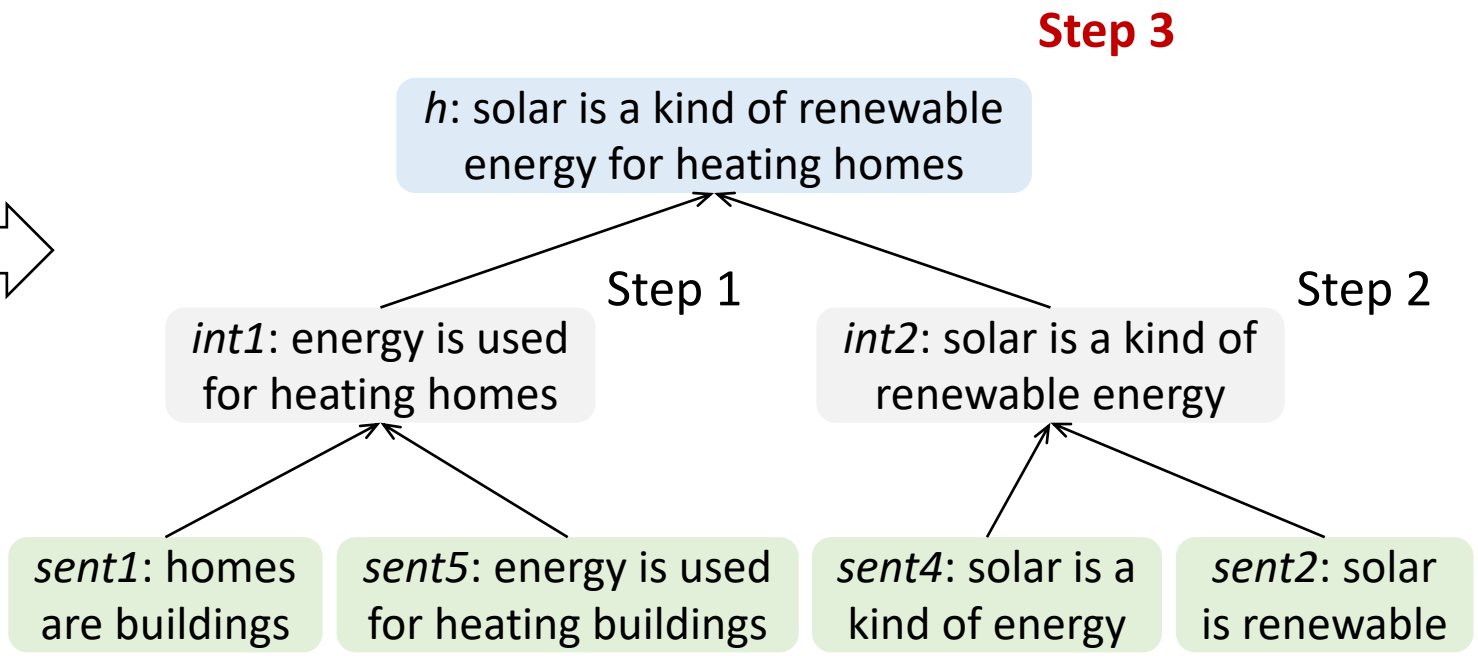
h: solar is a kind of renewable energy for heating homes

Supporting facts (C):

sent1: homes are buildings
sent2: solar is renewable
sent3: wind is a kind of energy
sent4: solar is a kind of energy
sent5: energy is used for heating buildings
sent6: coal is nonrenewable
…
…

Input

Proof tree (T):



Output

Stepwise Methods

[Tafjord et al. Findings of ACL 2021]
[Sanyal et al. ACL 2022]
[Bostrom et al. arXiv 2022]

Generate the proof step by step

- **Compositional; easier for the model to learn and generalize**
- **Limited success on real-world data**

Hypothesis (h):

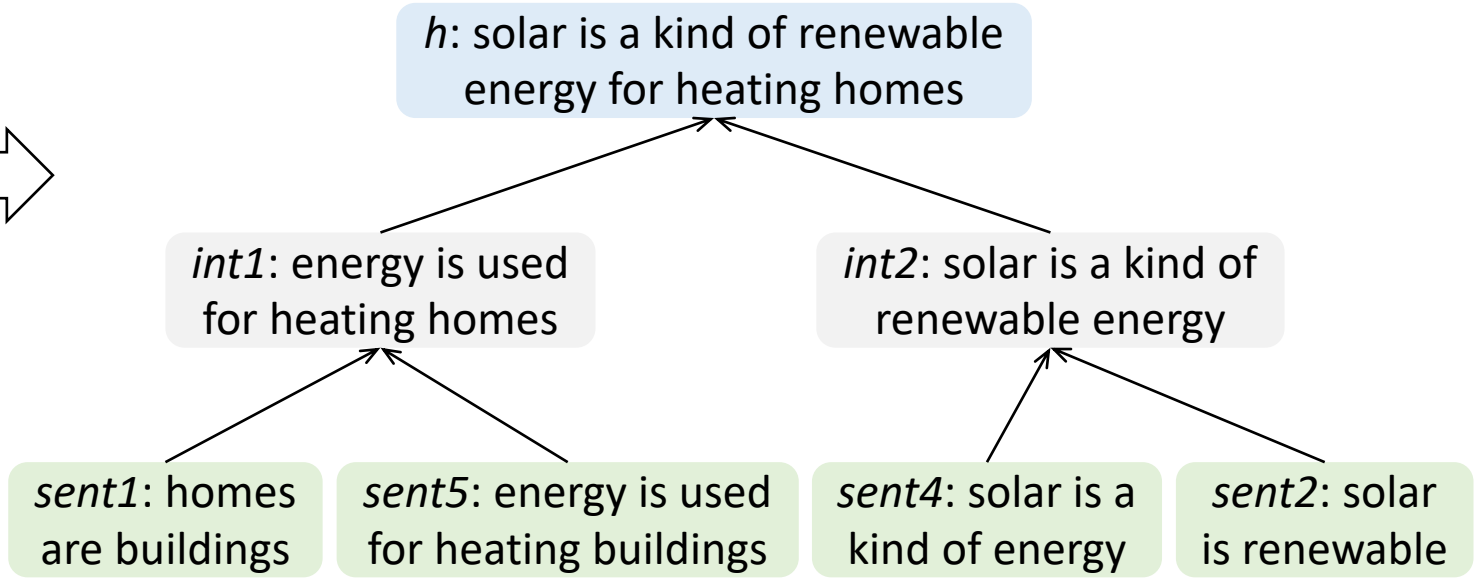
h: solar is a kind of renewable energy for heating homes

Supporting facts (C):

sent1: homes are buildings
sent2: solar is renewable
sent3: wind is a kind of energy
sent4: solar is a kind of energy
sent5: energy is used for heating buildings
sent6: coal is nonrenewable
…
…

Input

Proof tree (T):



Output

Stepwise Methods

[Tafjord et al. Findings of ACL 2021]
[Sanyal et al. ACL 2022]
[Bostrom et al. arXiv 2022]

Generate the proof step by step

- Compositional; easier for the model to learn and generalize
- Limited success on real-world data

Hypothesis (h):

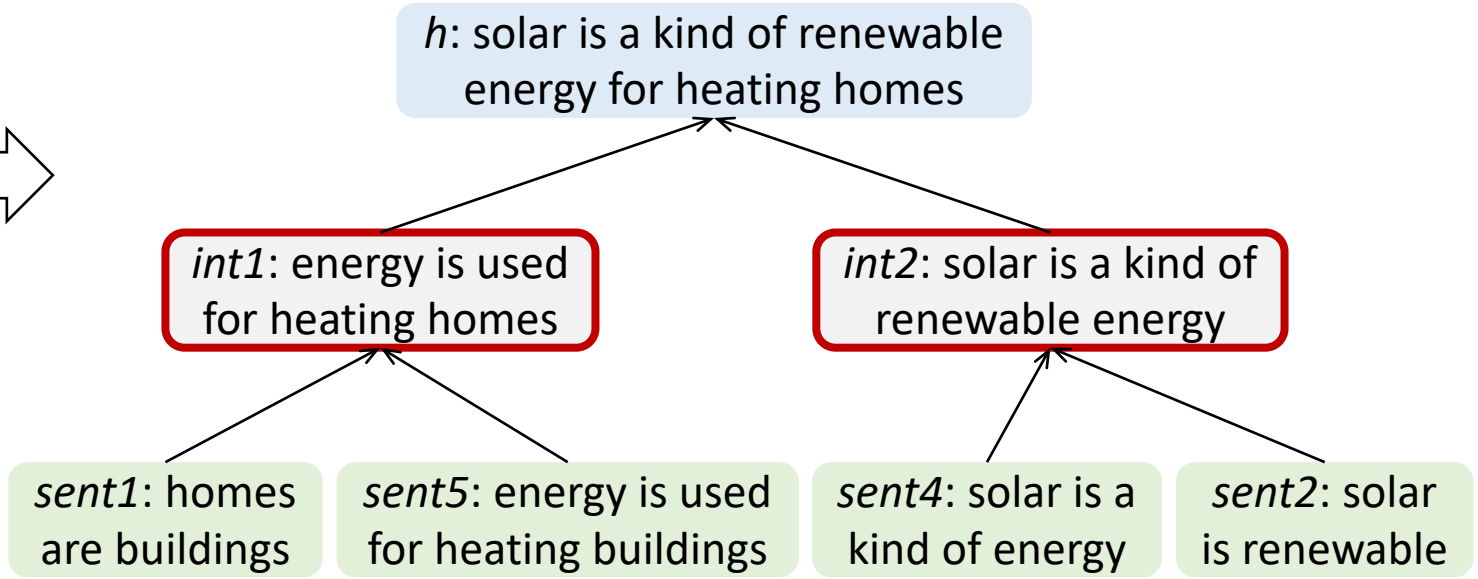
h: solar is a kind of renewable energy for heating homes

Supporting facts (C):

sent1: homes are buildings
sent2: solar is renewable
sent3: wind is a kind of energy
sent4: solar is a kind of energy
sent5: energy is used for heating buildings
sent6: coal is nonrenewable
…
…

Input

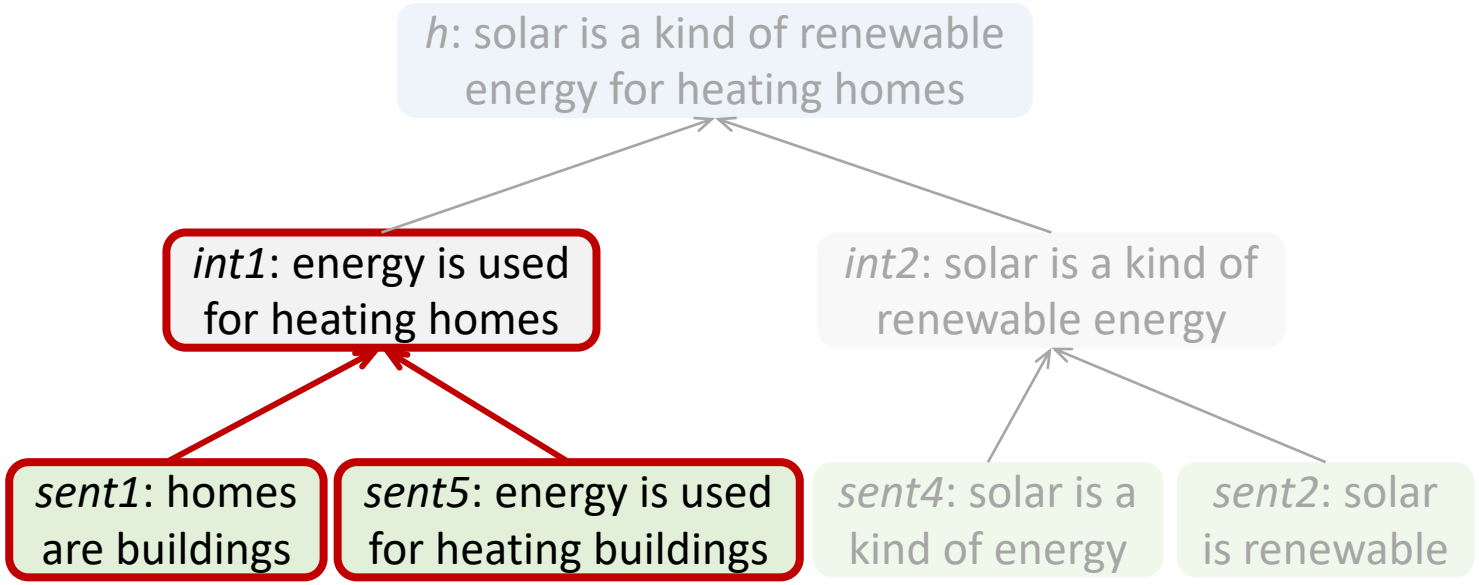
Proof tree (T):



Output

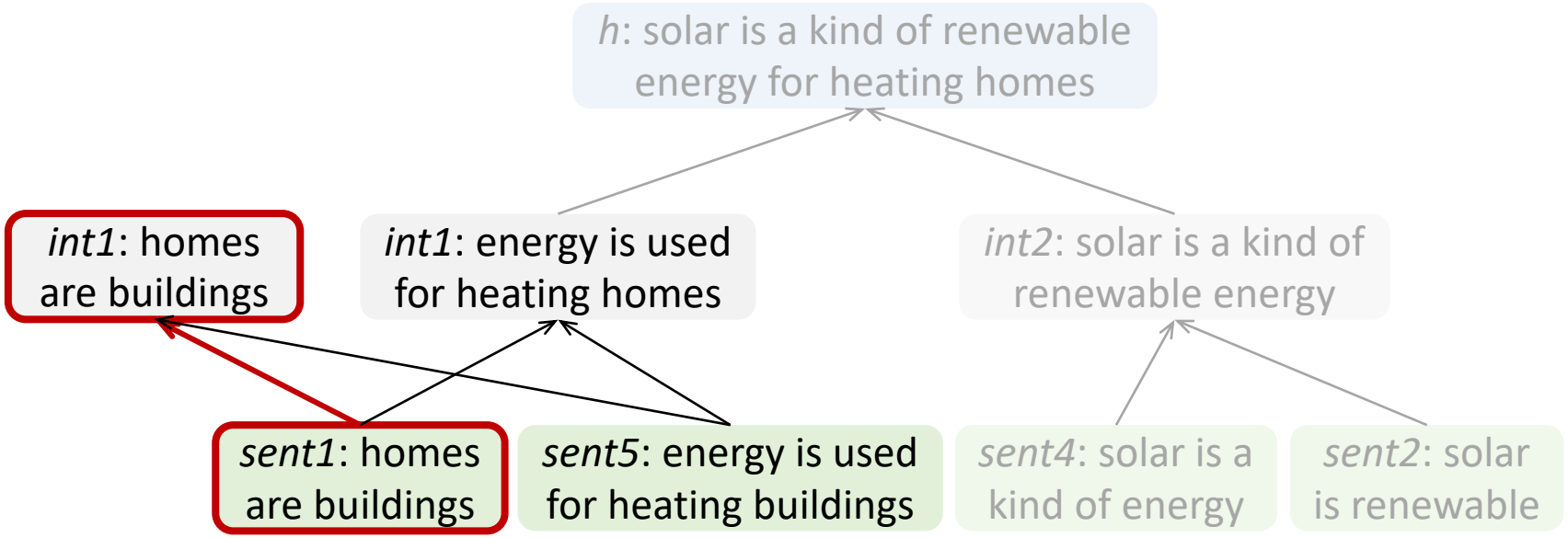
Relevance vs. Validity

- Many valid steps are irrelevant (not useful for proving the hypothesis)



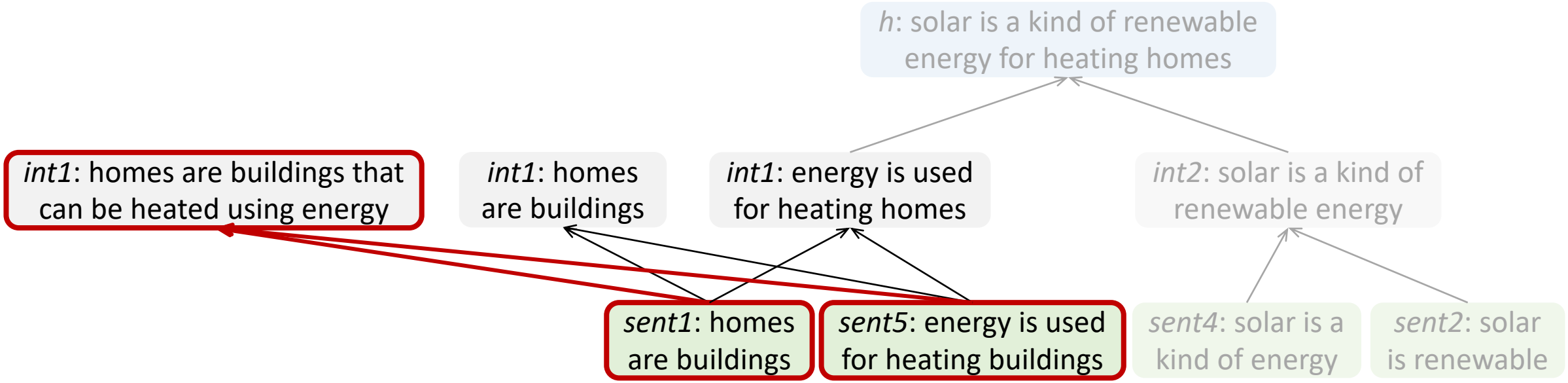
Relevance vs. Validity

- Many valid steps are irrelevant (not useful for proving the hypothesis)



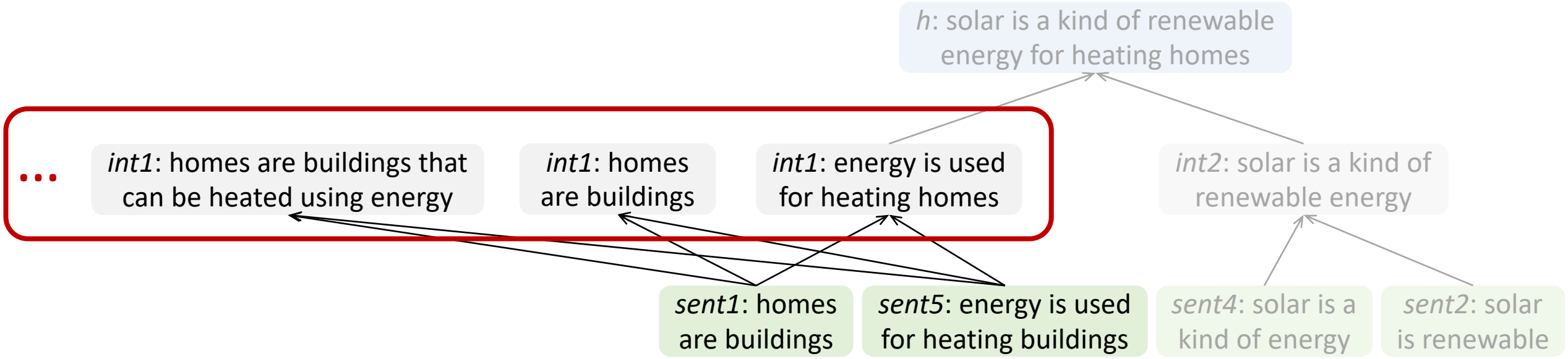
Relevance vs. Validity

- Many valid steps are irrelevant (not useful for proving the hypothesis)



Relevance vs. Validity

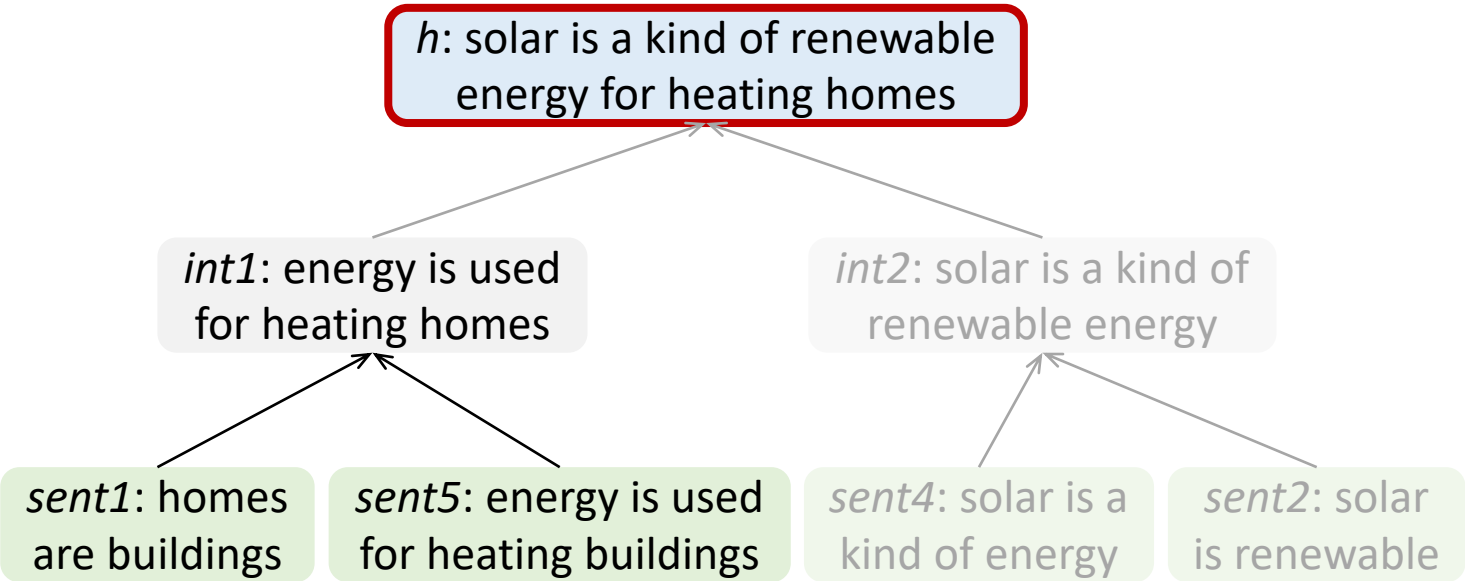
- Many valid steps are irrelevant (not useful for proving the hypothesis)



Relevance vs. Validity

- Many valid steps are irrelevant (not useful for proving the hypothesis)
- The model hallucinates invalid steps

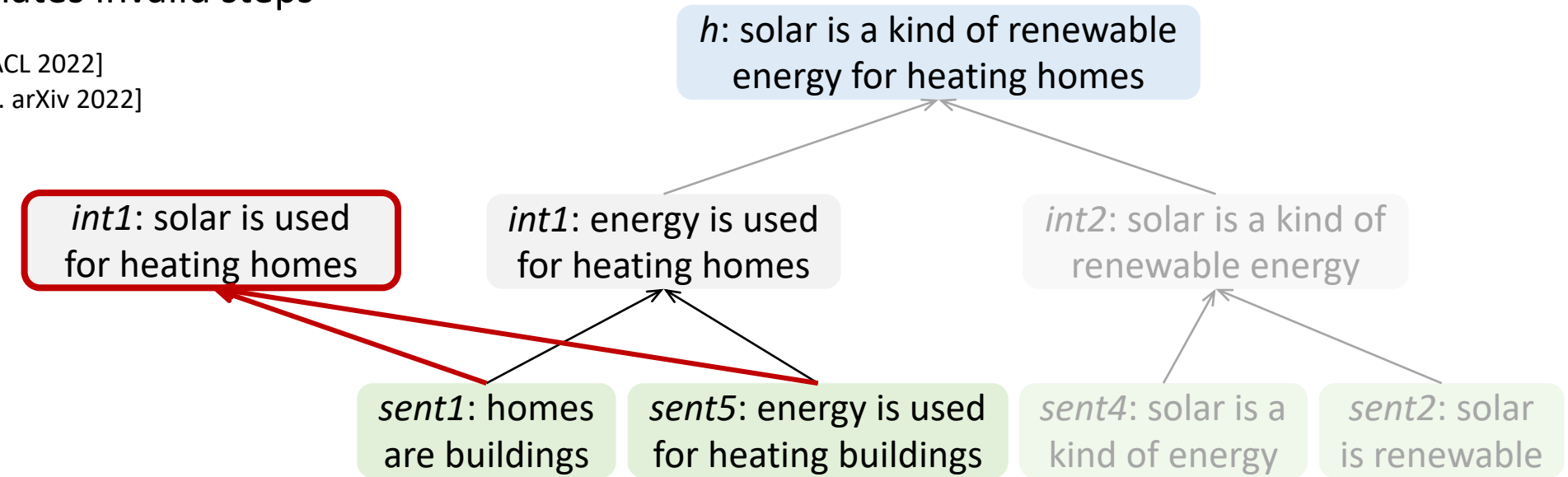
[Sanyal et al. ACL 2022]
[Bostrom et al. arXiv 2022]



Relevance vs. Validity

- Many valid steps are irrelevant (not useful for proving the hypothesis)
- The model hallucinates invalid steps

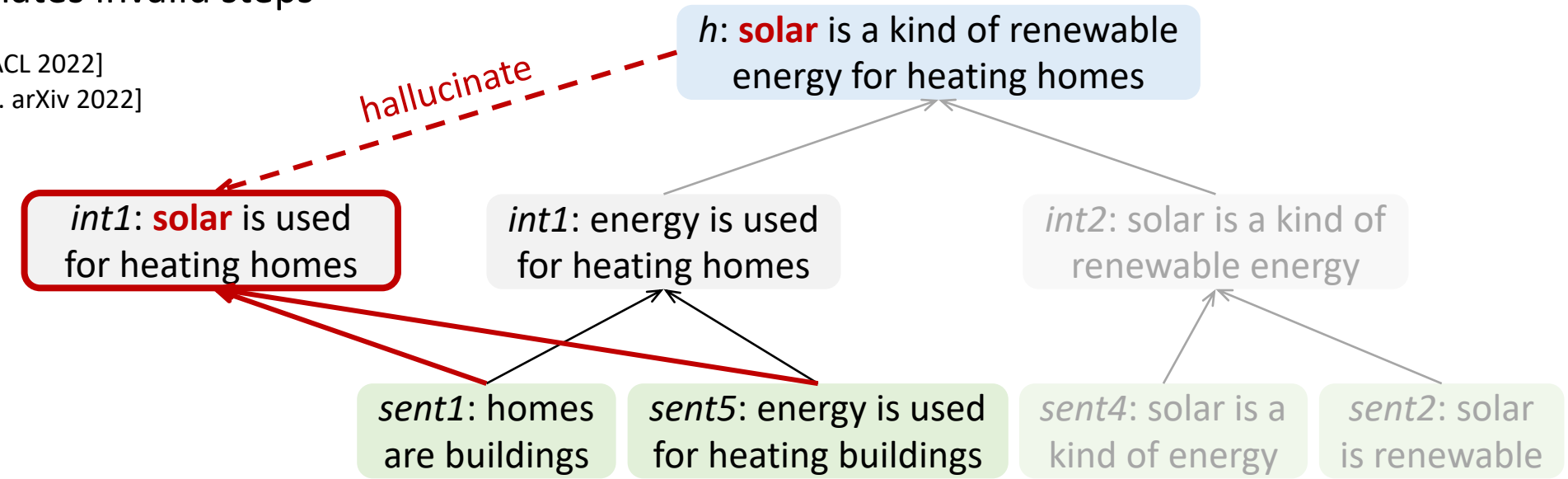
[Sanyal et al. ACL 2022]
[Bostrom et al. arXiv 2022]



Relevance vs. Validity

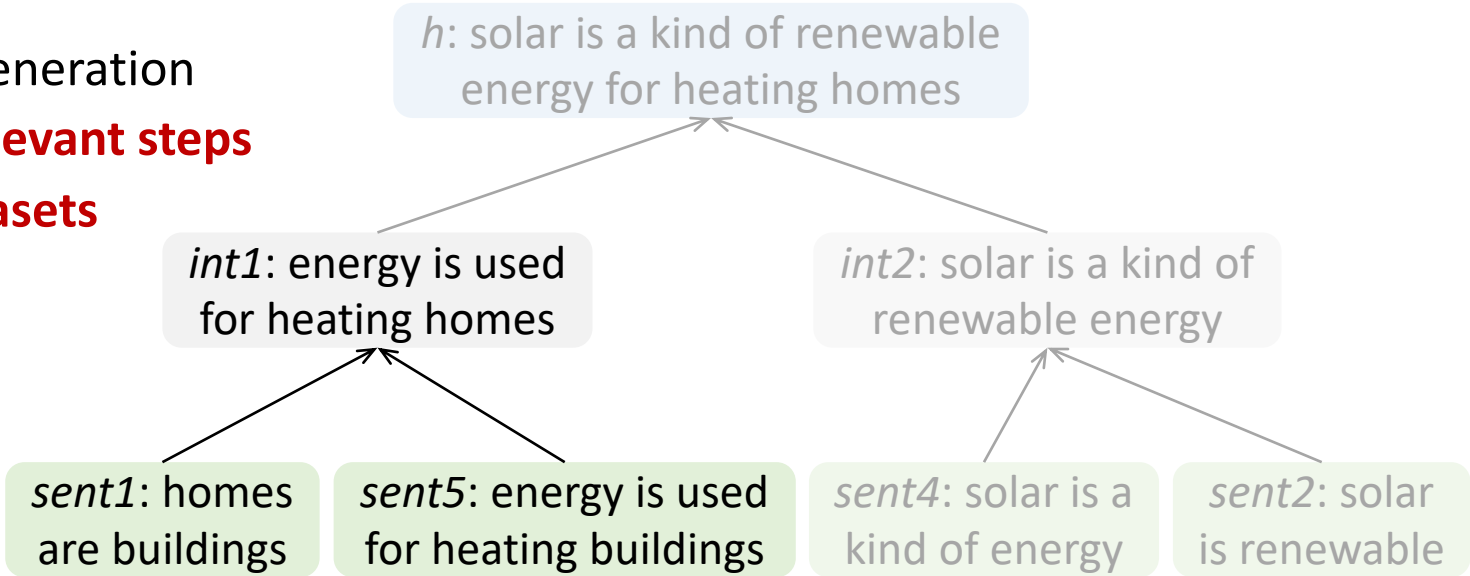
- Many valid steps are irrelevant (not useful for proving the hypothesis)
- The model hallucinates invalid steps

[Sanyal et al. ACL 2022]
[Bostrom et al. arXiv 2022]



Relevance vs. Validity

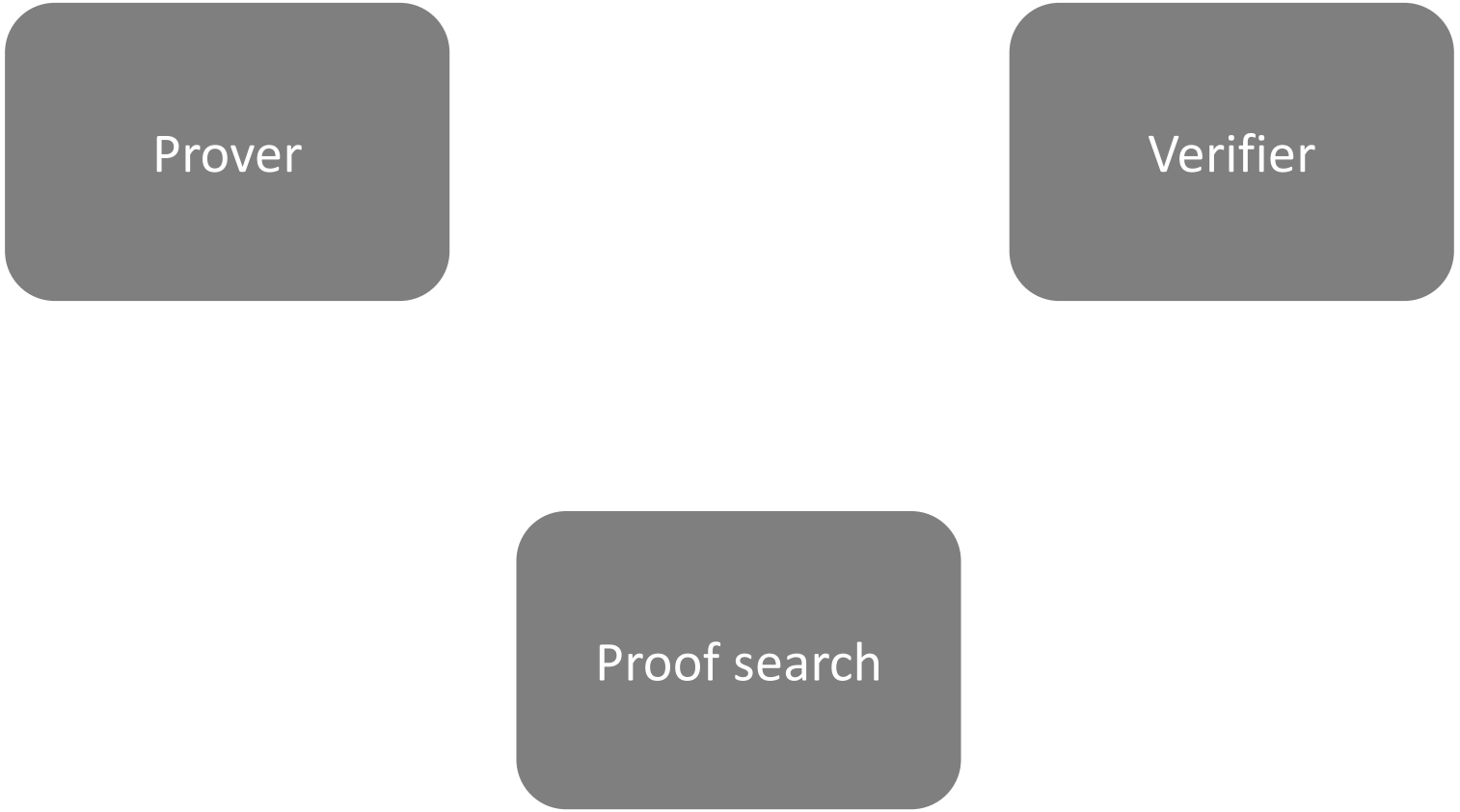
- Many valid steps are irrelevant (not useful for proving the hypothesis)
- The model hallucinates invalid steps
- Existing methods for stepwise proof generation
  - **Struggle to generate valid and relevant steps**
  - **Underperform on real-world datasets**



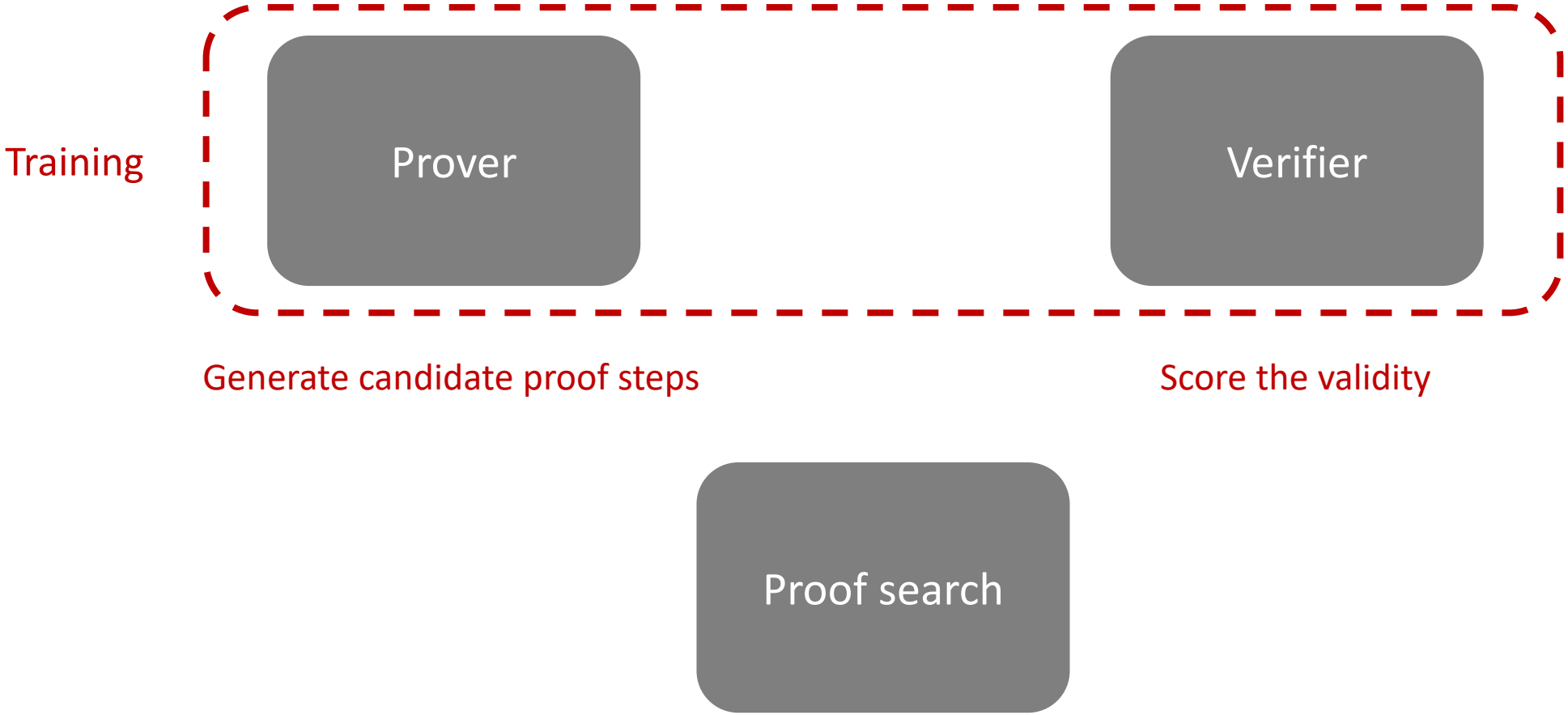
NLProofS: Natural Language Proof Search

- A new method for stepwise proof generation
- To generate relevant steps: Condition on the hypothesis
- To generate valid steps: Train an independent **verifier** to prevent hallucination
- Effective in generating challenging, human-authored proofs

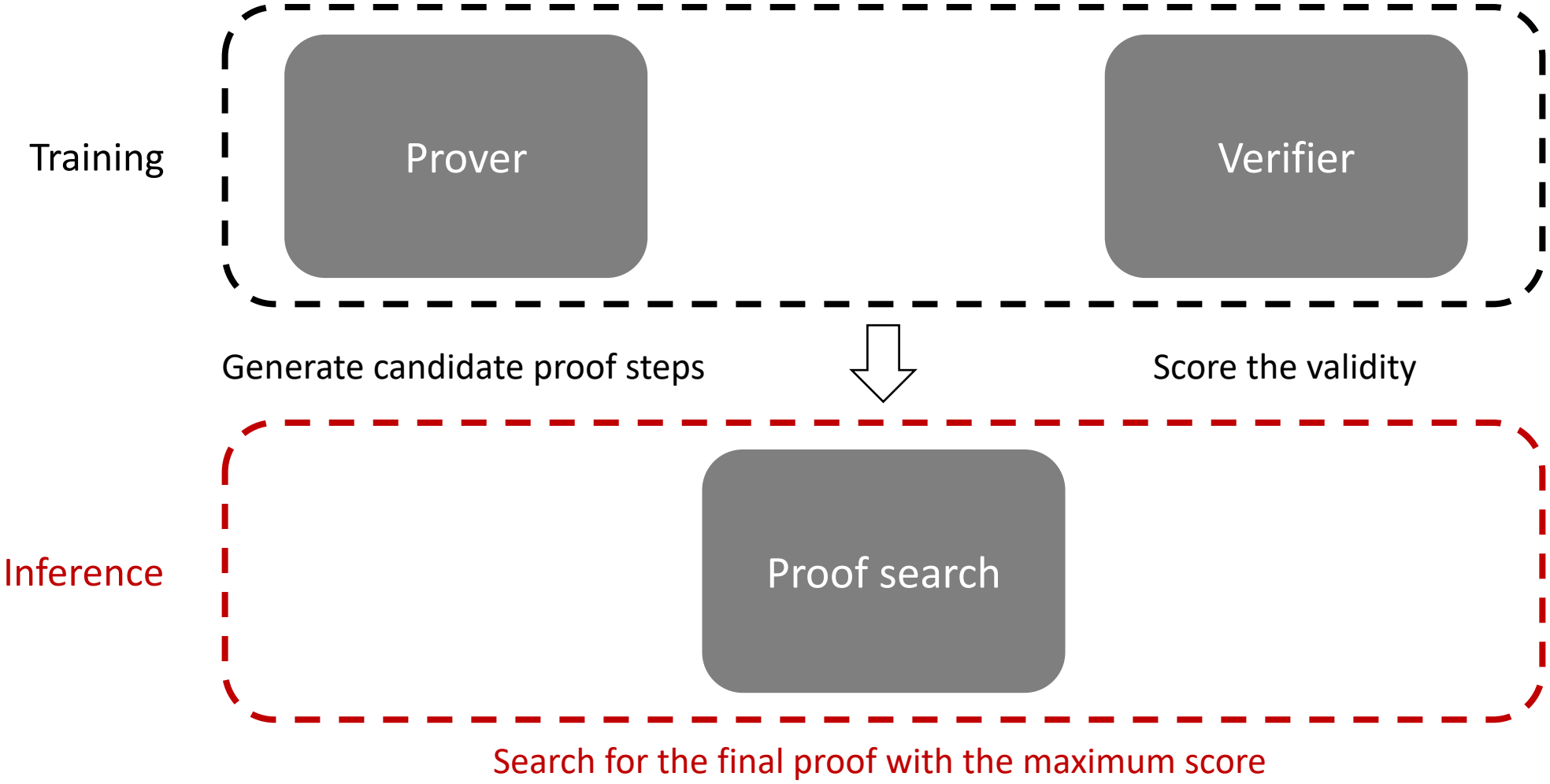
Overview of NLProofS



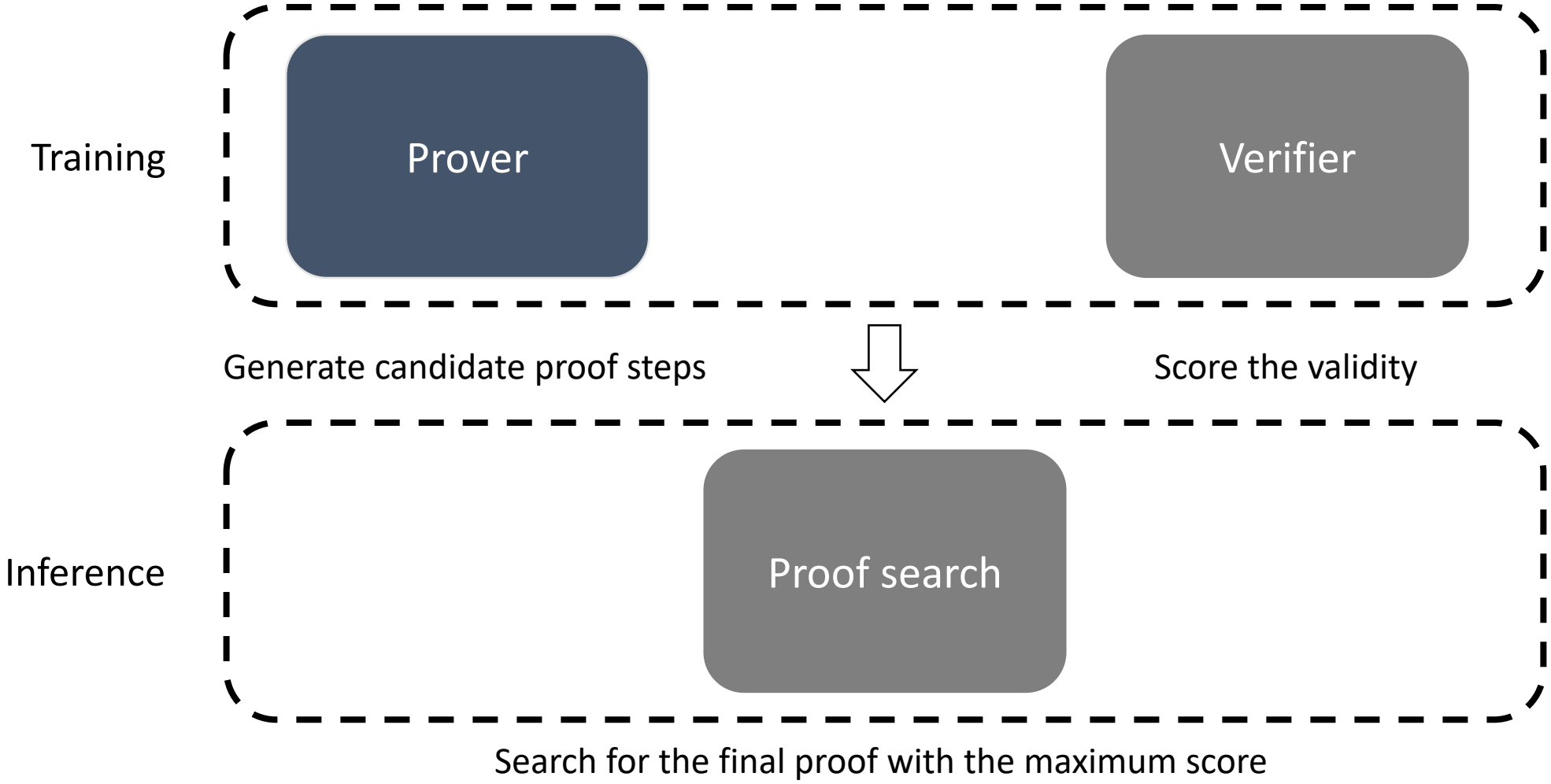
Overview of NLProofS



Overview of NLProofS



Overview of NLProofS



Stepwise Prover

[Raffle et al. JMLR 2020] [Tafjord et al. Findings of ACL 2021]

- Finetune a T5 model to predict the next proof step

Stepwise Prover

[Raffle et al. JMLR 2020] [Tafjord et al. Findings of ACL 2021]

- Finetune a T5 model to predict the next proof step

h: solar is a kind of renewable energy for heating homes

sent1: homes are buildings
sent2: solar is renewable
sent3: wind is a kind of energy
sent4: solar is a kind of energy
sent5: energy is used for heating buildings
sent6: coal is nonrenewable
…
…

1-step partial proof

int1: energy is used for heating homes

sent1: homes are buildings

sent5: energy is used for heating buildings

Stepwise Prover

[Raffle et al. JMLR 2020] [Tafjord et al. Findings of ACL 2021]

- Finetune a T5 model to predict the next proof step

h: solar is a kind of renewable energy for heating homes

sent1: homes are buildings
sent2: solar is renewable
sent3: wind is a kind of energy
sent4: solar is a kind of energy
sent5: energy is used for heating buildings
sent6: coal is nonrenewable
…
…

1-step partial proof

int1: energy is used for heating homes

sent1: homes are buildings

sent5: energy is used for heating buildings

$hypothesis$ = solar is a kind of renewable energy for heating homes ;
$facts$ = sent1: homes are buildings sent2: solar is renewable … ;
$partial_proof$ = sent1 & sent5 -> int1: energy is used for heating homes;

Stepwise Prover

[Raffle et al. JMLR 2020] [Tafjord et al. Findings of ACL 2021]

- Finetune a T5 model to predict the next proof step

h: solar is a kind of renewable energy for heating homes

sent1: homes are buildings
sent2: solar is renewable
sent3: wind is a kind of energy
sent4: solar is a kind of energy
sent5: energy is used for heating buildings
sent6: coal is nonrenewable
…
…

int1: energy is used for heating homes

int2: solar is a kind of renewable energy

sent1: homes are buildings

sent5: energy is used for heating buildings

sent4: solar is a kind of energy

sent2: solar is renewable

$hypothesis$ = solar is a kind of renewable energy for heating homes ;
$facts$ = sent1: homes are buildings sent2: solar is renewable … ;
$partial_proof$ = sent1 & sent5 -> int1: energy is used for heating homes;

T5

sent2 & sent4 -> int2: solar is a kind of renewable energy;

Stepwise Prover

[Raffle et al. JMLR 2020] [Tafjord et al. Findings of ACL 2021]

- Finetune a T5 model to predict the next proof step

h: solar is a kind of renewable energy for heating homes

sent1: homes are buildings
sent2: solar is renewable
sent3: wind is a kind of energy
sent4: solar is a kind of energy
sent5: energy is used for heating buildings
sent6: coal is nonrenewable
…
…

- **Generate multiple candidate steps with beam search**
- **Filter out syntax errors**

int1: energy is used for heating homes

int2: solar is a kind of renewable energy

sent1: homes are buildings

sent5: energy is used for heating buildings

sent4: solar is a kind of energy

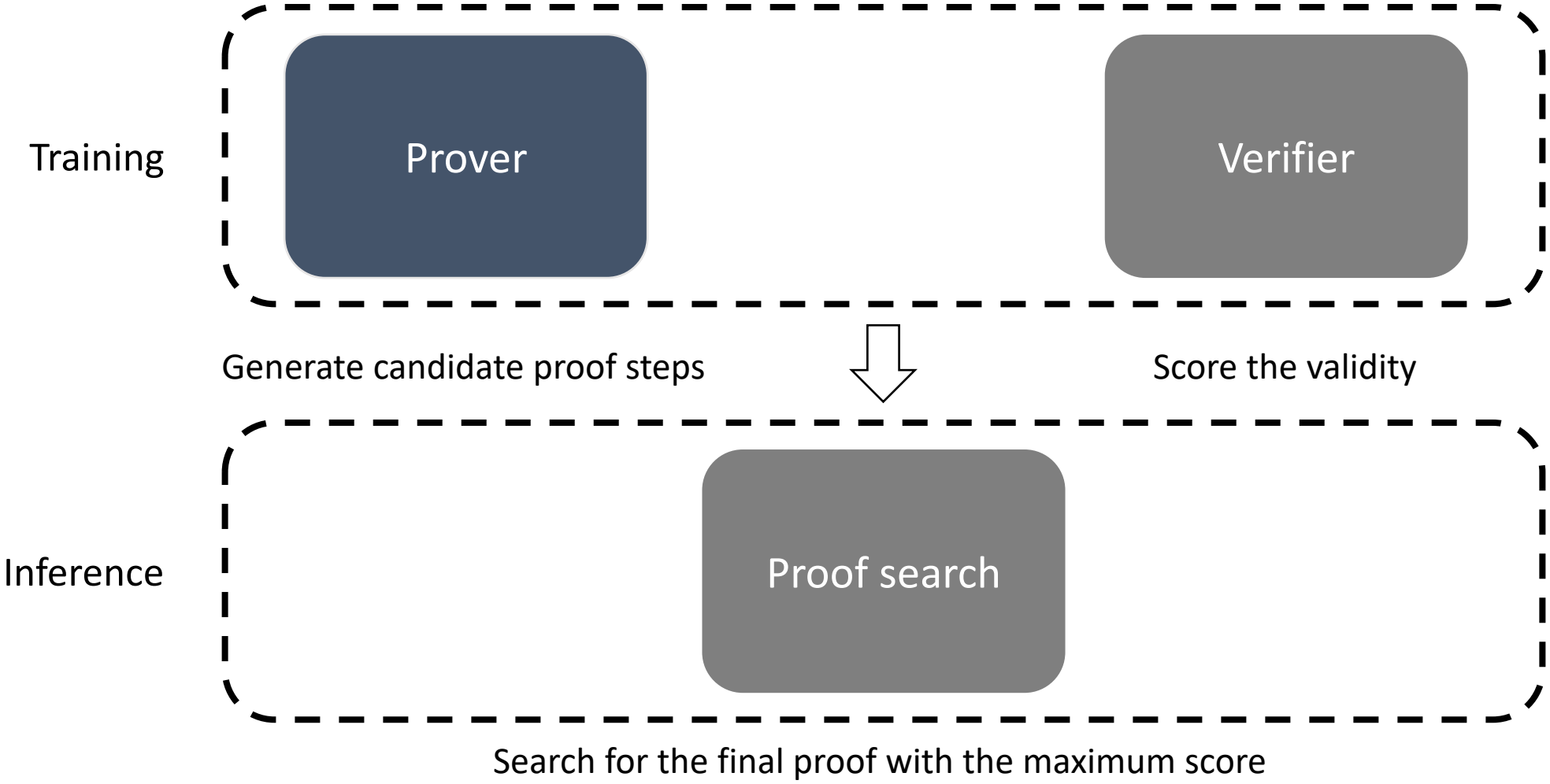
sent2: solar is renewable

$hypothesis$ = solar is a kind of renewable energy for heating homes ;
$facts$ = sent1: homes are buildings sent2: solar is renewable … ;
$partial_proof$ = sent1 & sent5 -> int1: energy is used for heating homes;

T5

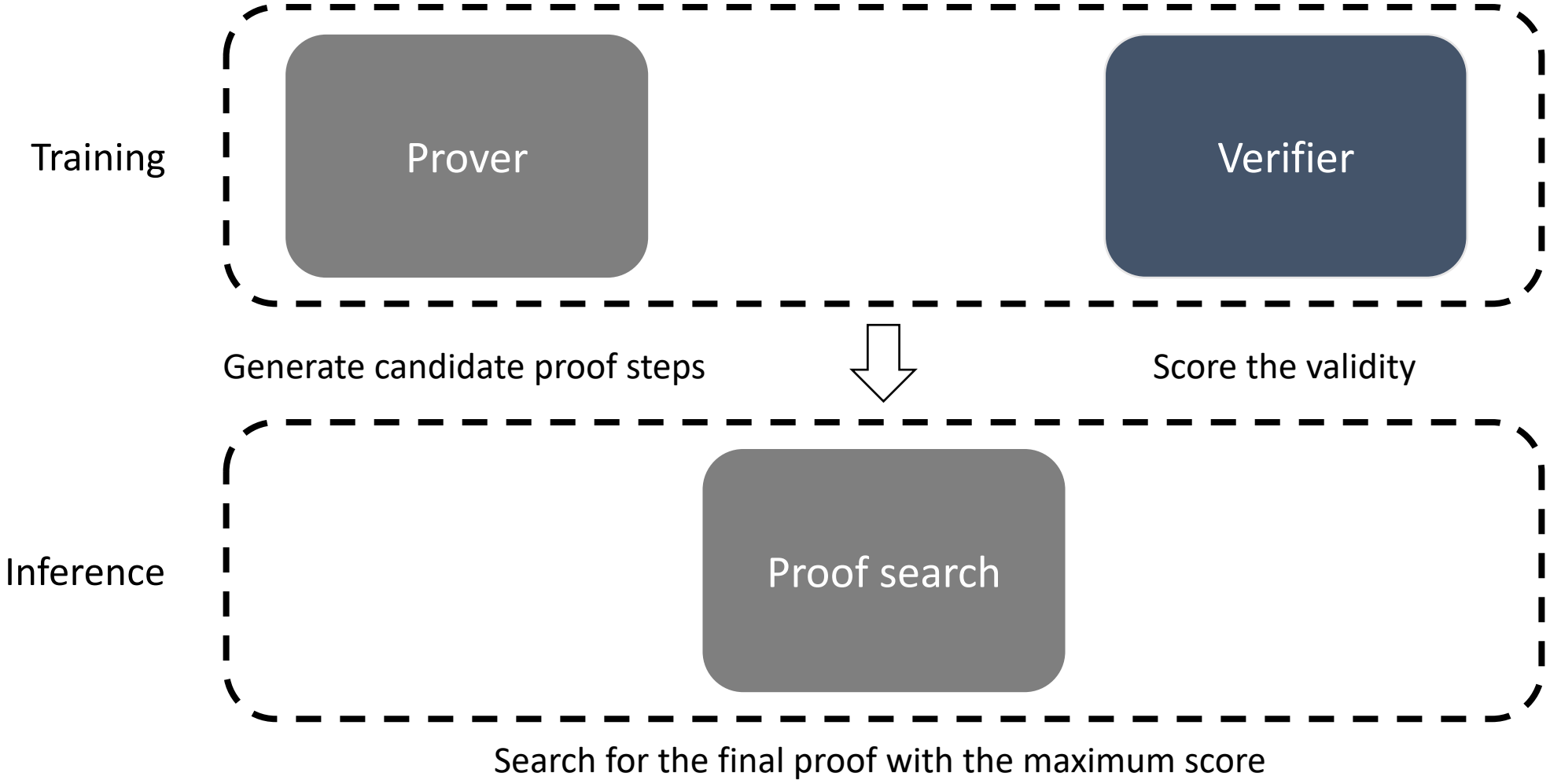
sent2 & sent4 -> int2: solar is a kind of renewable energy;

Overview of NLProofS



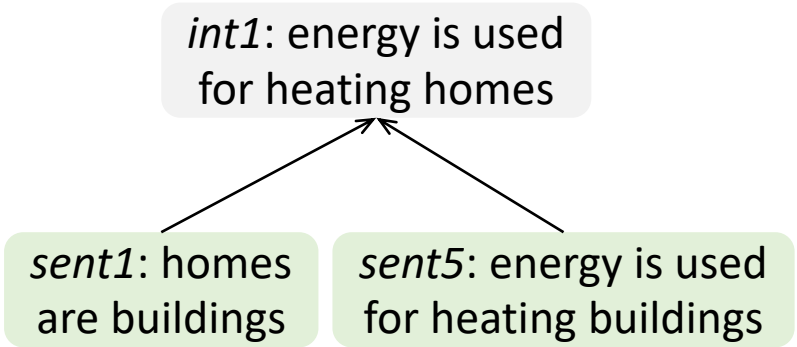
Overview of NLProofS

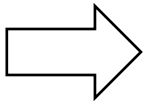
- **How to score a single step?**
- **How to score the entire proof?**



Verifier: Scoring a Proof Step

- **Input**: a proof step (multiple premises and one conclusion)
- **Output**: a continuous validity score in [0, 1]



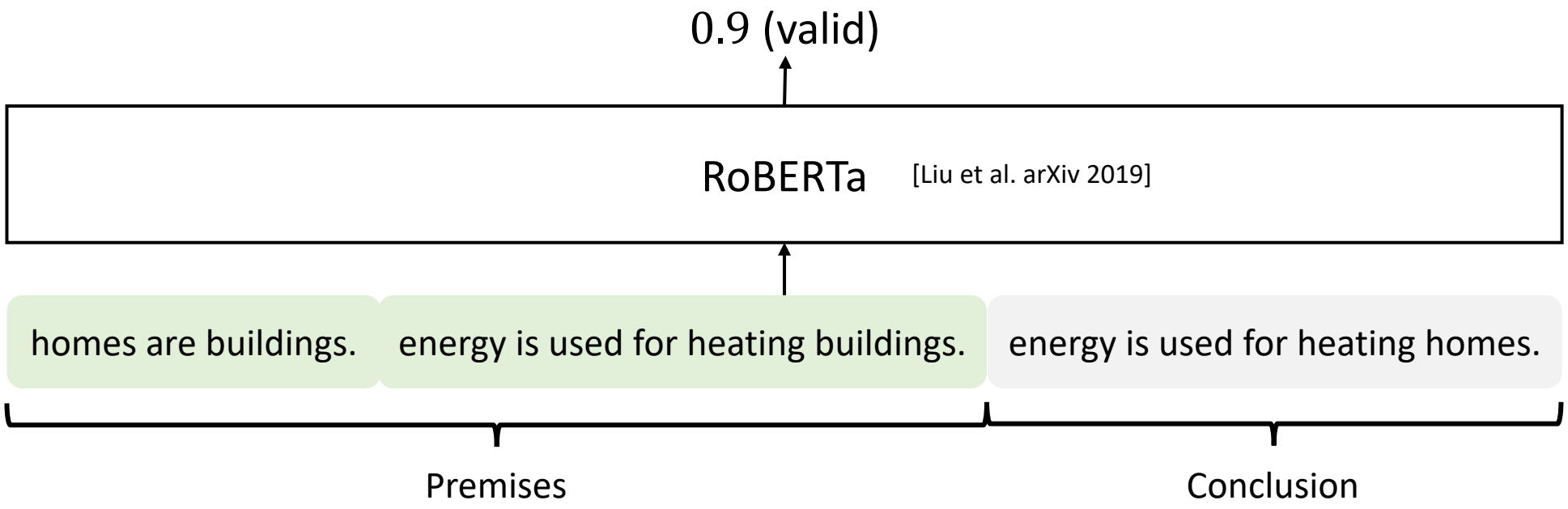


0.9

Input **Output**

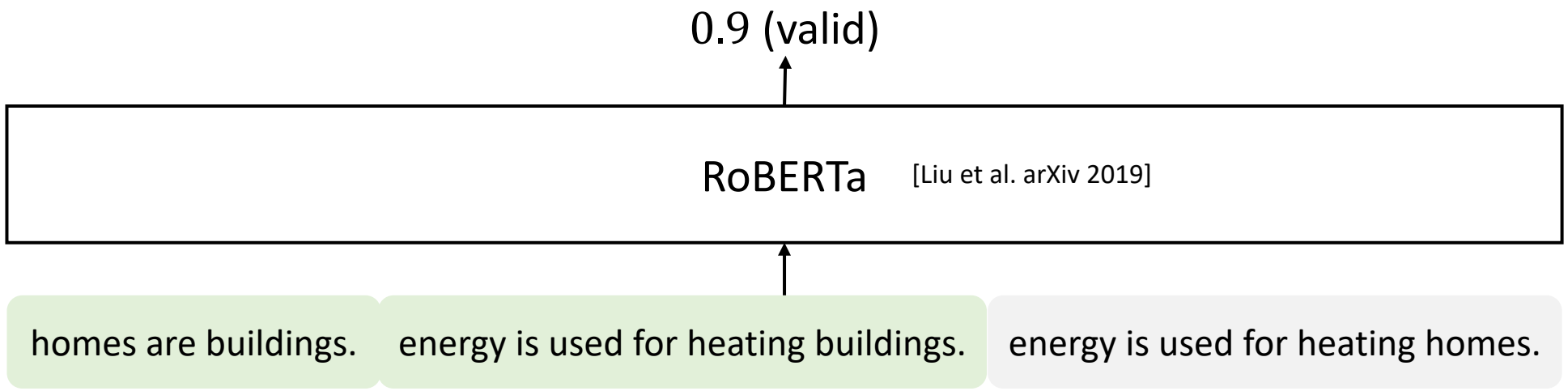
Verifier: Scoring a Proof Step

- Input: a proof step (multiple premises and one conclusion)
- Output: a continuous validity score in [0, 1]
- **Finetune a RoBERTa model to classify steps as valid/invalid**



Verifier: Scoring a Proof Step

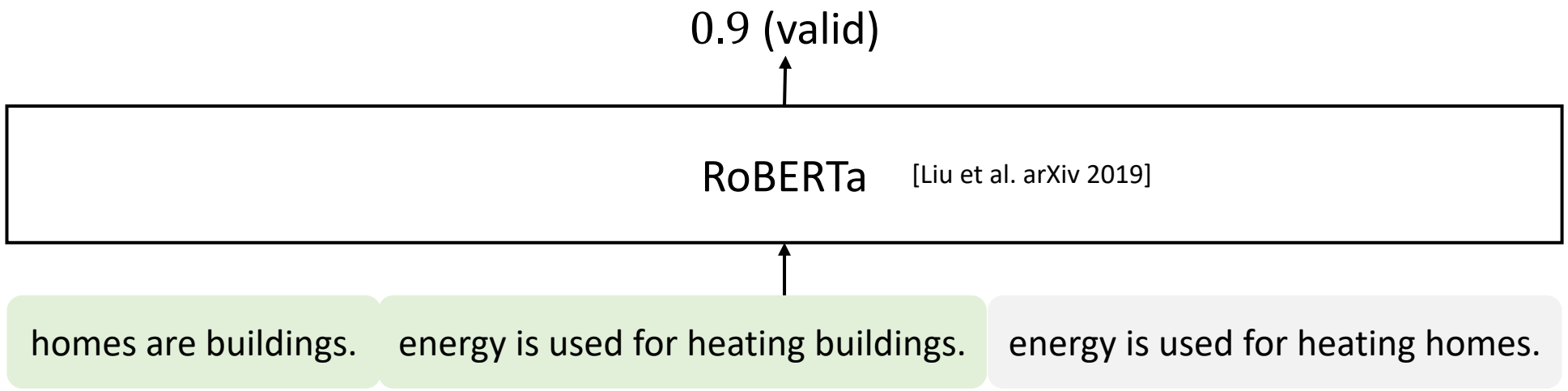
- Input: a proof step (multiple premises and one conclusion)
- Output: a continuous validity score in [0, 1]
- Finetune a RoBERTa model to classify steps as valid/invalid



- **How to generate negative examples (invalid proof steps) for training?**

Verifier: Scoring a Proof Step

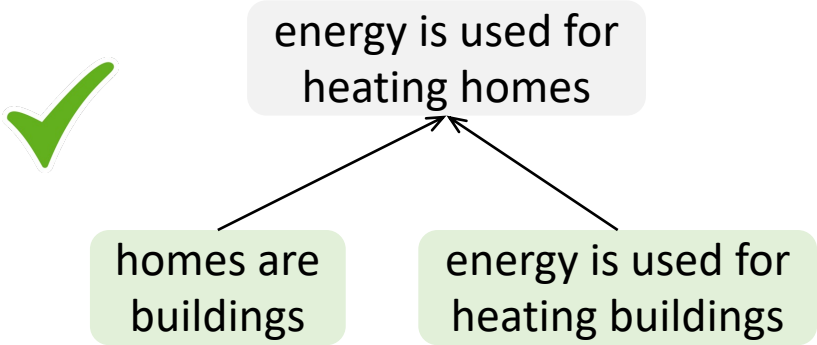
- Input: a proof step (multiple premises and one conclusion)
- Output: a continuous validity score in [0, 1]
- Finetune a RoBERTa model to classify steps as valid/invalid



- How to generate negative examples (invalid proof steps) for training?
  - Automatically generate **pseudo-negative examples**, without additional annotation efforts

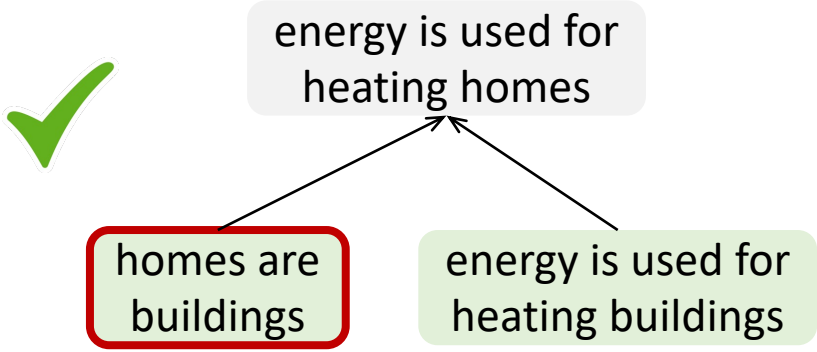
[Dalvi et al. arXiv 2022]

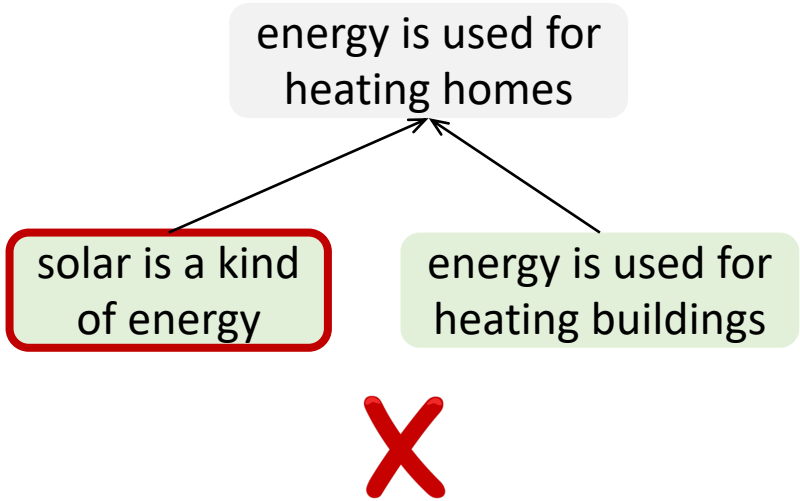
Pseudo-Negative Examples for Training the Verifier



Pseudo-Negative Examples for Training the Verifier

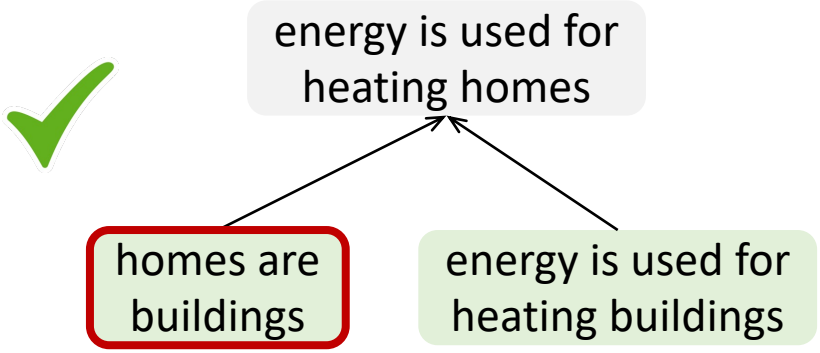
- **Replace premises**

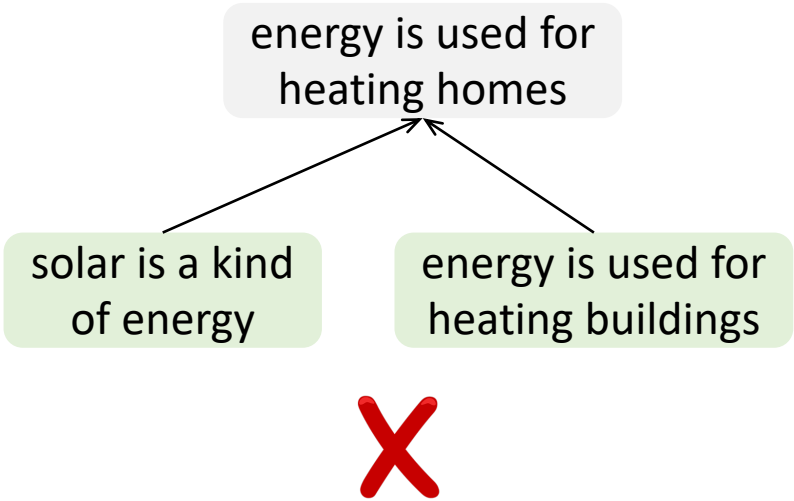


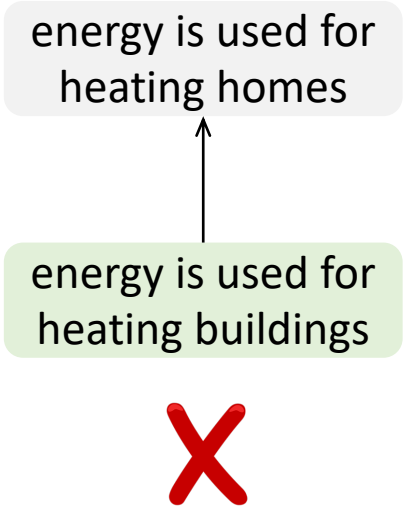


Pseudo-Negative Examples for Training the Verifier

- Replace premises
- **Remove premises**

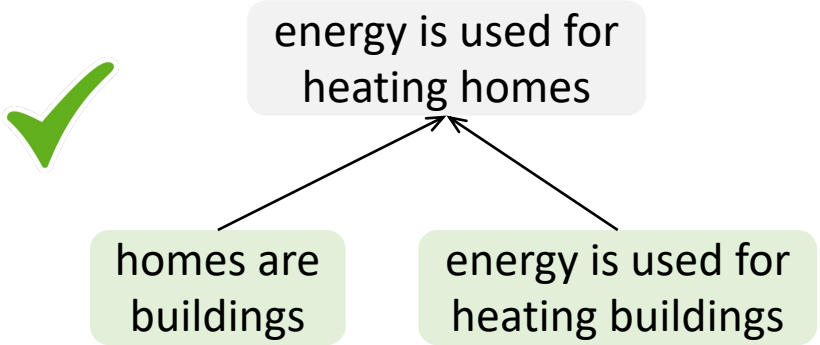


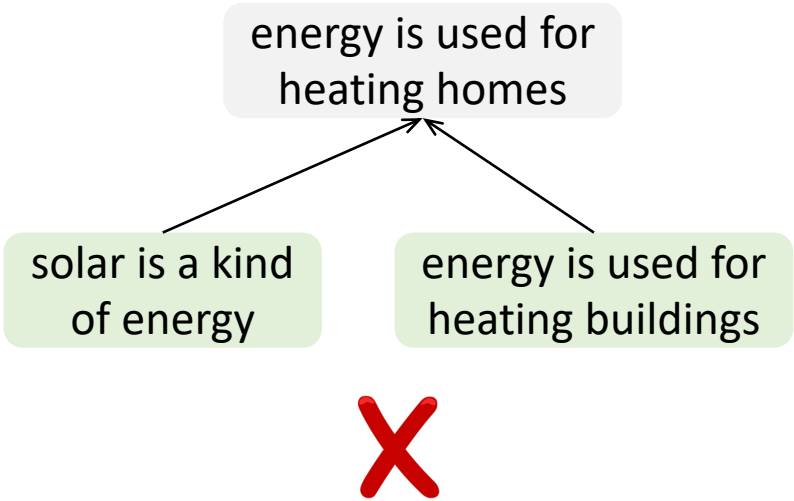


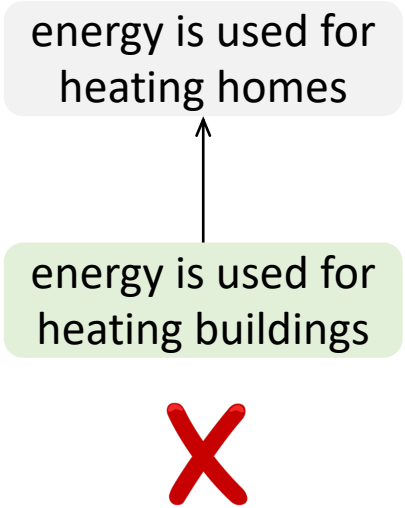


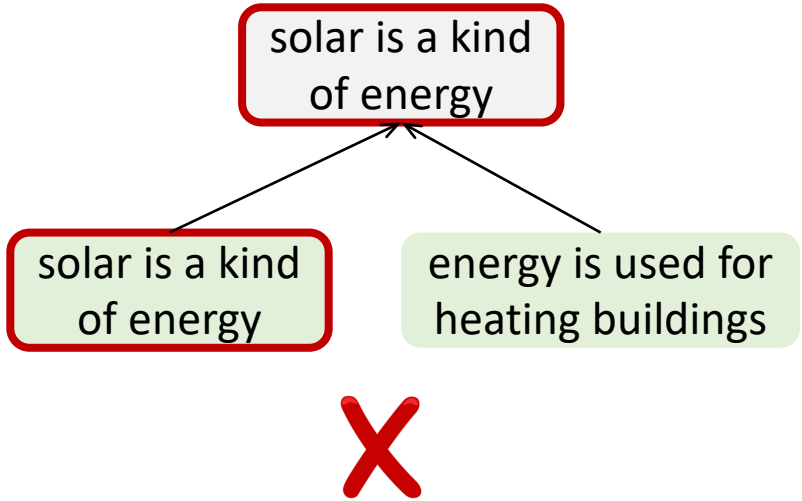
Pseudo-Negative Examples for Training the Verifier

- Replace premises
- Remove premises
- **Copy premises**



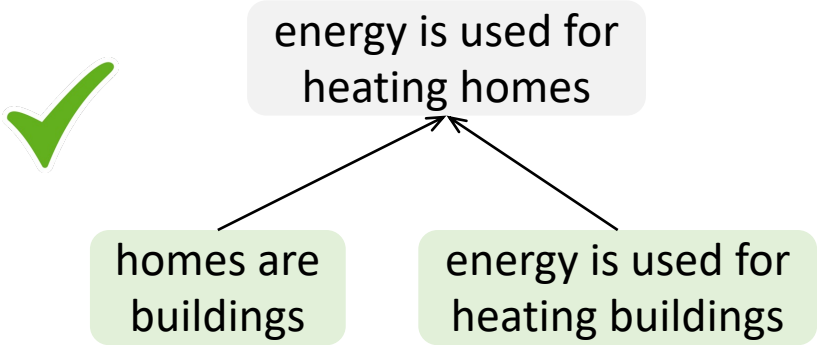


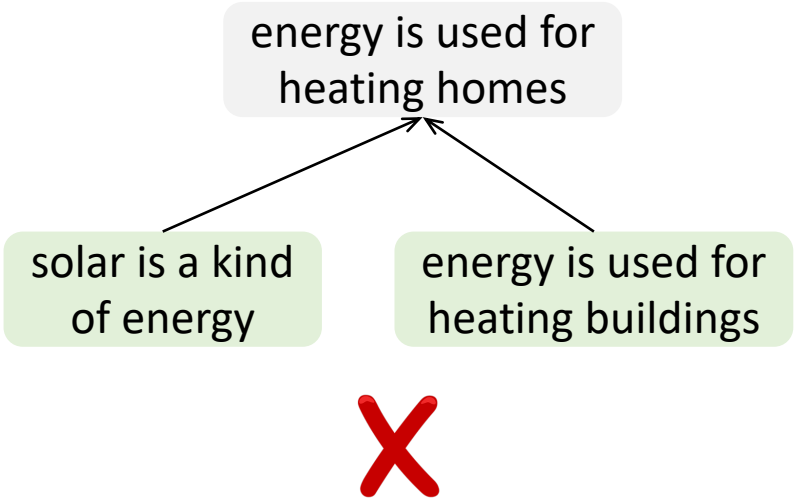


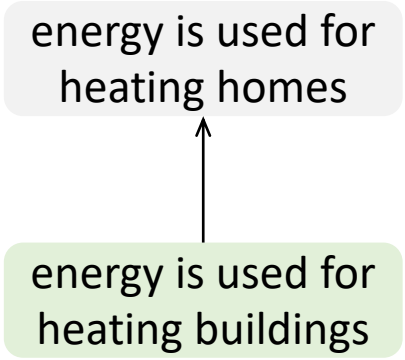


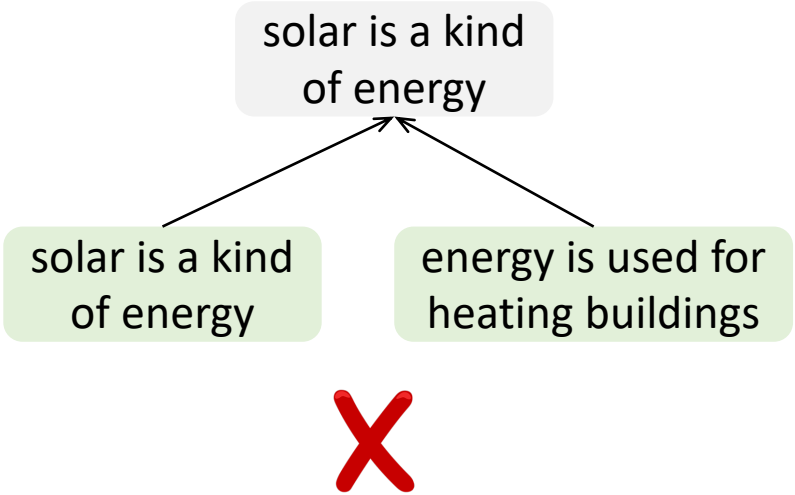
Pseudo-Negative Examples for Training the Verifier

- Replace premises
- Remove premises
- Copy premises

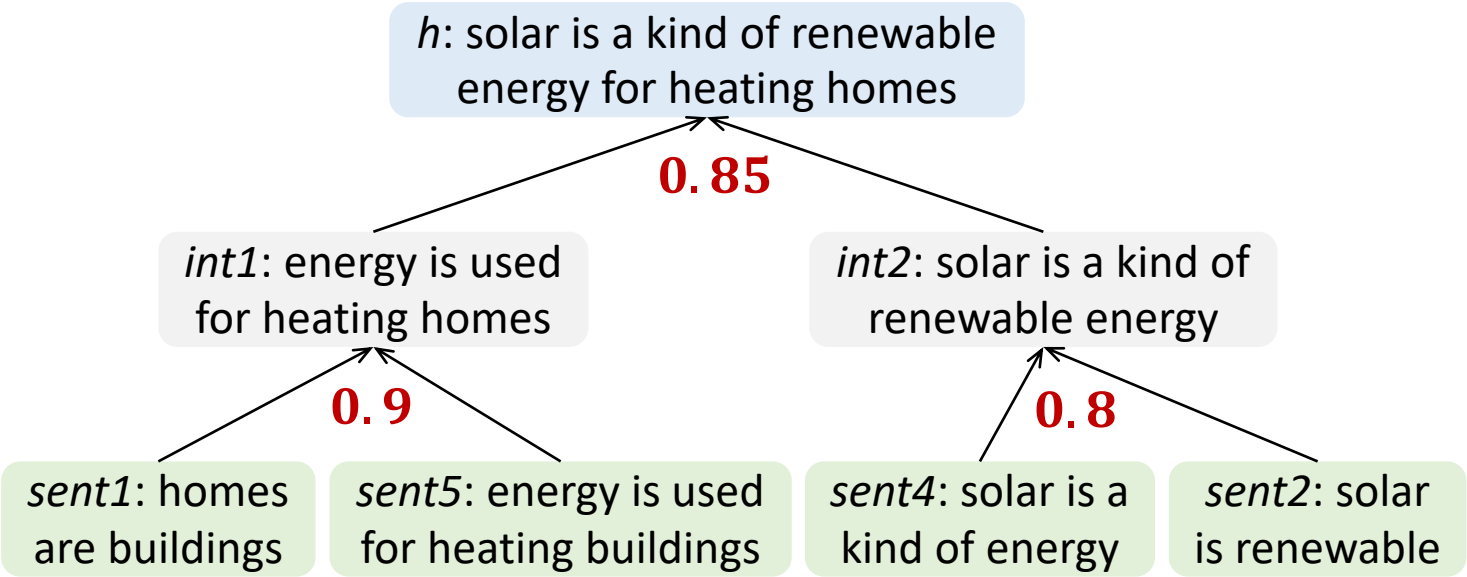






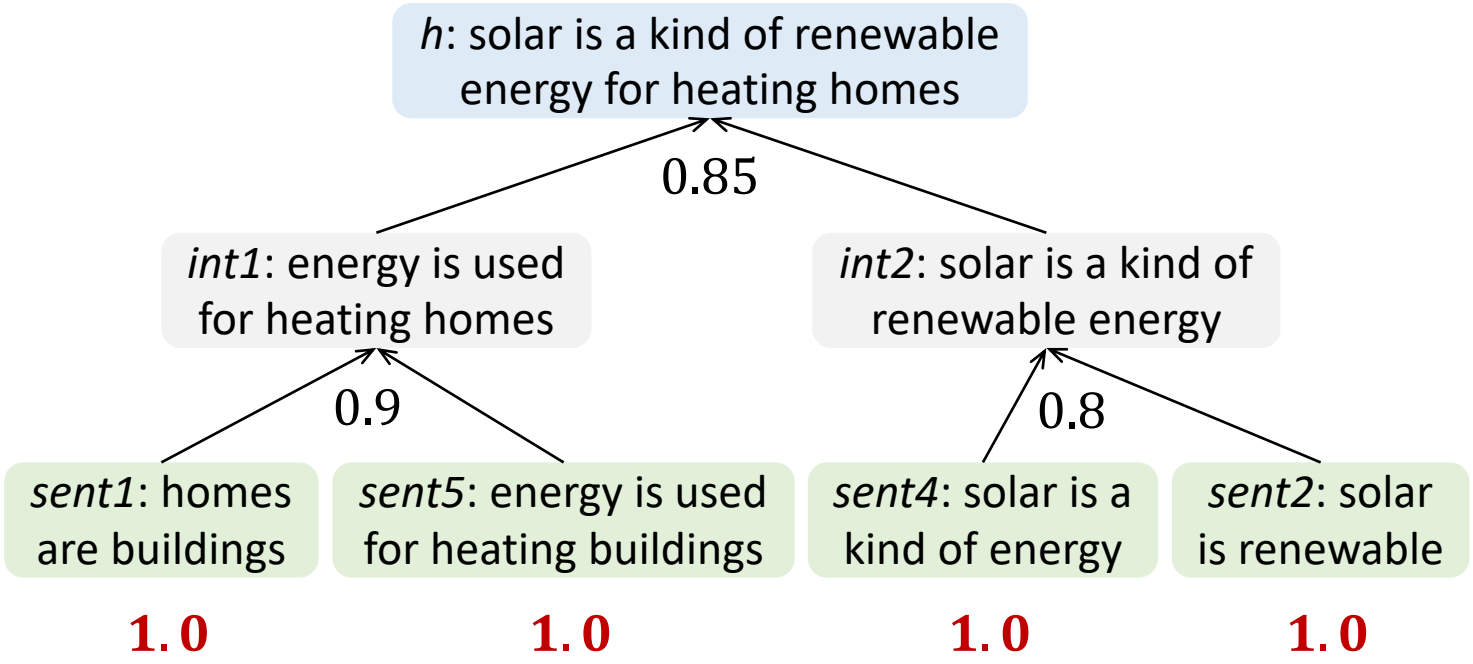


Verifier: Aggregating Scores for the Entire Proof



Verifier: Aggregating Scores for the Entire Proof

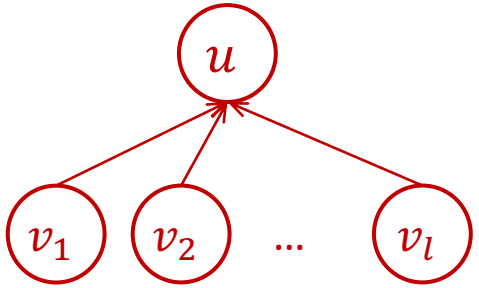
- Leaf nodes have a score of 1

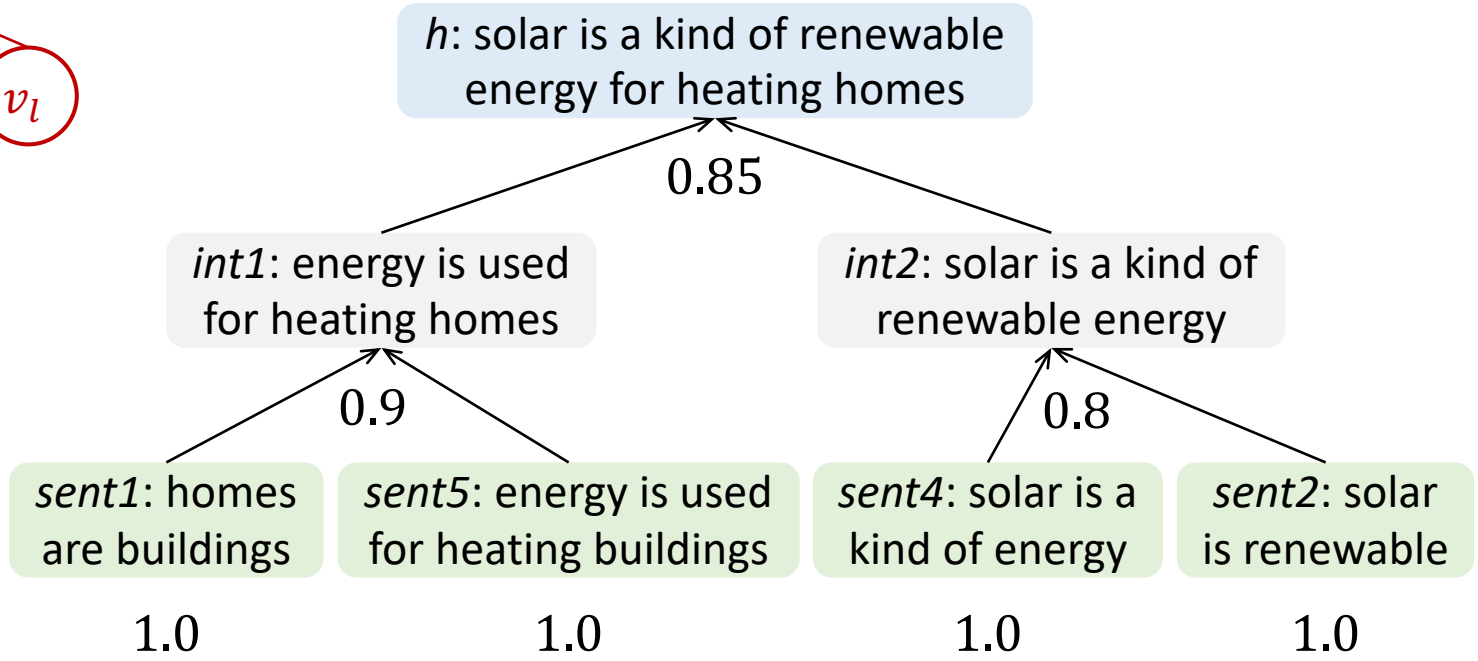


Verifier: Aggregating Scores for the Entire Proof

- Leaf nodes have a score of 1
- **Internal node u with l children $v_1, \dots, v_l$**

$$scr_n(u) = min(\underbrace{scr_s(s)}_{\text{Step score}}, \underbrace{scr_n(v_1), \dots, scr_n(v_l)}_{\text{Children's node scores}})$$

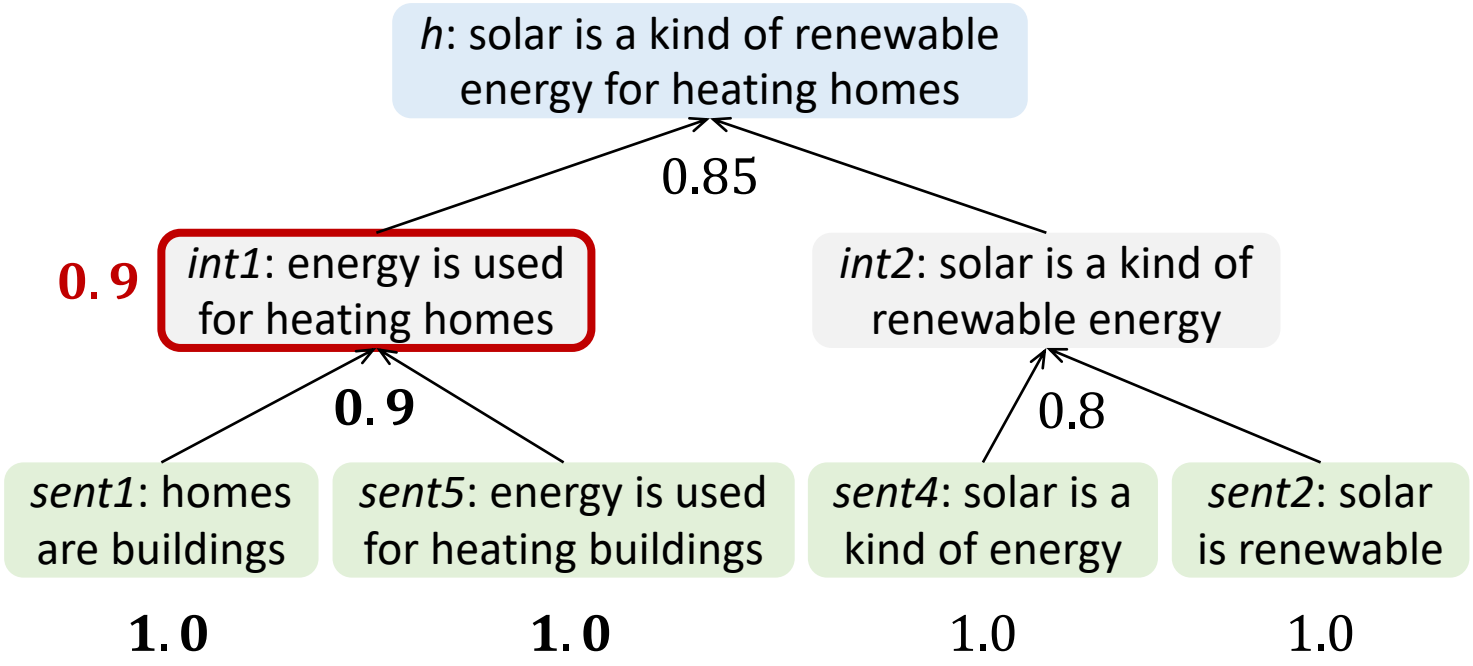




Verifier: Aggregating Scores for the Entire Proof

- Leaf nodes have a score of 1
- Internal node u with l children $v_1, \ldots, v_l$

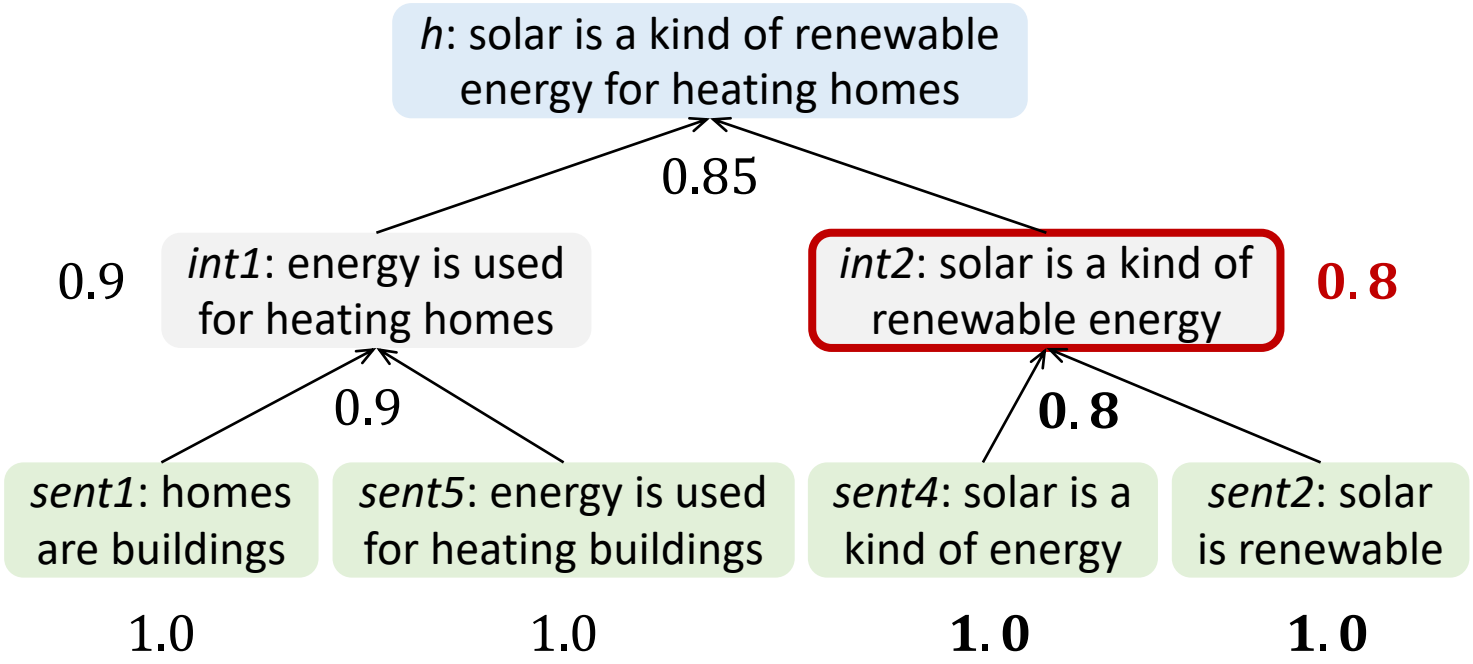
$$scr_n(u) = min(\underbrace{scr_s(s)}_{\text{Step score}}, \underbrace{scr_n(v_1), \ldots, scr_n(v_l)}_{\text{Children's node scores}})$$



Verifier: Aggregating Scores for the Entire Proof

- Leaf nodes have a score of 1
- Internal node u with l children $v_1, \ldots, v_l$

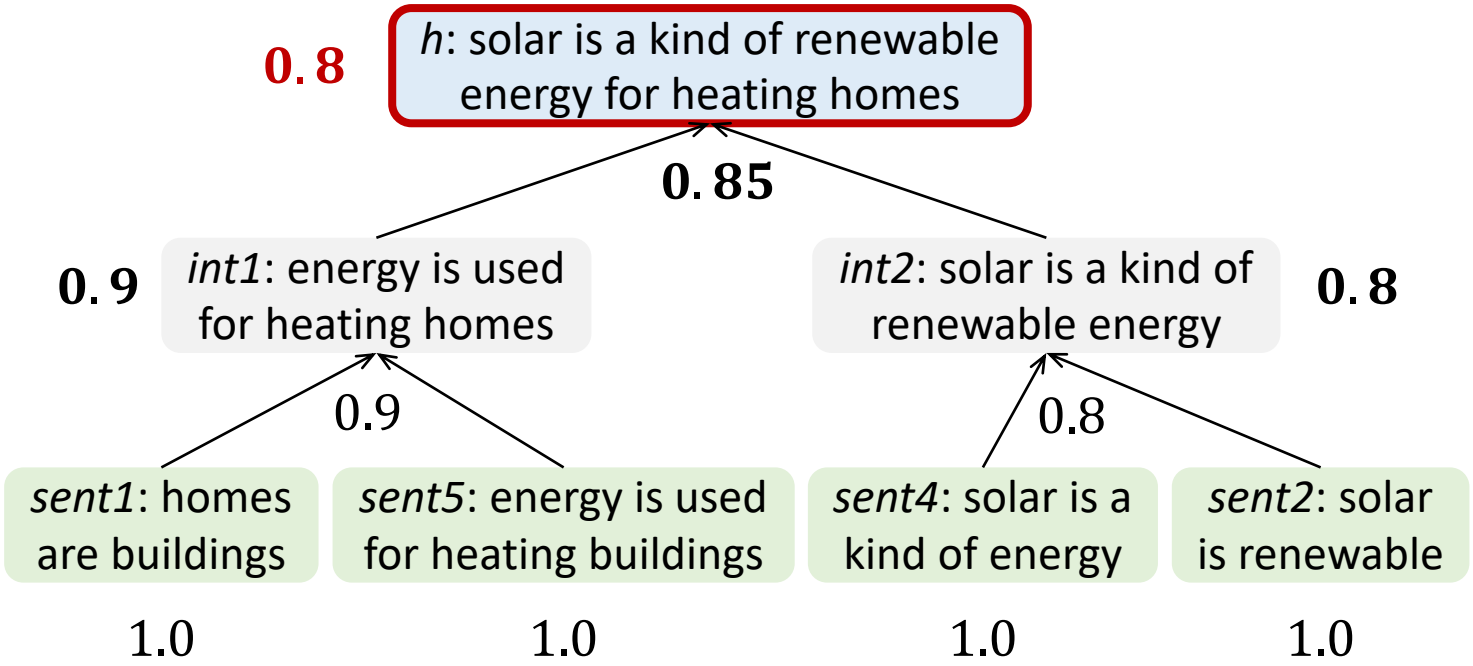
$$scr_n(u) = min(\underbrace{scr_s(s)}_{\text{Step score}}, \underbrace{scr_n(v_1), \ldots, scr_n(v_l)}_{\text{Children's node scores}})$$



Verifier: Aggregating Scores for the Entire Proof

- Leaf nodes have a score of 1
- Internal node u with l children $v_1, \ldots, v_l$

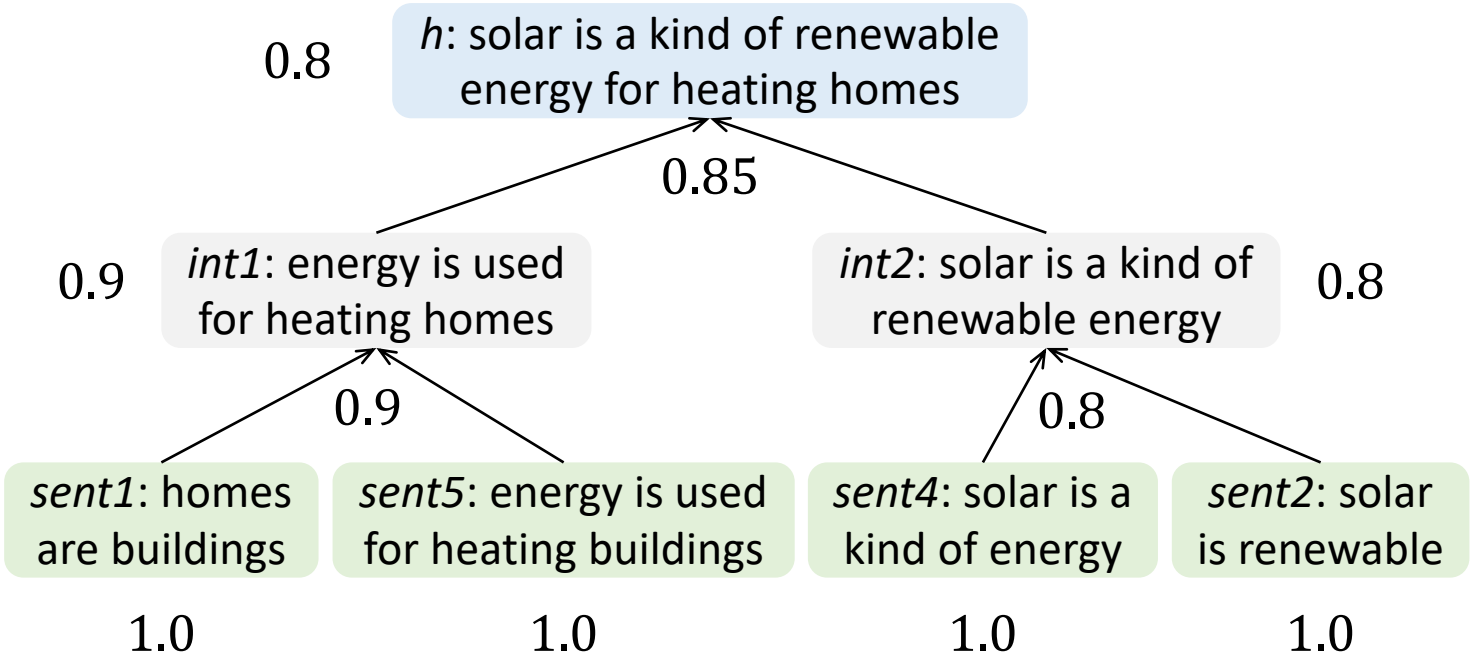
$$scr_n(u) = min(\underbrace{scr_s(s)}_{\text{Step score}}, \underbrace{scr_n(v_1), \ldots, scr_n(v_l)}_{\text{Children's node scores}})$$



Verifier: Aggregating Scores for the Entire Proof

- Leaf nodes have a score of 1
- Internal node u with l children $v_1, \dots, v_l$

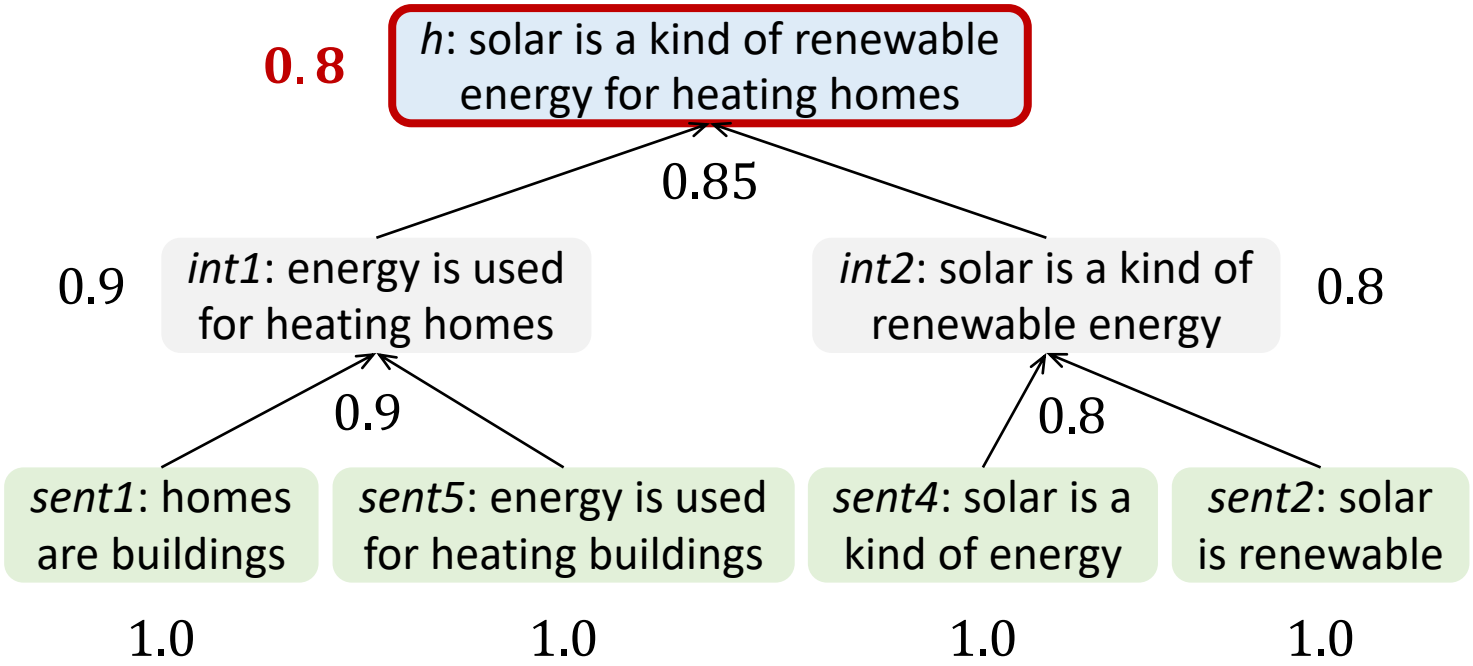
$$\boldsymbol{scr_n(u) = min(\underbrace{scr_s(s)}_{\text{Step score}}, \underbrace{scr_n(v_1), \dots, scr_n(v_l)}_{\text{Children's node scores}})}$$



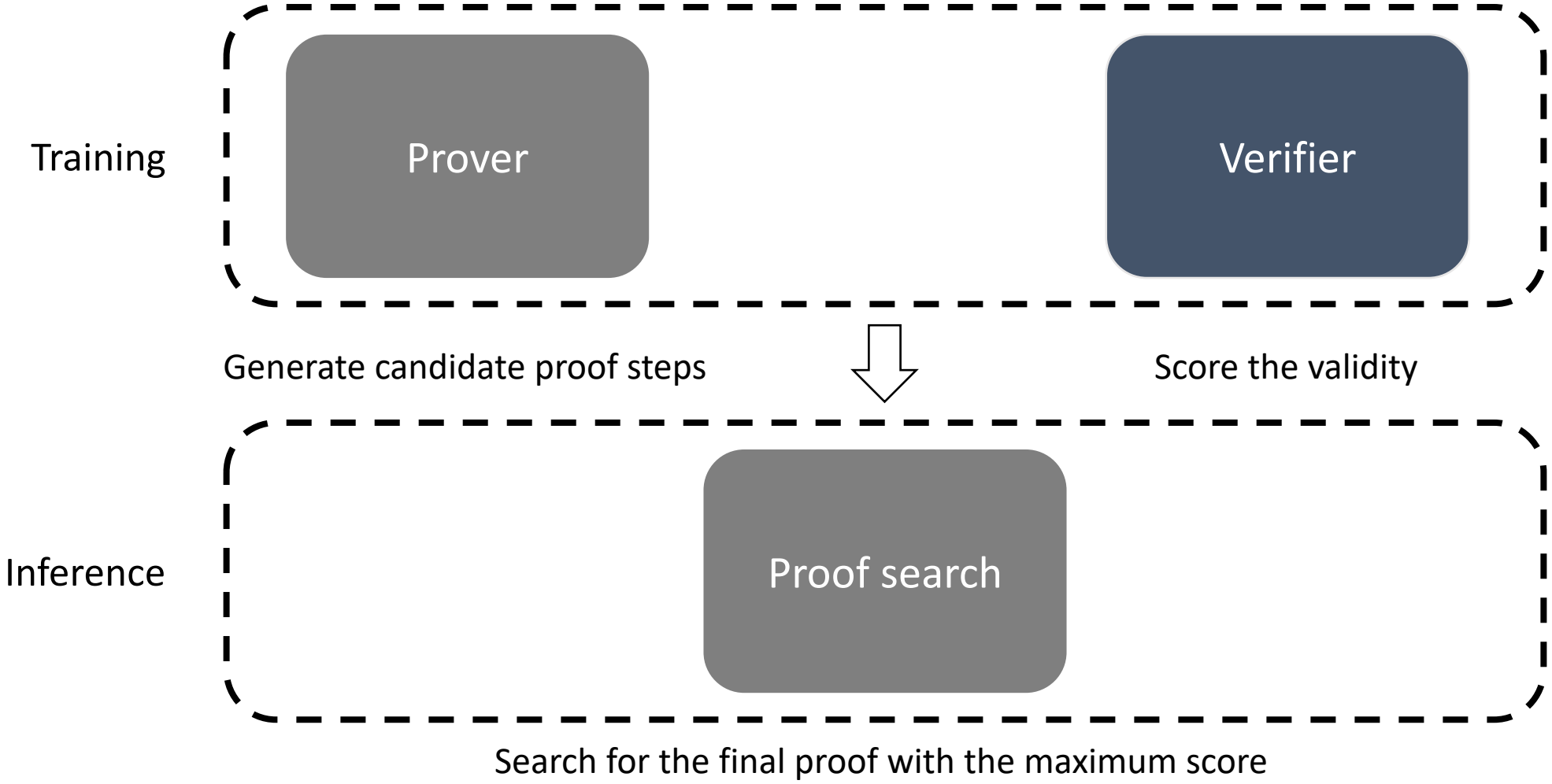
Verifier: Aggregating Scores for the Entire Proof

- Leaf nodes have a score of 1
- Internal node u with l children $v_1, \dots, v_l$
- **The proof score is the root's node score $\boldsymbol{scr_n(h)}$**

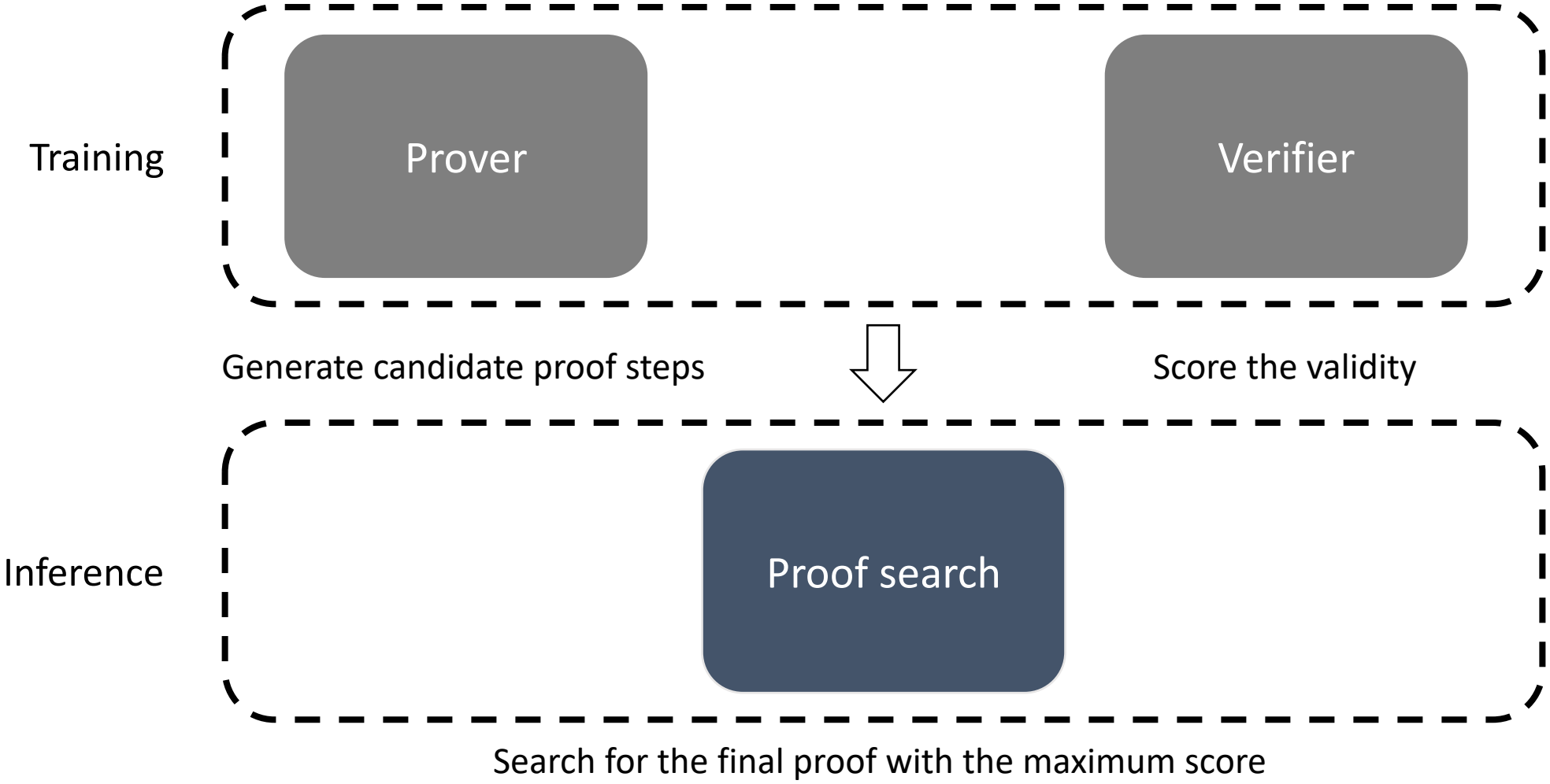
$$scr_n(u) = \min(\underbrace{scr_s(s)}_{\text{Step score}}, \underbrace{scr_n(v_1), \dots, scr_n(v_l)}_{\text{Children's node scores}})$$



Overview of NLProofS



Overview of NLProofS



Proof Search: Overview

- Iteratively expand a graph using proof steps proposed by the prover
- Score the nodes using both prover and the verifier
- Extract the proof with optimal score

Algorithm 1: Proof search.

Input : Hypothesis h, supporting facts C, stepwise prover $\mathcal{P}$, verifier $\mathcal{V}$

Output : Proof tree T

Proof Search: Initialization

- **Initialize the graph with a greedy proof generated by the prover alone**

Algorithm 1: Proof search.

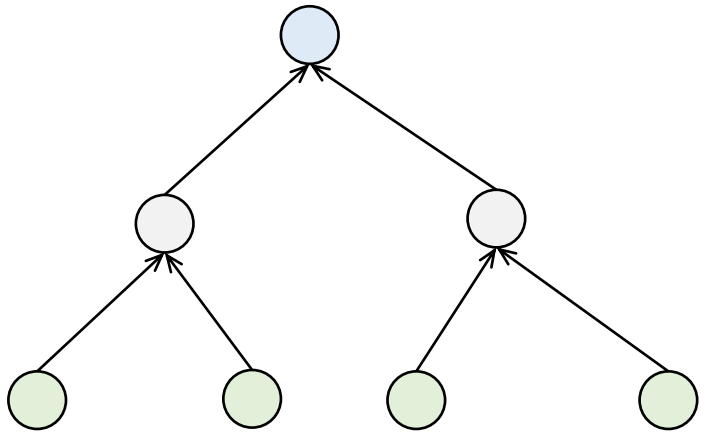
Input : Hypothesis h, supporting facts C, stepwise prover $\mathcal{P}$, verifier $\mathcal{V}$

Output : Proof tree T

$\mathcal{G} \leftarrow$ `generate_greedy`$(\mathcal{P}, h, C)$

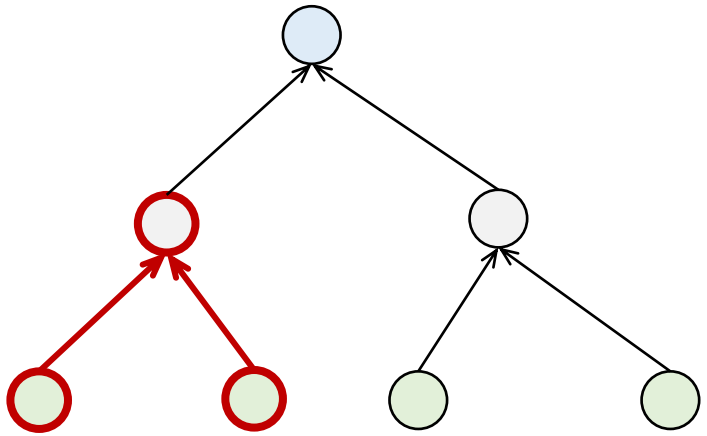
$\mathcal{PG} \leftarrow$ `initialize_graph`$(\mathcal{G})$

explored $\leftarrow \varnothing$



Proof Search: Iteration

- **Sample a partial proof from the graph**



Algorithm 1: Proof search.

Input : Hypothesis h, supporting facts C, stepwise prover $\mathcal{P}$, verifier $\mathcal{V}$

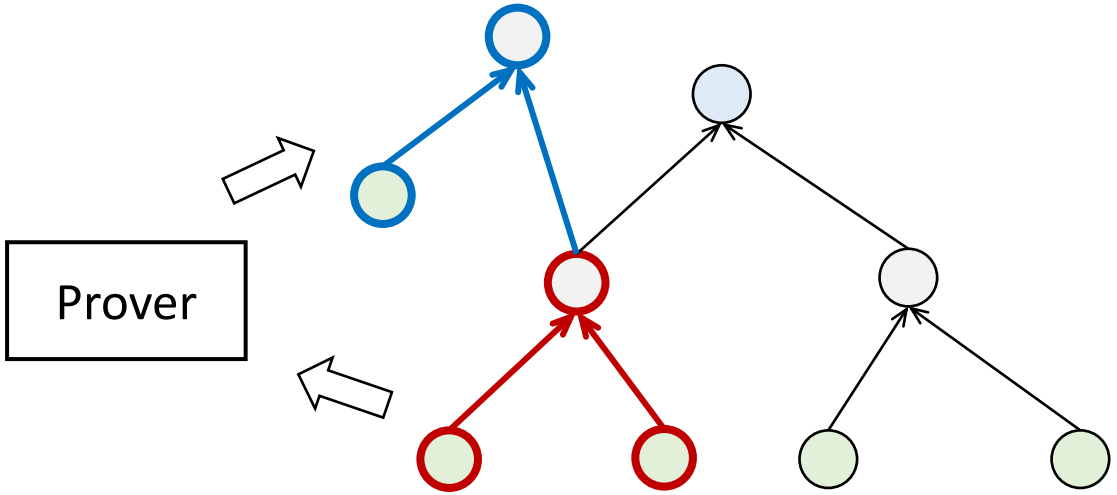
Output : Proof tree T

$\mathcal{G} \leftarrow \texttt{generate_greedy}(\mathcal{P}, h, C)$
$\mathcal{PG} \leftarrow \texttt{initialize_graph}(\mathcal{G})$
explored $\leftarrow \varnothing$
while *true* **do**
  partial_proof $\leftarrow \texttt{sample_new}(\mathcal{PG}, \text{explored})$
  explored $\leftarrow$ explored $\cup$ {partial_proof}
  steps, p_scrs $\leftarrow \texttt{generate}(\mathcal{P}, \text{partial_proof})$
  v_scrs $\leftarrow \texttt{verify}(\mathcal{V}, \text{steps})$
  scrs $\leftarrow$ (p_scrs + v_scrs)/2
  $\mathcal{PG}' \leftarrow \texttt{update}(\mathcal{PG}, \text{steps}, \text{scrs})$
  if $\mathcal{PG}' = \mathcal{PG}$ **then**
    break
  $\mathcal{PG} \leftarrow \mathcal{PG}'$

Proof Search: Iteration

- Use the prover to generate proof steps

  Prover score: likelihood scores in beam search



Algorithm 1: Proof search.

Input : Hypothesis h, supporting facts C, stepwise prover $\mathcal{P}$, verifier $\mathcal{V}$

Output : Proof tree T

$\mathcal{G} \leftarrow \texttt{generate_greedy}(\mathcal{P}, h, C)$

$\mathcal{PG} \leftarrow \texttt{initialize_graph}(\mathcal{G})$

explored $\leftarrow \varnothing$

while *true* **do**

  partial_proof $\leftarrow \texttt{sample_new}(\mathcal{PG}, \text{explored})$

  explored $\leftarrow$ explored $\cup$ {partial_proof}

  steps, p_scrs $\leftarrow \texttt{generate}(\mathcal{P}, \text{partial_proof})$

  v_scrs $\leftarrow \texttt{verify}(\mathcal{V}, \text{steps})$

  scrs $\leftarrow$ (p_scrs + v_scrs)/2

  $\mathcal{PG}' \leftarrow \texttt{update}(\mathcal{PG}, \text{steps}, \text{scrs})$

  if $\mathcal{PG}' = \mathcal{PG}$ **then**

    break

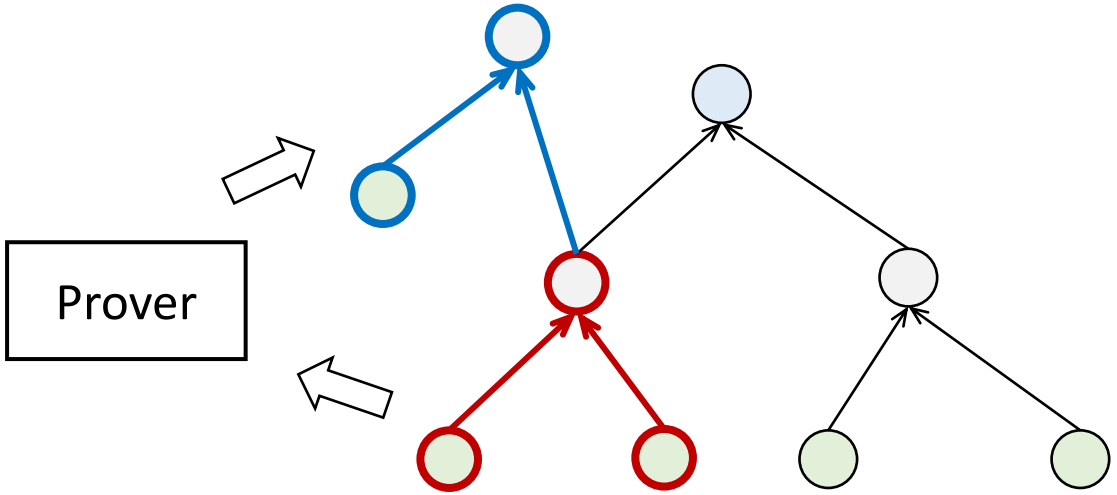
  $\mathcal{PG} \leftarrow \mathcal{PG}'$

Proof Search: Iteration

- Use the prover to generate proof steps

Prover score: likelihood scores in beam search

Step score: (prover score + verifier score) / 2



Algorithm 1: Proof search.

Input : Hypothesis h, supporting facts C, stepwise prover $\mathcal{P}$, verifier $\mathcal{V}$

Output : Proof tree T

$\mathcal{G} \leftarrow \texttt{generate_greedy}(\mathcal{P}, h, C)$

$\mathcal{PG} \leftarrow \texttt{initialize_graph}(\mathcal{G})$

explored $\leftarrow \varnothing$

while *true* **do**

  partial_proof $\leftarrow \texttt{sample_new}(\mathcal{PG}, \text{explored})$

  explored $\leftarrow$ explored $\cup$ {partial_proof}

  steps, p_scrs $\leftarrow \texttt{generate}(\mathcal{P}, \text{partial_proof})$

  v_scrs $\leftarrow \texttt{verify}(\mathcal{V}, \text{steps})$

  scrs $\leftarrow$ (p_scrs + v_scrs)/2

  $\mathcal{PG}' \leftarrow \texttt{update}(\mathcal{PG}, \text{steps}, \text{scrs})$

  if $\mathcal{PG}' = \mathcal{PG}$ **then**

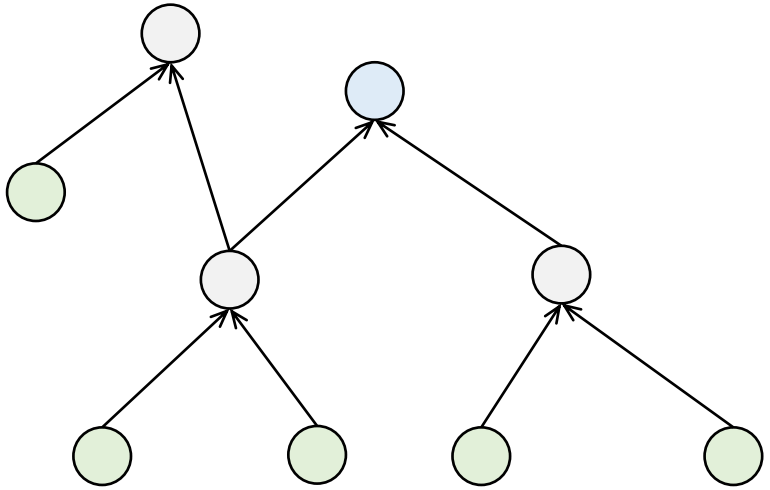
    break

  $\mathcal{PG} \leftarrow \mathcal{PG}'$

Proof Search: Iteration

- Iteratively expand the graph

  Prover score: likelihood scores in beam search
  Step score: (prover score + verifier score) / 2



Algorithm 1: Proof search.

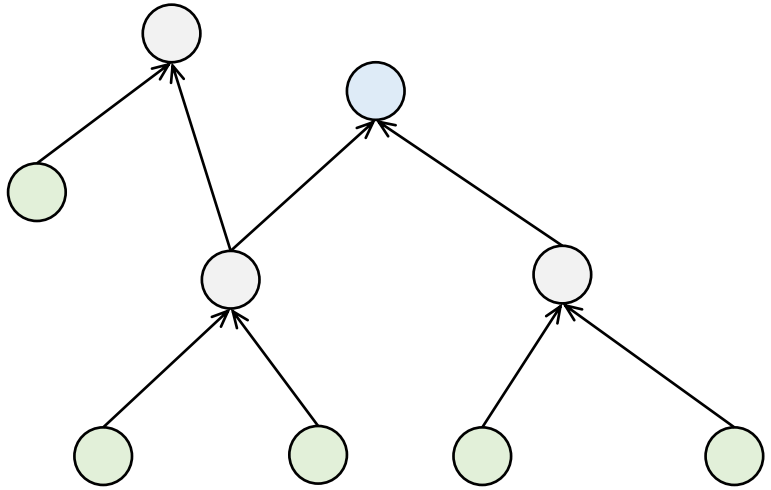
Input : Hypothesis h, supporting facts C, stepwise prover $\mathcal{P}$, verifier $\mathcal{V}$
Output : Proof tree T

$\mathcal{G} \leftarrow \texttt{generate_greedy}(\mathcal{P}, h, C)$
$\mathcal{PG} \leftarrow \texttt{initialize_graph}(\mathcal{G})$
explored $\leftarrow \varnothing$
while *true* **do**
  partial_proof $\leftarrow \texttt{sample_new}(\mathcal{PG}, \text{explored})$
  explored $\leftarrow$ explored $\cup$ {partial_proof}
  steps, p_scrs $\leftarrow \texttt{generate}(\mathcal{P}, \text{partial_proof})$
  v_scrs $\leftarrow \texttt{verify}(\mathcal{V}, \text{steps})$
  scrs $\leftarrow$ (p_scrs + v_scrs)/2
  $\mathcal{PG}' \leftarrow \texttt{update}(\mathcal{PG}, \text{steps}, \text{scrs})$
  if $\mathcal{PG}' = \mathcal{PG}$ **then**
    break
  $\mathcal{PG} \leftarrow \mathcal{PG}'$

Proof Search: Iteration

- Iteratively expand the graph

  Prover score: likelihood scores in beam search
  Step score: (prover score + verifier score) / 2



Algorithm 1: Proof search.

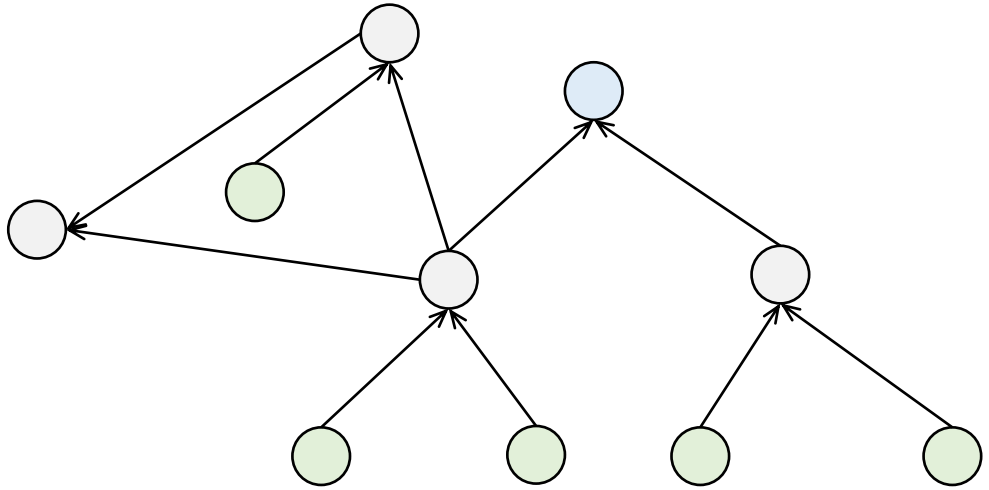
Input : Hypothesis h, supporting facts C, stepwise prover $\mathcal{P}$, verifier $\mathcal{V}$
Output : Proof tree T

$\mathcal{G} \leftarrow \texttt{generate_greedy}(\mathcal{P}, h, C)$
$\mathcal{PG} \leftarrow \texttt{initialize_graph}(\mathcal{G})$
explored $\leftarrow \varnothing$
while *true* **do**
  partial_proof $\leftarrow \texttt{sample_new}(\mathcal{PG}, \text{explored})$
  explored $\leftarrow$ explored $\cup$ {partial_proof}
  steps, p_scrs $\leftarrow \texttt{generate}(\mathcal{P}, \text{partial_proof})$
  v_scrs $\leftarrow \texttt{verify}(\mathcal{V}, \text{steps})$
  scrs $\leftarrow$ (p_scrs + v_scrs)/2
  $\mathcal{PG}' \leftarrow \texttt{update}(\mathcal{PG}, \text{steps}, \text{scrs})$
  if $\mathcal{PG}' = \mathcal{PG}$ **then**
    break
  $\mathcal{PG} \leftarrow \mathcal{PG}'$

Proof Search: Iteration

- Iteratively expand the graph

  Prover score: likelihood scores in beam search
  Step score: (prover score + verifier score) / 2



Algorithm 1: Proof search.

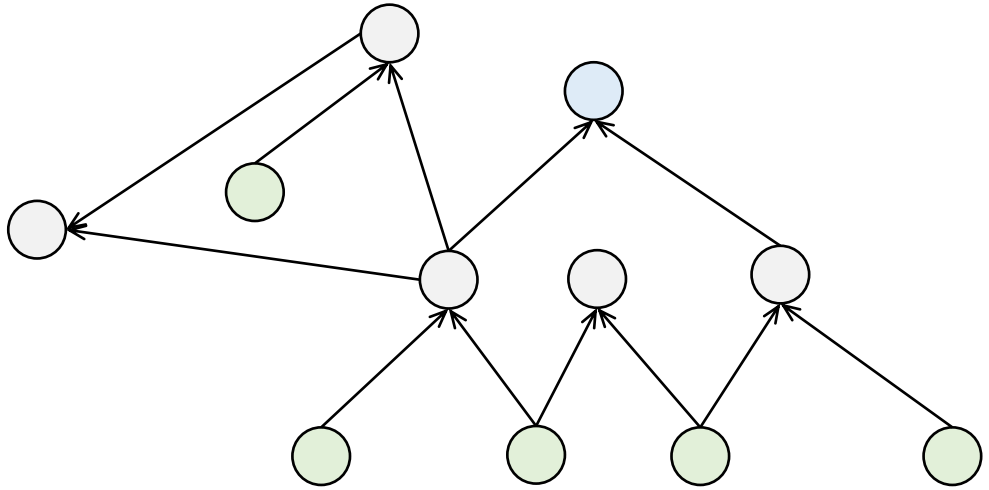
Input : Hypothesis h, supporting facts C, stepwise prover $\mathcal{P}$, verifier $\mathcal{V}$
Output : Proof tree T

$\mathcal{G} \leftarrow \texttt{generate_greedy}(\mathcal{P}, h, C)$
$\mathcal{PG} \leftarrow \texttt{initialize_graph}(\mathcal{G})$
explored $\leftarrow \varnothing$
while *true* **do**
  partial_proof $\leftarrow \texttt{sample_new}(\mathcal{PG}, \text{explored})$
  explored $\leftarrow$ explored $\cup$ {partial_proof}
  steps, p_scrs $\leftarrow \texttt{generate}(\mathcal{P}, \text{partial_proof})$
  v_scrs $\leftarrow \texttt{verify}(\mathcal{V}, \text{steps})$
  scrs $\leftarrow$ (p_scrs + v_scrs)/2
  $\mathcal{PG}' \leftarrow \texttt{update}(\mathcal{PG}, \text{steps}, \text{scrs})$
  if $\mathcal{PG}' = \mathcal{PG}$ **then**
    break
  $\mathcal{PG} \leftarrow \mathcal{PG}'$

Proof Search: Iteration

- Iteratively expand the graph

  Prover score: likelihood scores in beam search
  Step score: (prover score + verifier score) / 2



Algorithm 1: Proof search.

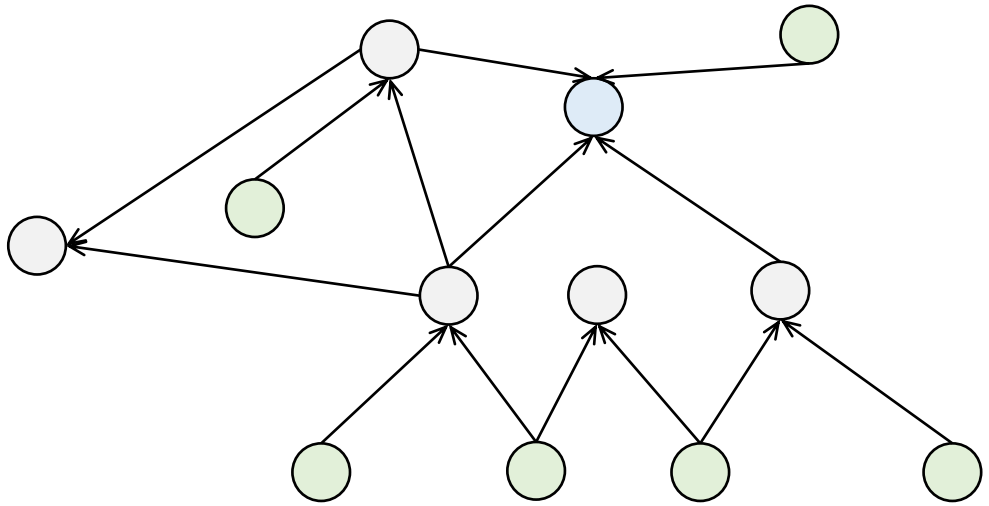
Input : Hypothesis h, supporting facts C, stepwise prover $\mathcal{P}$, verifier $\mathcal{V}$
Output : Proof tree T

$\mathcal{G} \leftarrow \texttt{generate_greedy}(\mathcal{P}, h, C)$
$\mathcal{PG} \leftarrow \texttt{initialize_graph}(\mathcal{G})$
explored $\leftarrow \varnothing$
while *true* **do**
  partial_proof $\leftarrow \texttt{sample_new}(\mathcal{PG}, \text{explored})$
  explored $\leftarrow$ explored $\cup$ {partial_proof}
  steps, p_scrs $\leftarrow \texttt{generate}(\mathcal{P}, \text{partial_proof})$
  v_scrs $\leftarrow \texttt{verify}(\mathcal{V}, \text{steps})$
  scrs $\leftarrow$ (p_scrs + v_scrs)/2
  $\mathcal{PG}' \leftarrow \texttt{update}(\mathcal{PG}, \text{steps}, \text{scrs})$
  if $\mathcal{PG}' = \mathcal{PG}$ **then**
    break
  $\mathcal{PG} \leftarrow \mathcal{PG}'$

Proof Search: Iteration

- Iteratively expand the graph

Prover score: likelihood scores in beam search
Step score: (prover score + verifier score) / 2



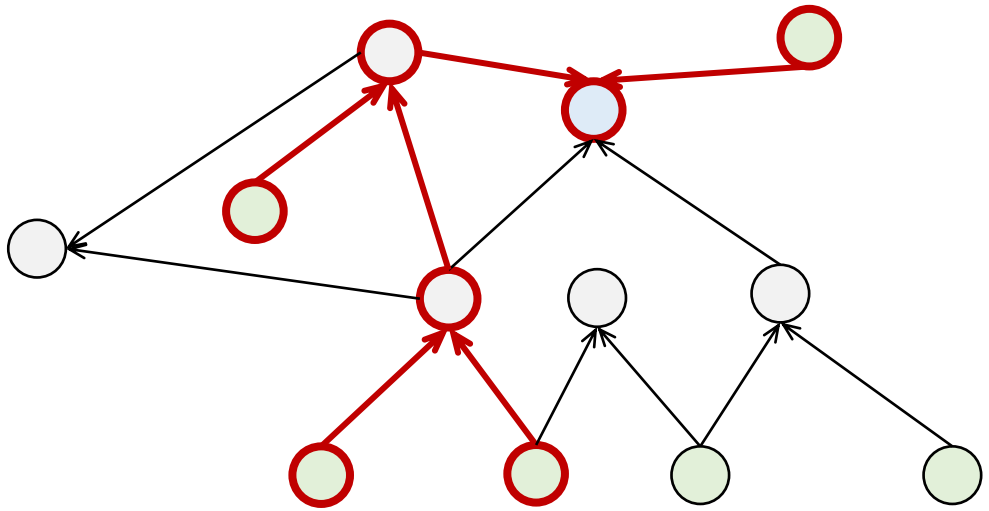
Algorithm 1: Proof search.

Input : Hypothesis h, supporting facts C, stepwise prover $\mathcal{P}$, verifier $\mathcal{V}$
Output : Proof tree T

$\mathcal{G} \leftarrow \texttt{generate_greedy}(\mathcal{P}, h, C)$
$\mathcal{PG} \leftarrow \texttt{initialize_graph}(\mathcal{G})$
explored $\leftarrow \varnothing$
while *true* **do**
  partial_proof $\leftarrow \texttt{sample_new}(\mathcal{PG}, \text{explored})$
  explored $\leftarrow$ explored $\cup$ {partial_proof}
  steps, p_scrs $\leftarrow \texttt{generate}(\mathcal{P}, \text{partial_proof})$
  v_scrs $\leftarrow \texttt{verify}(\mathcal{V}, \text{steps})$
  scrs $\leftarrow$ (p_scrs + v_scrs)/2
  $\mathcal{PG}' \leftarrow \texttt{update}(\mathcal{PG}, \text{steps}, \text{scrs})$
  if $\mathcal{PG}' = \mathcal{PG}$ **then**
    break
  $\mathcal{PG} \leftarrow \mathcal{PG}'$

Proof Search: Extraction

- **Extract a proof tree with optimal score**



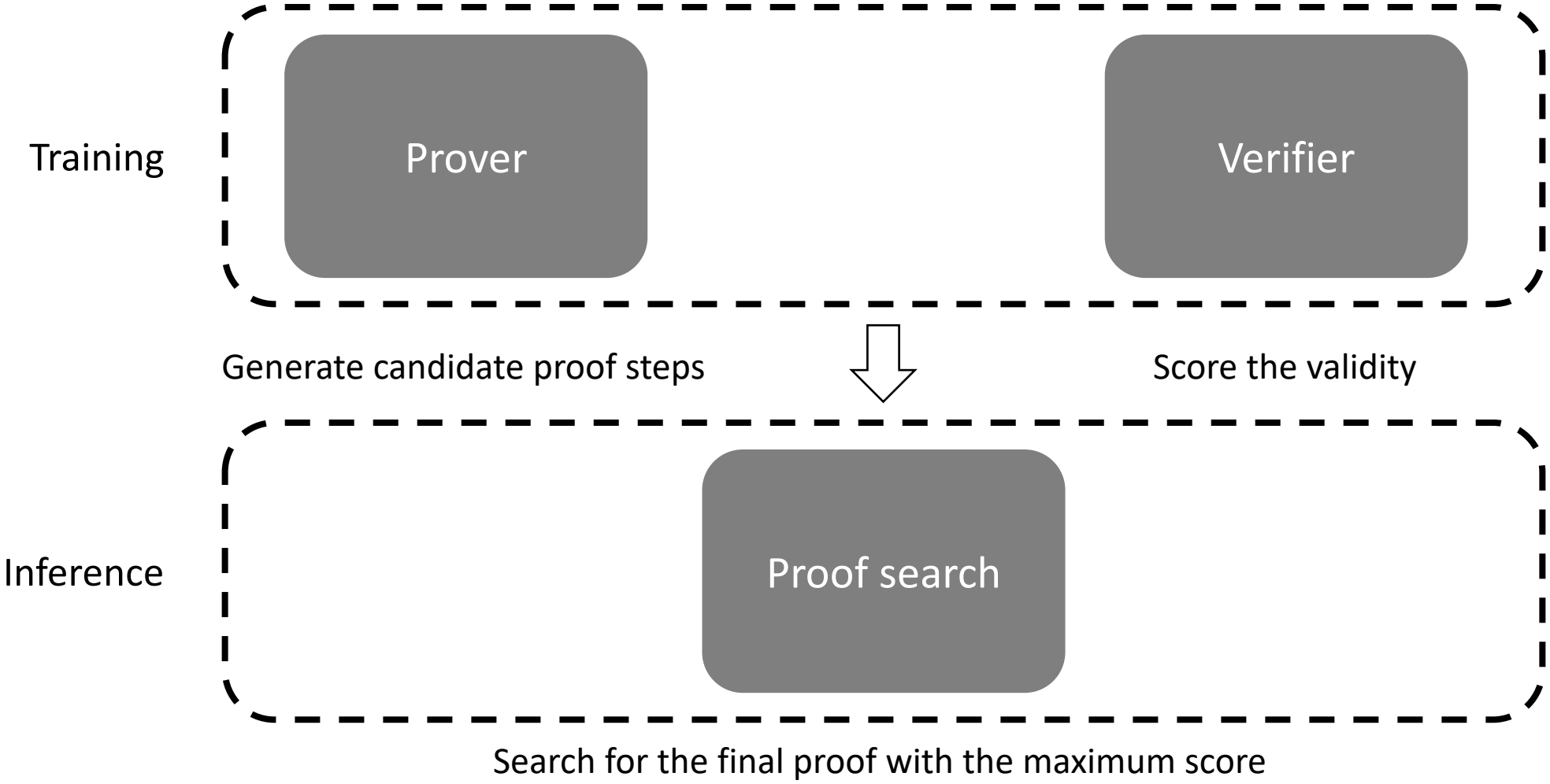
Algorithm 1: Proof search.

Input : Hypothesis h, supporting facts C, stepwise prover $\mathcal{P}$, verifier $\mathcal{V}$

Output : Proof tree T

$\mathcal{G} \leftarrow \texttt{generate_greedy}(\mathcal{P}, h, C)$
$\mathcal{PG} \leftarrow \texttt{initialize_graph}(\mathcal{G})$
explored $\leftarrow \varnothing$
while *true* **do**
  partial_proof $\leftarrow \texttt{sample_new}(\mathcal{PG}, \text{explored})$
  explored $\leftarrow$ explored $\cup$ {partial_proof}
  steps, p_scrs $\leftarrow \texttt{generate}(\mathcal{P}, \text{partial_proof})$
  v_scrs $\leftarrow \texttt{verify}(\mathcal{V}, \text{steps})$
  scrs $\leftarrow$ (p_scrs + v_scrs)/2
  $\mathcal{PG}' \leftarrow \texttt{update}(\mathcal{PG}, \text{steps}, \text{scrs})$
  if $\mathcal{PG}' = \mathcal{PG}$ **then**
    break
  $\mathcal{PG} \leftarrow \mathcal{PG}'$
return $\texttt{extract_proof}(\mathcal{PG})$

Overview of NLProofS



Experiments

Experiments Overview

- Evaluate on two benchmarks
  - **EntailmentBank**: Challenging, human-written proofs [Tafjord et al. Findings of ACL 2021]
  - **RuleTaker**: Simple, synthetic proofs [Dalvi et al. EMNLP 2021]
- State-of-the-art results on both
- Ablations highlight the importance of the verifier

Experiments Overview

- Evaluate on two benchmarks
  - **EntailmentBank**: Challenging, human-written proofs [Tafjord et al. Findings of ACL 2021]
  - **RuleTaker**: Simple, synthetic proofs [Dalvi et al. EMNLP 2021]
- State-of-the-art results on both
- Ablations highlight the importance of the verifier

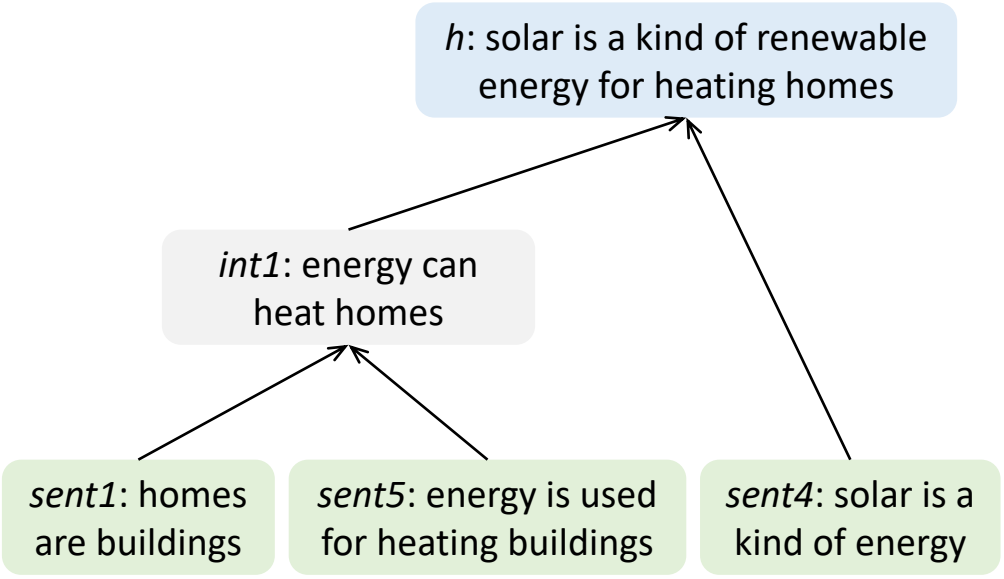
[Dalvi et al. EMNLP 2021]

EntailmentBank

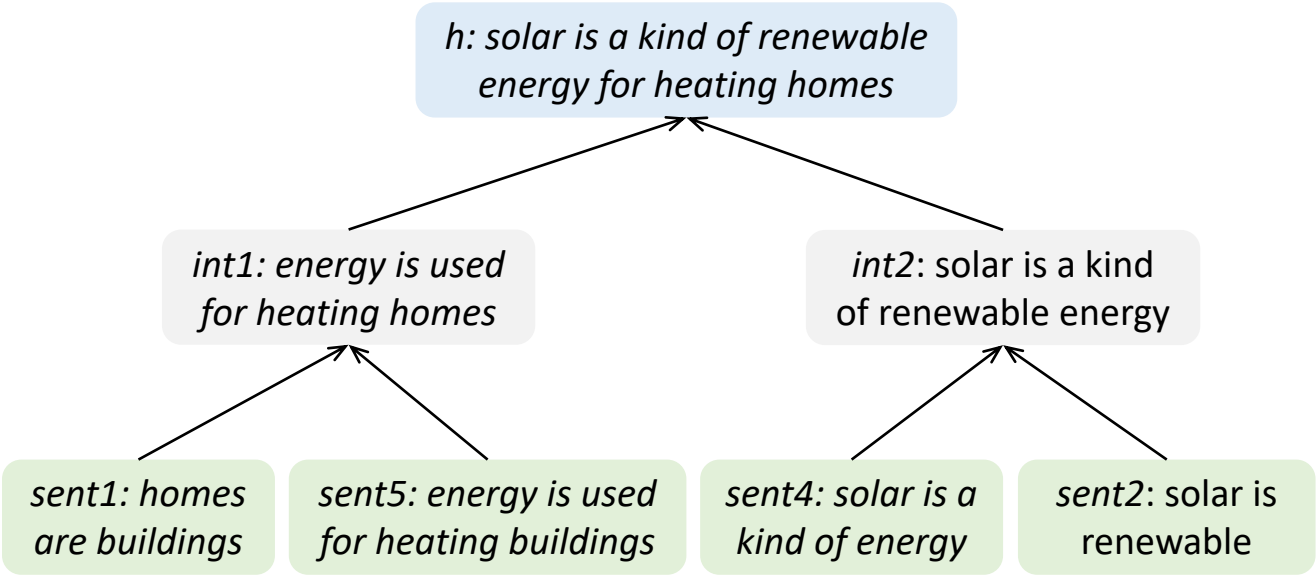
- 1,840 proof trees constructed by experts: 1,313/187/340 for training/validation/testing
- How to compare a predicted proof tree against the ground truth?
  - Four official evaluation : Leaves, Steps, Intermediates, Overall

[Dalvi et al. EMNLP 2021]

EntailmentBank: Evaluation Metrics



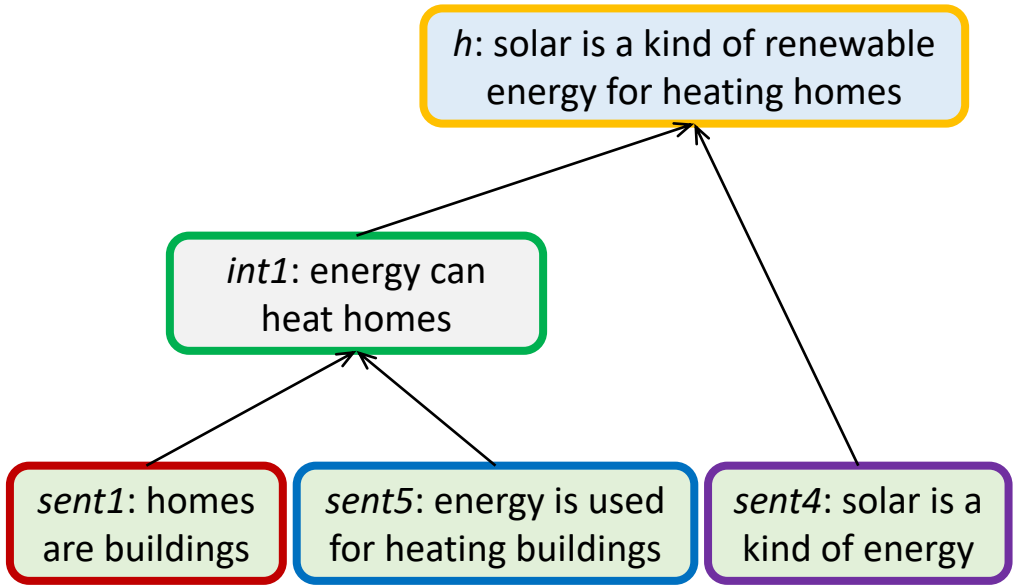
Predicted proof tree



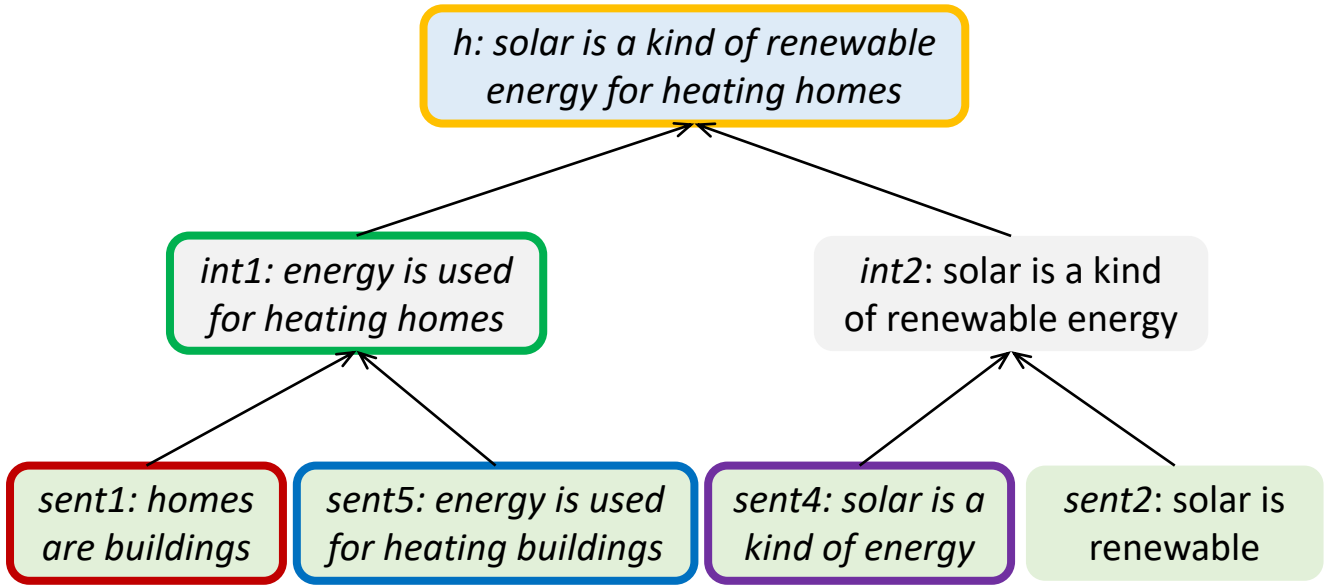
Ground truth proof tree

[Dalvi et al. EMNLP 2021]

EntailmentBank: Evaluation Metrics



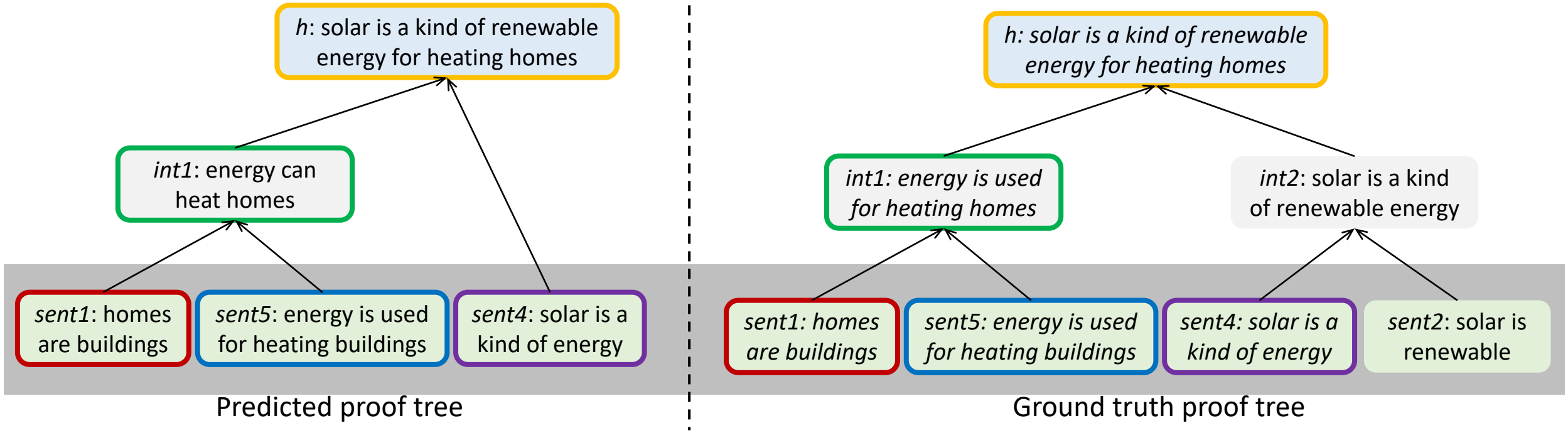
Predicted proof tree



Ground truth proof tree

[Dalvi et al. EMNLP 2021]

EntailmentBank: Evaluation Metrics

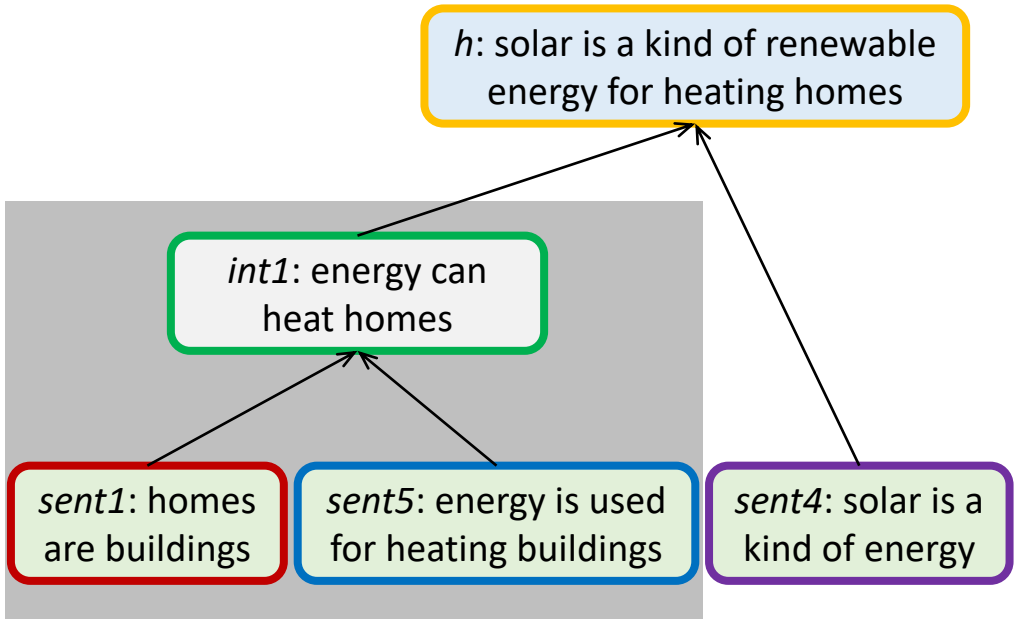


- Leaves: Accuracy and F1 score of the predicted leaf nodes

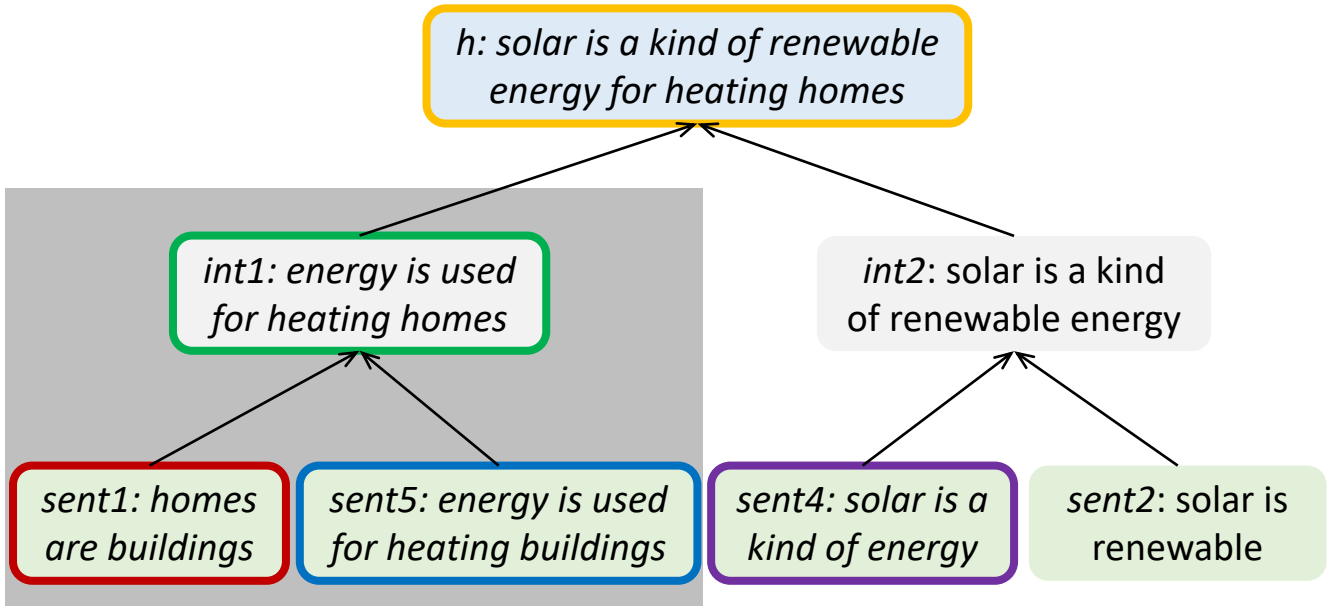
$$accuracy = 0 \qquad precision = \frac{3}{3} = 1 \qquad recall = \frac{3}{4} = 0.75 \qquad F1 = 1/\frac{1+\frac{4}{3}}{2} \approx 0.86$$

[Dalvi et al. EMNLP 2021]

EntailmentBank: Evaluation Metrics



Predicted proof tree

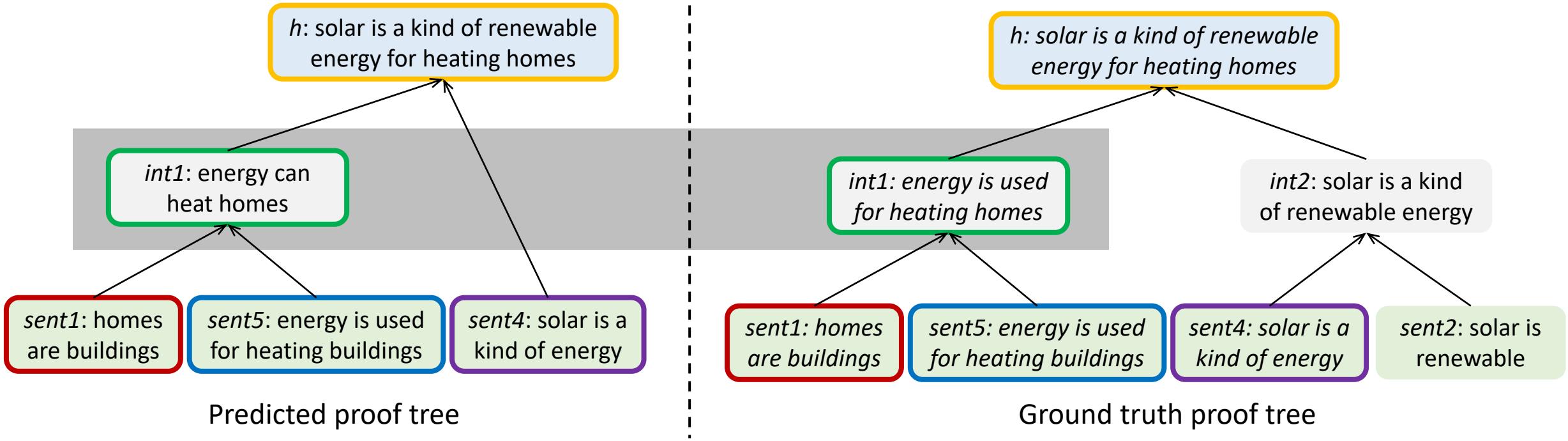


Ground truth proof tree

- Leaves: Accuracy and F1 score of the predicted leaf nodes
- **Steps: Proof steps are "structurally correct"**

[Dalvi et al. EMNLP 2021]

EntailmentBank: Evaluation Metrics

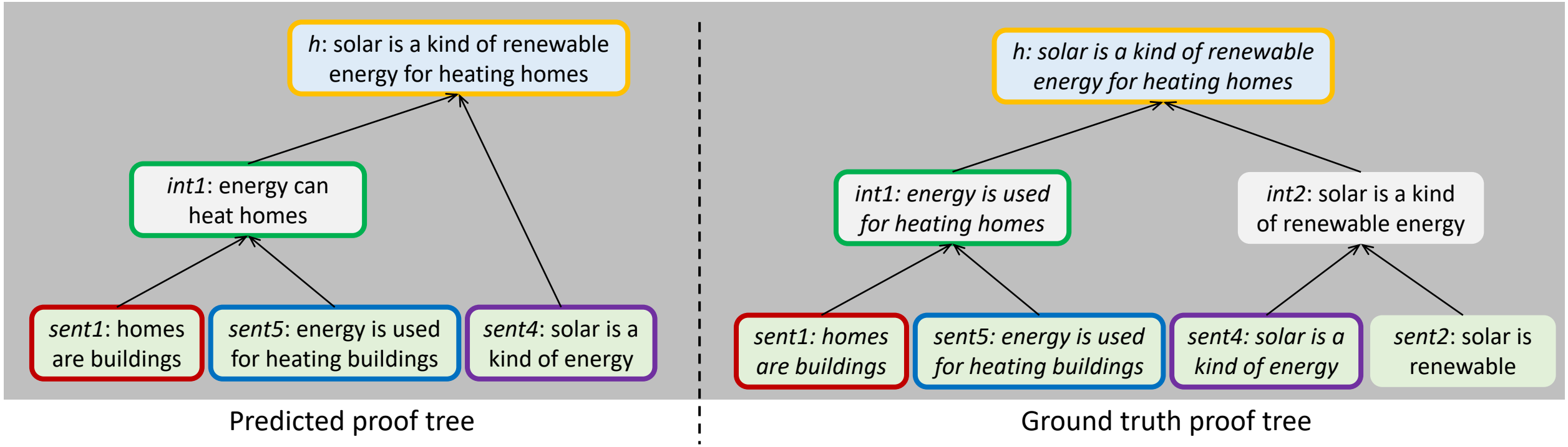


- Leaves: Accuracy and F1 score of the predicted leaf nodes
- Steps: Proof steps are "structurally correct"
- **Intermediates: Intermediate conclusions match, judged by the BLEURT score**

[Sellam et al. ACL 2020]

[Dalvi et al. EMNLP 2021]

EntailmentBank: Evaluation Metrics

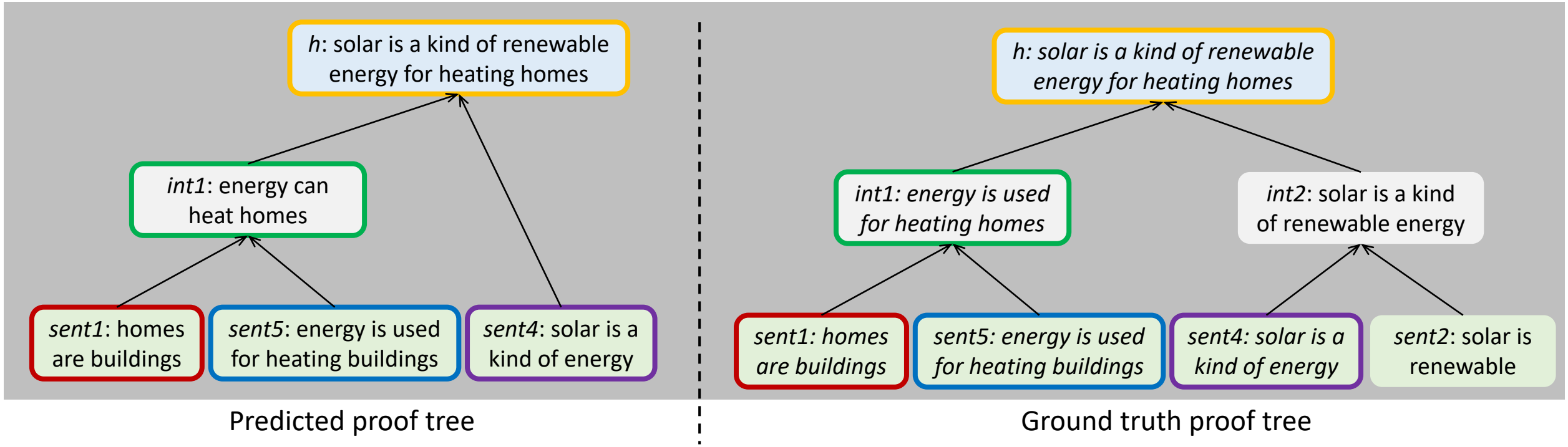


- Leaves: Accuracy and F1 score of the predicted leaf nodes
- Steps: Proof steps are "structurally correct"
- Intermediates: Intermediate conclusions match, judged by the BLEURT score
- **Overall: The entire proof is correct**

[Sellam et al. ACL 2020]

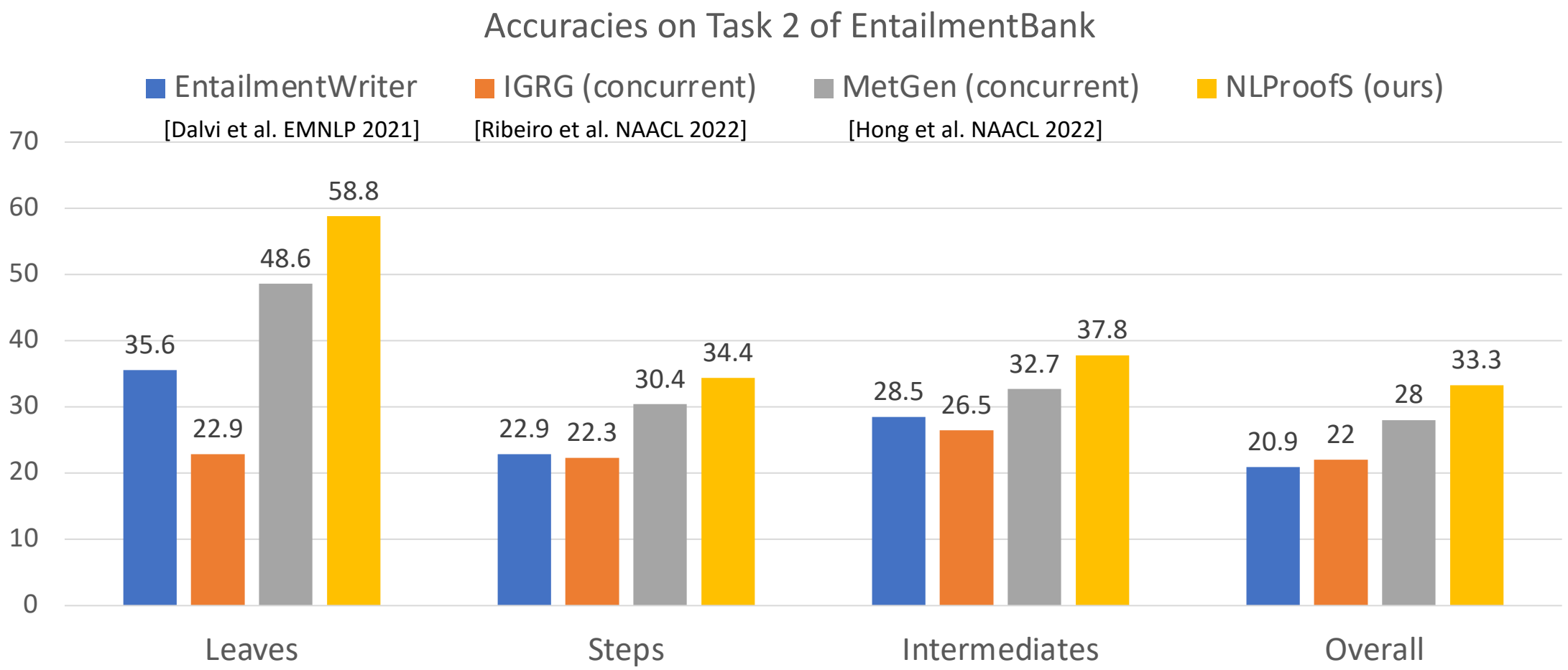
[Dalvi et al. EMNLP 2021]

EntailmentBank: Evaluation Metrics

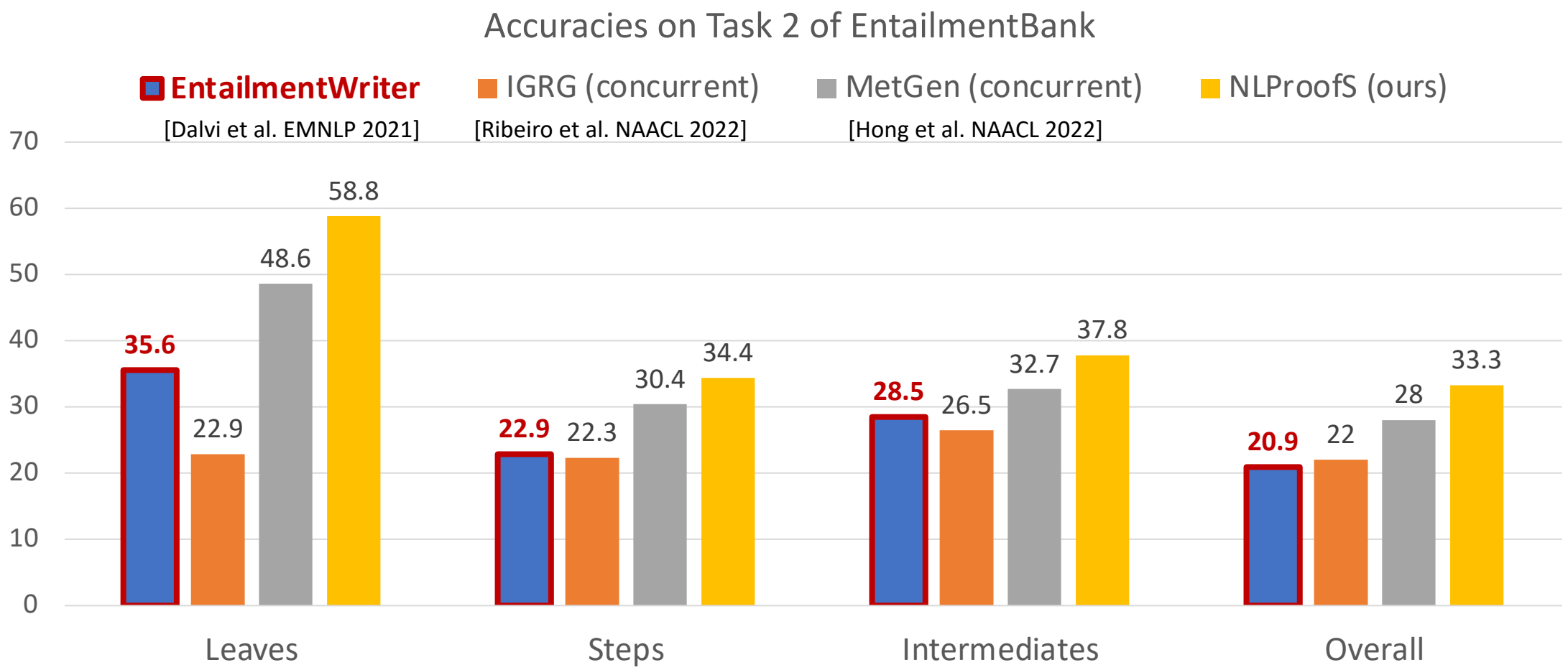


- **Limitations: Does not handle correct predictions that are different from the ground truth**

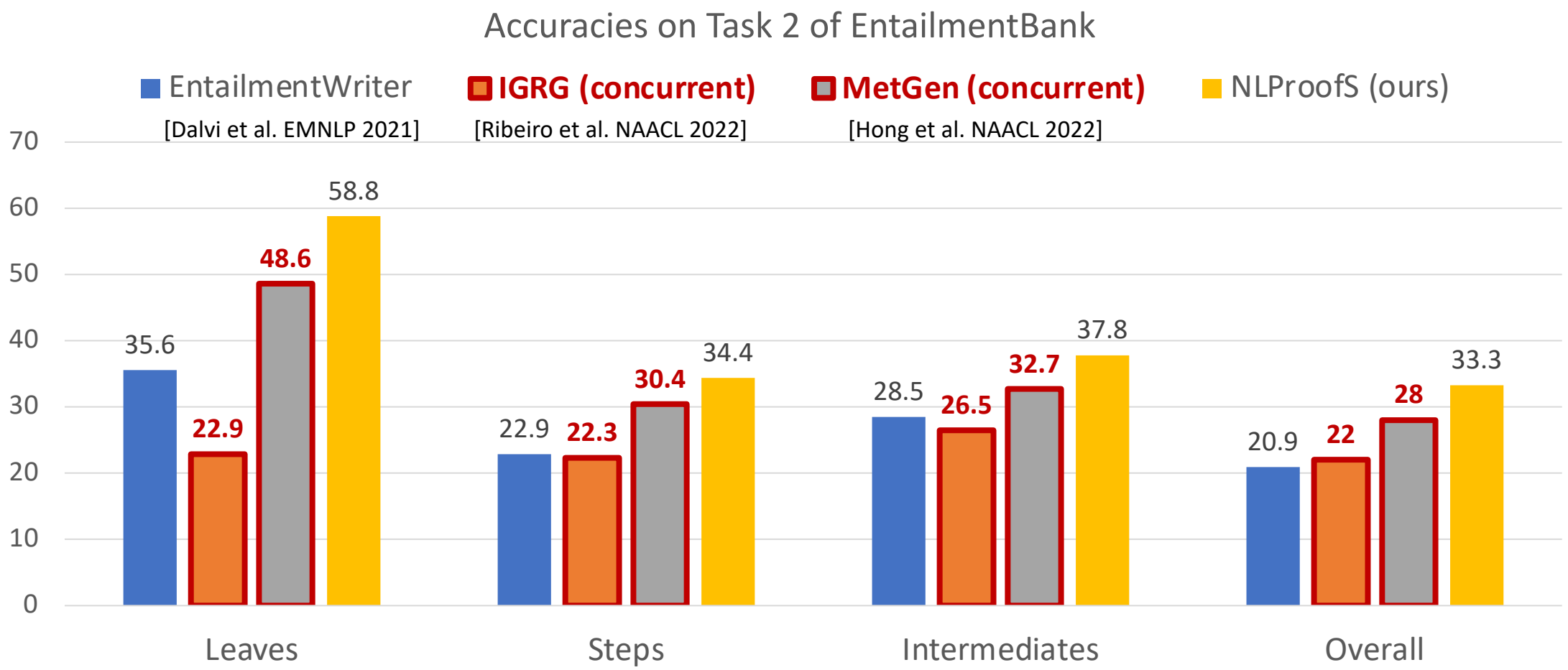
State-of-the-art Performance on EntailmentBank



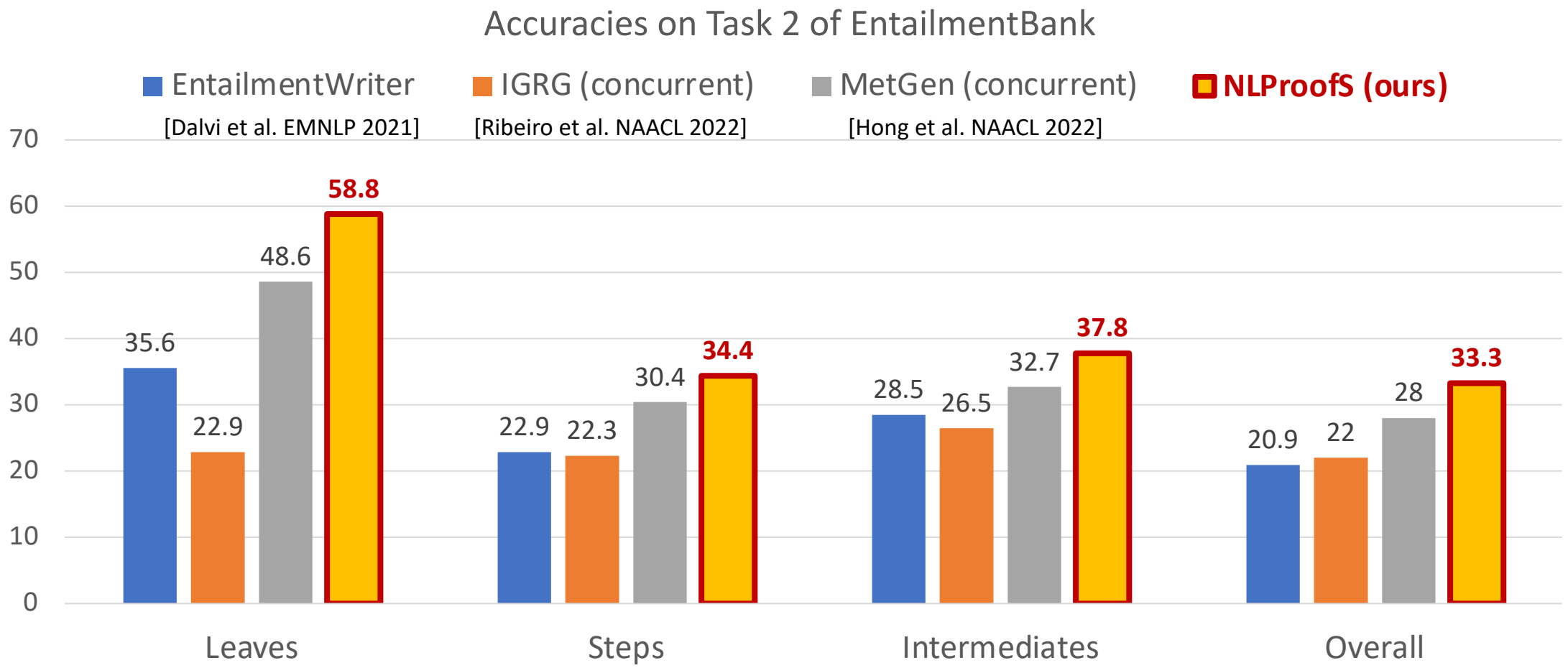
State-of-the-art Performance on EntailmentBank



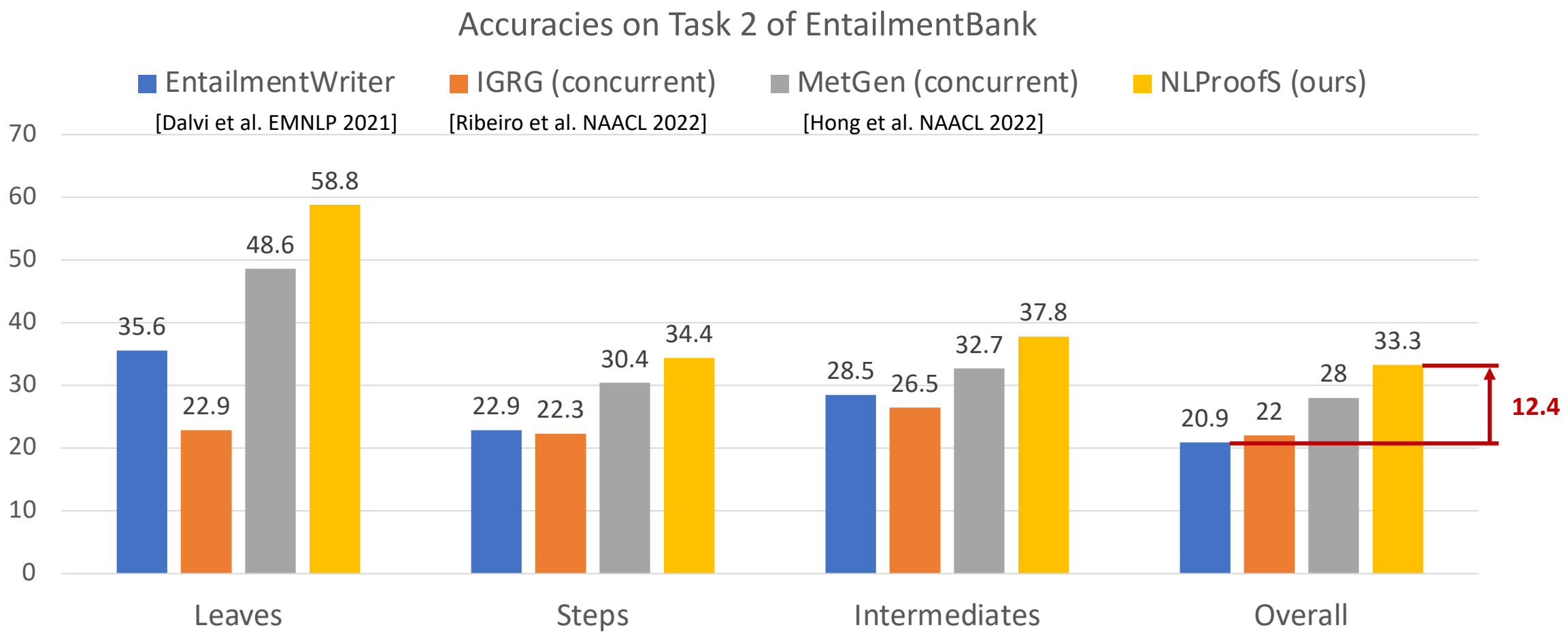
State-of-the-art Performance on EntailmentBank



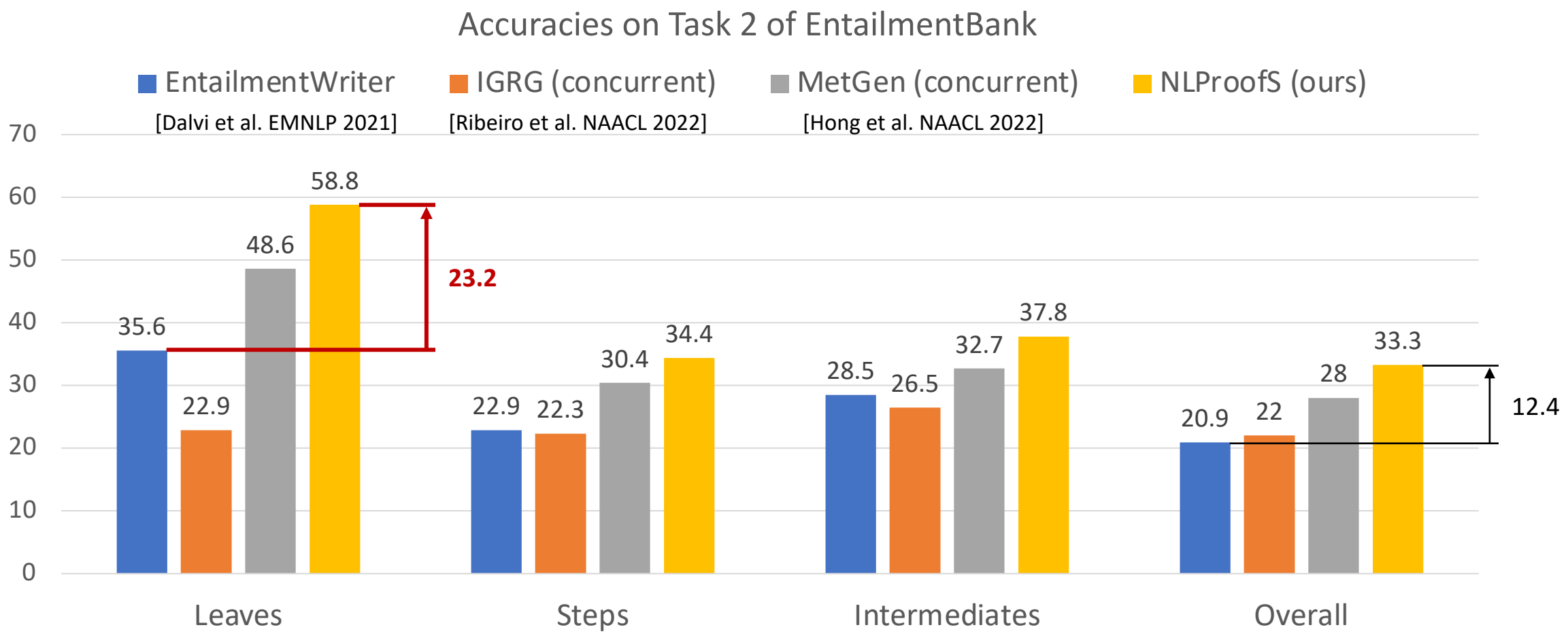
State-of-the-art Performance on EntailmentBank



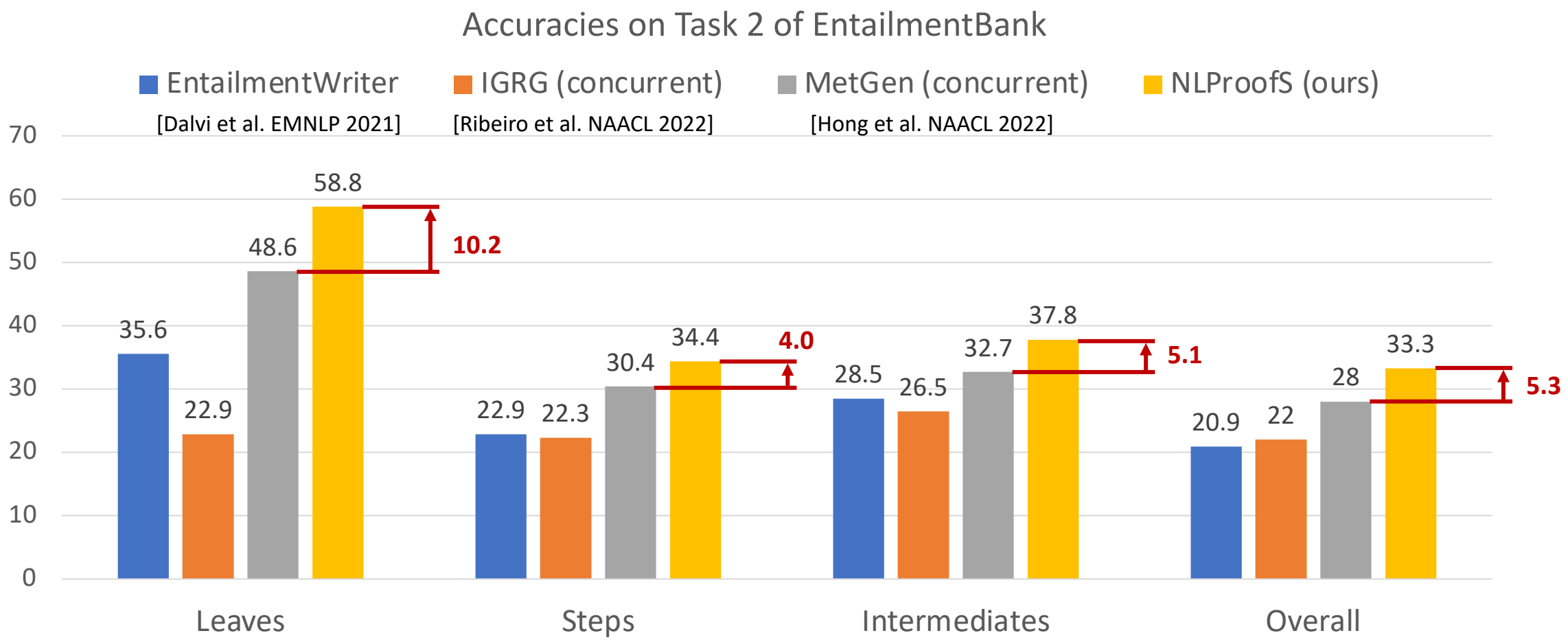
State-of-the-art Performance on EntailmentBank



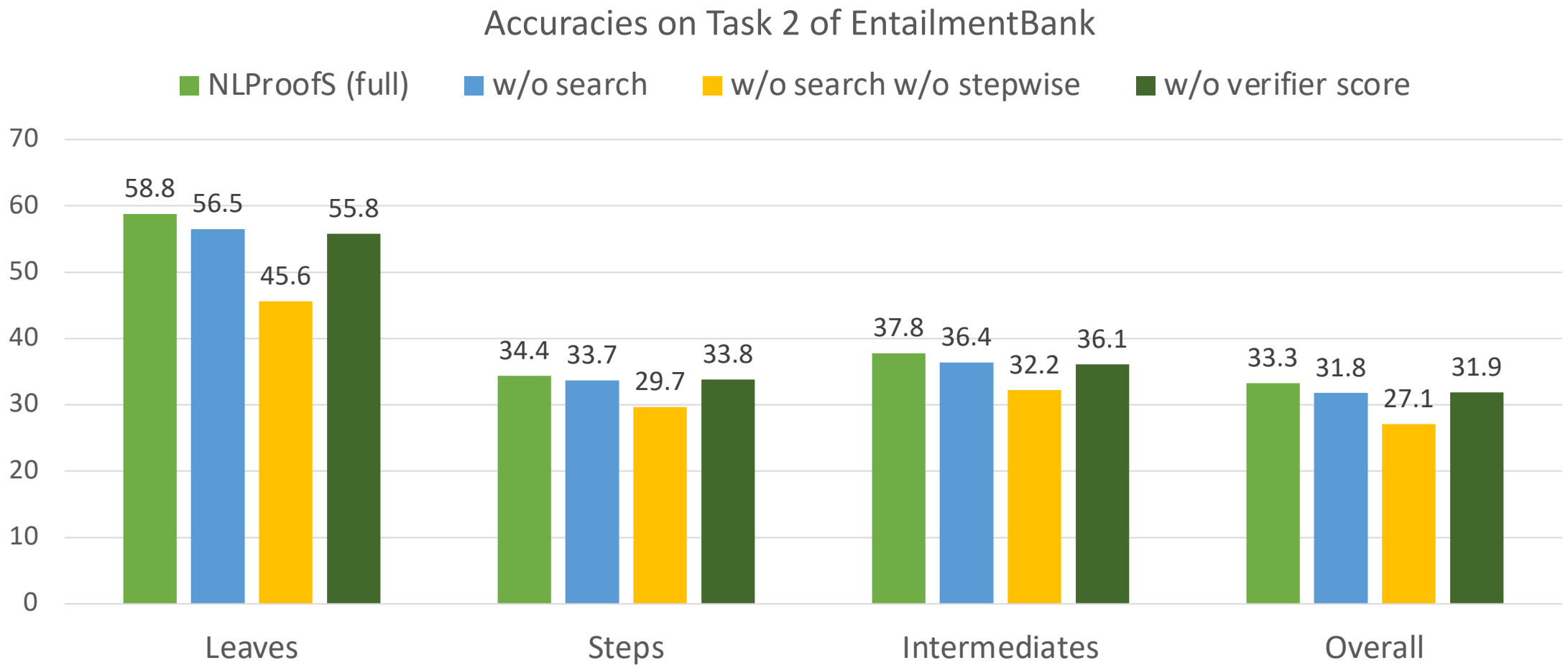
State-of-the-art Performance on EntailmentBank



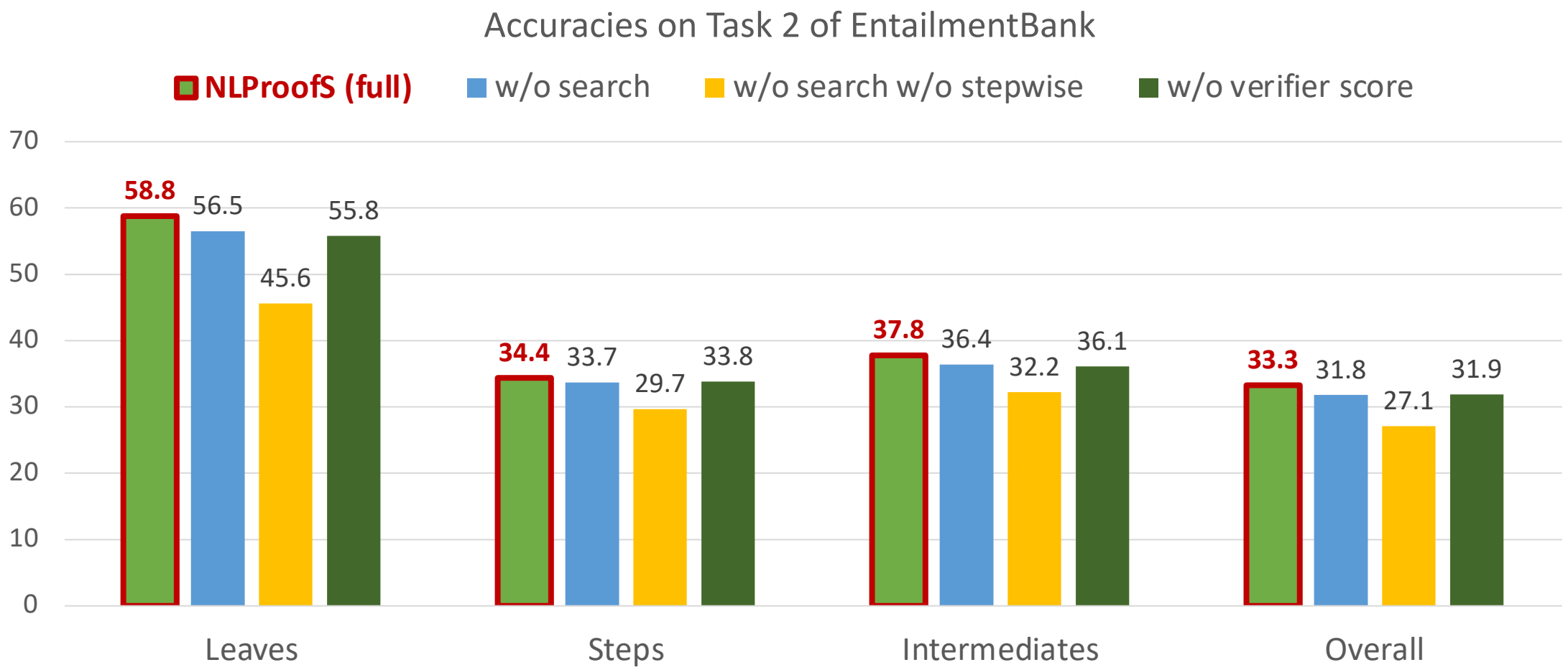
State-of-the-art Performance on EntailmentBank



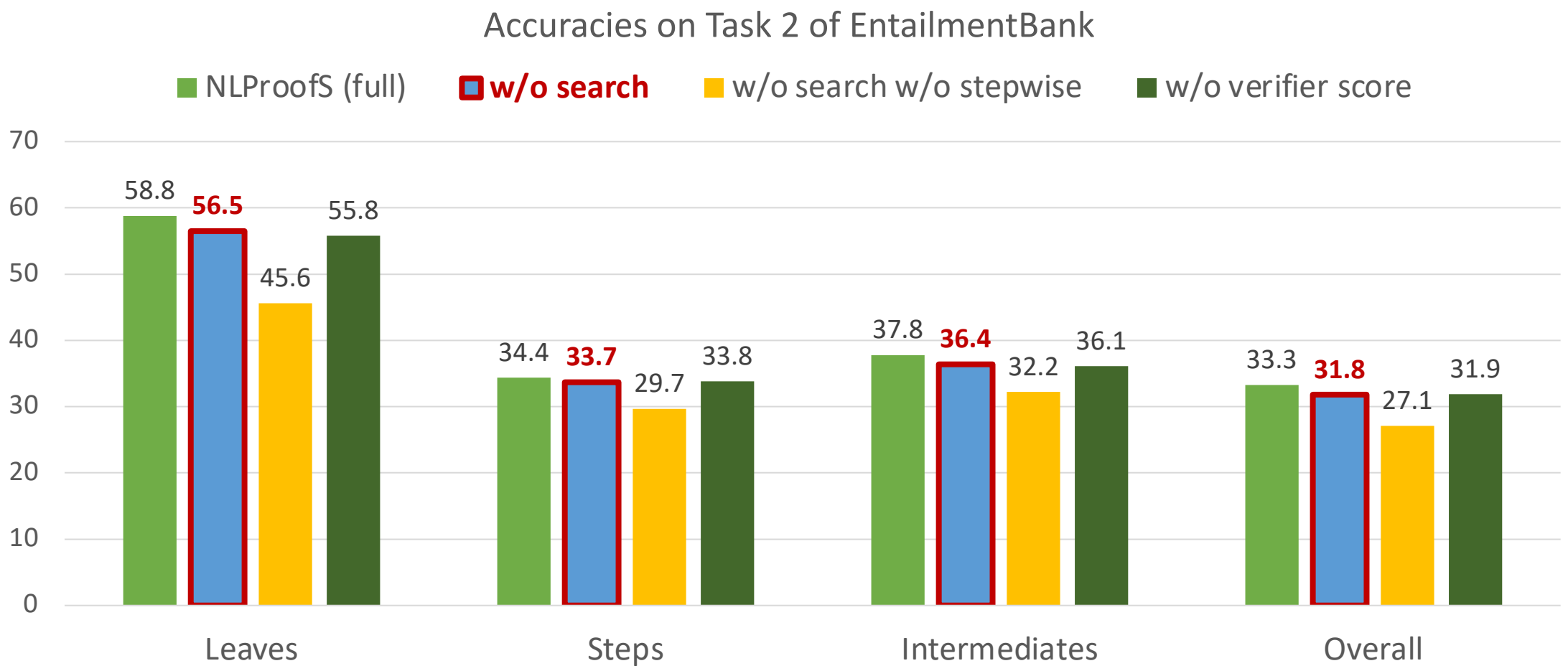
Ablations


Accuracies on Task 2 of EntailmentBank
NLProofS (full)
w/o search
w/o search w/o stepwise
w/o verifier score
70
60
50
40
30
20
10
0
58.8
56.5
45.6
55.8
34.4
33.7
29.7
33.8
37.8
36.4
32.2
36.1
33.3
31.8
27.1
31.9
Leaves
Steps
Intermediates
Overall

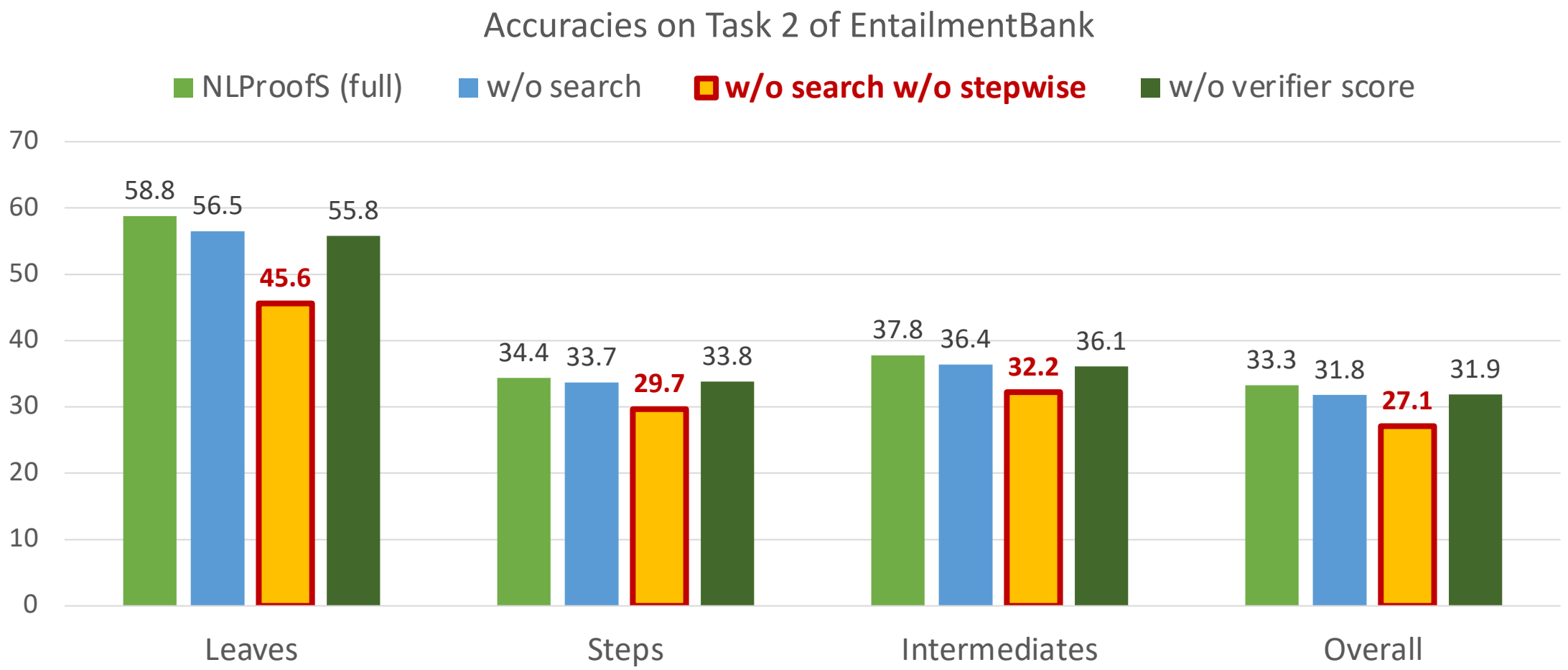
Ablations


Accuracies on Task 2 of EntailmentBank
NLProofS (full)
w/o search
w/o search w/o stepwise
w/o verifier score
70
60
50
40
30
20
10
0
58.8
56.5
45.6
55.8
34.4
33.7
29.7
33.8
37.8
36.4
32.2
36.1
33.3
31.8
27.1
31.9
Leaves
Steps
Intermediates
Overall

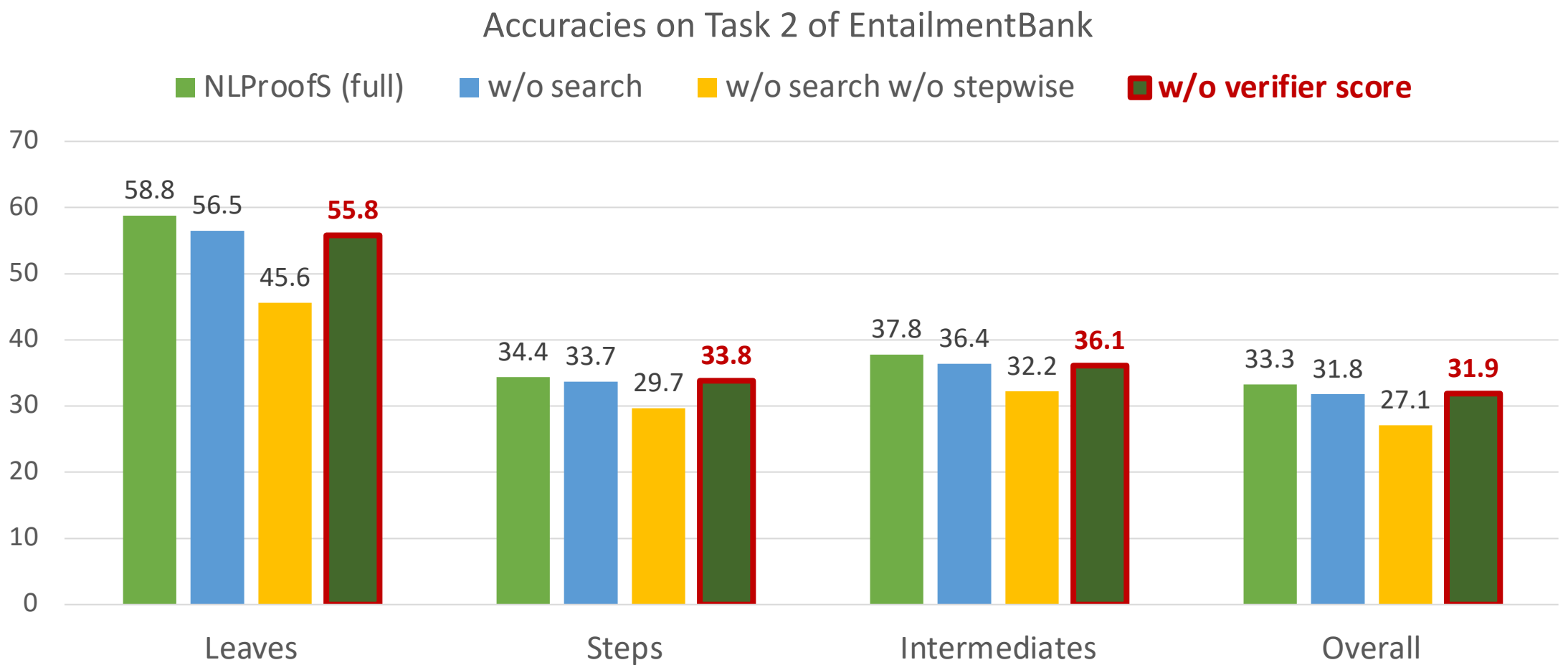
Verifier-Guided Proof Search Is Helpful



Stepwise Generation Is Helpful



The Verifier Is Necessary for Proof Search



Examples

Hypothesis	Premises	Conclusions generated by the model
The next new moon will occur on June 30.	1. A new moon is a kind of phase of the moon. 2. A moon phase occurs 28 days after the last time it occurs.	**EntailmentWriter**: The next new moon will occur 28 days after June 2.
		NLProofS (ours): The next new moon will occur 28 days after the last new moon.

Examples

Hypothesis	Premises	Conclusions generated by the model
The next new moon will occur on June 30.	1. A new moon is a kind of phase of the moon. 2. A moon phase occurs 28 days after the last time it occurs.	**EntailmentWriter**: The next new moon will occur 28 days after June 2.
		NLProofS (ours): The next new moon will occur 28 days after the last new moon.

Examples

Hypothesis	Premises	Conclusions generated by the model
The next new moon will occur on June 30.	**1. A new moon is a kind of phase of the moon. 2. A moon phase occurs 28 days after the last time it occurs.**	**EntailmentWriter**: The next new moon will occur 28 days after June 2.
		NLProofS (ours): The next new moon will occur 28 days after the last new moon.

Examples

Hypothesis	Premises	Conclusions generated by the model
The next new moon will occur on June 30.	1. A new moon is a kind of phase of the moon. 2. A moon phase occurs 28 days after the last time it occurs.	**EntailmentWriter**: The next new moon will occur 28 days after June 2.
		NLProofS (ours): The next new moon will occur 28 days after the last new moon.

Examples: Hallucination

Hypothesis	Premises	Conclusions generated by the model
The next new moon will occur on **June** 30.	1. A new moon is a kind of phase of the moon. 2. A moon phase occurs 28 days after the last time it occurs.	**EntailmentWriter**: The next new moon will occur 28 days after **June** 2.
		NLProofS (ours): The next new moon will occur 28 days after the last new moon.

Examples: Copying from Premises

Hypothesis	Premises	Conclusions generated by the model
The next new moon will occur on June 30.	1. A new moon is a kind of phase of the moon. 2. A moon phase occurs 28 days after the last time it occurs.	**EntailmentWriter**: The next new moon will occur 28 days after June 2.
		NLProofS (ours): The next new moon will occur 28 days after the last new moon.
Planting trees prevents soil from washing away.	1. **Planting trees increases the amount of trees in an environment.** 2. Tree roots decrease / reduce soil erosion.	**EntailmentWriter**: **Plants trees increases the amount of trees in an environment.**
		NLProofS (ours): Planting trees decreases soil erosion.

Conclusion and Future Work

Key Takeaways

- The verifier is important
  - **Prevent hallucinated generations**

Key Takeaways

- The verifier is important
  - Prevent hallucinated generations
  - **Also explored in other contexts, e.g., math word problems, code generation**

[Cobbe et al. "Training Verifiers to Solve Math Word Problems", arXiv 2021]
[Le and Wang et al. “CodeRL: Mastering Code Generation through Pretrained Models and Deep Reinforcement Learning” arXiv 2022]

Key Takeaways

- The verifier is important
  - Prevent hallucinated generations
  - Also explored in other contexts, e.g., math word problems, code generation

[Cobbe et al. "Training Verifiers to Solve Math Word Problems", arXiv 2021]
[Le and Wang et al. “CodeRL: Mastering Code Generation through Pretrained Models and Deep Reinforcement Learning” arXiv 2022]

  - **Mitigate the problem that language models do not “understand” their own generations**

[Austin and Odena et al. "Program Synthesis with Large Language Models"]

Limitations and Future Directions

Input

h: solar is a kind of renewable energy for heating homes

sent1: homes are buildings
sent2: solar is renewable
sent3: wind is a kind of energy
sent4: solar is a kind of energy
sent5: energy is used for heating buildings
sent6: coal is nonrenewable
…
…

Limitations and Future Directions

Input

h: solar is a kind of renewable energy for heating homes

sent1: homes are buildings
sent2: solar is renewable
sent3: wind is a kind of energy
sent4: solar is a kind of energy
sent5: energy is used for heating buildings
sent6: coal is nonrenewable
…
…

Concatenate

$hypothesis$ = solar is a kind of renewable energy for heating homes ;
$facts$ = sent1: homes are buildings sent2: solar is renewable … ;
$partial_proof$ = …

Limitations and Future Directions

Input

h: solar is a kind of renewable energy for heating homes

sent1: homes are buildings
sent2: solar is renewable
sent3: wind is a kind of energy
sent4: solar is a kind of energy
sent5: energy is used for heating buildings
sent6: coal is nonrenewable
…
…

Concatenate

$hypothesis$ = solar is a kind of renewable energy for heating homes ;
$facts$ = sent1: homes are buildings sent2: solar is renewable … ;
$partial_proof$ = …

- **Scale poorly w.r.t. the number of supporting facts**

Limitations and Future Directions

Input

h: solar is a kind of renewable energy for heating homes

sent1: homes are buildings
sent2: solar is renewable
sent3: wind is a kind of energy
sent4: solar is a kind of energy
sent5: energy is used for heating buildings
sent6: coal is nonrenewable
…
…

Concatenate

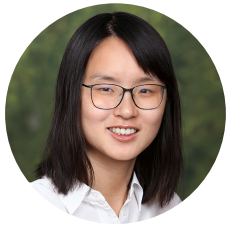
$hypothesis$ = solar is a kind of renewable energy for heating homes ;
$facts$ = sent1: homes are buildings sent2: solar is renewable … ;
$partial_proof$ = …

- Scale poorly w.r.t. the number of supporting facts
- Practically applications may have thousands or millions of supporting facts
- **Retrievers or memory-augmented transformers may help**

[Karpukhin et al. EMNLP 2020]
[Ribeiro et al. NAACL 2022]
[Wu et al. ICLR 2022]

Generating Natural Language Proofs with Verifier-Guided Search

Kaiyu Yang, Jia Deng, and Danqi Chen



PRINCETON UNIVERSITY